

Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y Eléctrica



Actividad Fundamental 3: PRODUCTO INTEGRADOR DE APRENDIZAJE

“AUTOMATIZACIÓN Y SOSTENIBILIDAD: UN SISTEMA DE RIEGO y COMPOSTAJE INTELIGENTE CON ESP32 IMPLEMENTADO EN LA ZONA METROPOLITANA DE NUEVO LEÓN (ECOSYNCRA)”

Por: Equipo 13

1. Valeria Abigail González Sada 1963075 - Grupo 010
2. Angie Sofía Hernández Tristán 2008178 – Grupo: 009
3. Jesús Adrián Moreno Contreras 1909558 – Grupo 010
4. Ingrid del Rosario García Sánchez 2109559 – Grupo 010

Asesores:

Jesús Daniel Garza Camarena

Fecha: 24 de Noviembre del 2025

1	COMPETENCIA ESPECÍFICA	5
2	APARTADOS DE LA AF1 ACTUALIZADOS	5
2.1	RESUMEN.....	5
2.2	INTRODUCCION.....	5
2.3	PLANTEAMIENTO DEL PROBLEMA	6
2.3.1	Problemática y fundamentación técnica	6
2.3.2	Causas identificadas.....	6
2.3.3	Evidencia relevante.....	6
2.4	OBJETIVOS	7
2.4.1	Objetivo General.....	7
2.4.2	Objetivos específicos	7
2.5	ALCANCE.....	8
2.6	Delimitaciones.....	8
2.7	JUSTIFICACIÓN TÉCNICA	8
2.7.1	Impactos esperados	9
2.8	METODOLOGÍA.....	9
2.8.1	Fase 1 – Análisis y Requerimientos.....	9
2.8.2	Fase 2 – Diseño del Sistema.....	9
2.8.3	Fase 3 – Integración y Prototipado	9
2.8.4	Fase 4 – Validación y Pruebas	10
2.8.5	Fase 5 – Conclusiones y optimización.....	10
2.9	DESCRIPCIÓN DE LOS SISTEMAS INVOLUCRADOS.....	10
2.9.1	Diagrama de bloques.....	11
2.9.2	Diagrama de flujo inicial.....	12
2.9.3	Diagrama de flujo actualizado.....	13

3	DISEÑO DEL PROYECTO	14
3.1	DISEÑO MECÁNICO	14
3.1.1	PARTE I.: Diseño inicial de EcoSyncra.....	14
3.1.2	B. DISEÑO ACTUALIZADO (FINAL)	20
3.2	ETAPA ELECTRÓNICA.....	35
3.2.1	Diagrama de bloques.....	36
3.2.2	Diagramas esquemáticos	37
3.2.3	Lista de componentes electrónicos involucrados	39
3.2.4	Simulación electrónica.....	42
3.2.5	Diseño de PCB.....	42
3.3	ETAPA DE CONTROL/ALGORITMICA	47
3.3.1	Pseucódigo	48
3.3.2	Simulaciones de control.....	48
3.3.3	ALGORITMO	53
4	RESULTADOS	57
4.1.1	PRUEBAS FINALES DE CONTROL.....	63
5	IMPLEMENTACIÓN	67
5.1.1	PROCESO DE ENSAMBLAJE FÍSICO.....	67
5.1.2	COSTO TOTAL DEL PROTOTIPO	77
5.1.3	VIDEO DEL PROTOTIPO FUNCIONAL.....	78
6	CONCLUSIONES	78
7	SUGERENCIAS FUTURAS.....	81
8	REVISIÓN BIBLIOGRÁFICA	82
9	ANEXOS	86
9.1	Fotografía del equipo completo.....	86

9.2	Fotografías de los integrantes individuales.....	86
9.3	Repartición de tareas del equipo	87
9.4	Códigos y pseudocódigos utilizados.....	88
9.4.1	Pseudocodigo.....	88
9.4.2	Pseudocodigo detallado composta.....	95
9.4.3	Pseudocodigo composta + plantas.....	96
9.4.4	Codigo del dashboard web de EcoSyncra.....	98
9.4.5	Código de la interfaz web.....	99

1 COMPETENCIA ESPECÍFICA

El estudiante será capaz de documentar y analizar de manera integral el desarrollo del proyecto mecatrónico, incluyendo los resultados obtenidos, el cumplimiento de los objetivos y las conclusiones técnicas derivadas del trabajo realizado, en un reporte final estructurado y actualizado.

2 APARTADOS DE LA AF1 ACTUALIZADOS

2.1 RESUMEN

EcoSyncra es un sistema mecatrónico automatizado diseñado para optimizar el riego y el compostaje doméstico en entornos urbanos, en respuesta a la crisis ambiental y la creciente escasez hídrica. El proyecto surge ante la necesidad de fomentar prácticas sostenibles que reduzcan el consumo de agua y mejoren la gestión de residuos orgánicos, contribuyendo a los Objetivos de Desarrollo Sostenible (ODS) 2 y 6, relacionados con hambre cero y agua limpia y saneamiento. Su objetivo general es desarrollar un prototipo funcional que integre sensores de humedad, temperatura, radiación solar y gases conectados a un microcontrolador ESP32, el cual regula automáticamente el riego y la aireación del compost mediante algoritmos de control lógico. El sistema incluye un dashboard web que permiten el monitoreo remoto en tiempo real. La metodología propuesta abarca el diseño electrónico y mecánico, la programación embebida, la simulación en software especializado (Proteus, SolidWorks, Easyeda, kicad, visualstudiocode,) y la validación experimental del prototipo. Se espera como resultado un dispositivo energéticamente eficiente, escalable y de bajo costo, capaz de reducir el desperdicio de agua, acelerar el proceso de compostaje y promover hábitos sostenibles en entornos urbanos, fortaleciendo así la conciencia ambiental y el bienestar comunitario.

2.2 INTRODUCCIÓN

EcoSyncra – “Cuidando tus cultivos, optimizando el agua y protegiendo al planeta” es un proyecto mecatrónico que busca automatizar el riego y el compostaje doméstico mediante sensores, control lógico y visualización inteligente, demostrando cómo la tecnología puede contribuir a la sostenibilidad urbana desde el hogar. Entre sus objetivos se incluyen: diseñar una arquitectura modular con sensores para monitoreo en tiempo real, implementar

algoritmos de control que regulen riego y compostaje, construir un prototipo funcional con hardware, software y visualización, desarrollar una interfaz digital para monitoreo remoto y validar el desempeño técnico. Además, se enfoca en la alineación con los ODS 2 y 6, promover prácticas agrícolas sostenibles, priorizar eficiencia, escalabilidad y sostenibilidad, y comunicar el proyecto con un enfoque técnico, visual y social.

2.3 PLANTEAMIENTO DEL PROBLEMA

La mecatrónica, al integrar mecánica, electrónica, informática y control, se consolida como una disciplina clave para enfrentar los desafíos del siglo XXI, especialmente en procesos más eficientes y sostenibles. En la agricultura urbana y la jardinería doméstica, los métodos tradicionales —como riego manual y compostaje sin monitoreo— generan desperdicio de recursos y dificultan la adopción de prácticas sostenibles. EcoSyncra surge como una respuesta a esta necesidad, integrando sensores ambientales, un microcontrolador ESP32 y actuadores para regular riego y compostaje de forma autónoma, combinando percepción, decisión y acción y demostrando que la mecatrónica puede generar soluciones accesibles, escalables y alineadas con los objetivos de desarrollo sostenible.

2.3.1 Problemática y fundamentación técnica

En contextos urbanos y suburbanos, existe desconexión con prácticas agrícolas sostenibles, reflejada en escasa vegetación, desperdicio de recursos y pérdida de bienestar social. Dos factores técnicos que agravan la situación son el riego ineficiente y la gestión deficiente de residuos orgánicos.

2.3.2 Causas identificadas

La falta de automatización impide que los sistemas respondan a las condiciones reales; la ausencia de monitoreo genera decisiones intuitivas sin datos precisos; y las barreras de acceso hacen que las soluciones actuales sean costosas o complejas.

2.3.3 Evidencia relevante

Más del 40% de la población mundial enfrenta escasez de agua, y el sector agrícola desperdicia gran parte del consumo. En ciudades como Monterrey, la cobertura de áreas verdes está por debajo del estándar de 16 m² por habitante recomendado por la OMS. Más del 50% de los residuos urbanos en México son orgánicos, generando emisiones de metano

y desaprovechando su potencial como compost. La falta de tiempo y desconocimiento son los principales obstáculos para tener plantas en casa.

2.4 OBJETIVOS

2.4.1 Objetivo General

Se busca el diseño y la implementación de un sistema automatizado de autorriego y compostaje doméstico, el cual está basado en sensores y controladores digitales, que ayude a facilitar el cuidado de las plantas en entornos urbanos, optimizando el uso del agua y residuos orgánicos y contribuya al bienestar ambiental, social y personal del usuario.

2.4.2 Objetivos específicos

El proyecto EcoSyncra concluyó exitosamente tras un proceso integral que permitió establecer la lógica de operación del sistema mediante diagramas de flujo que articularon la lectura de sensores y la activación de actuadores. La metodología de desarrollo se ejecutó de manera completa: desde el análisis de requerimientos hasta el diseño mecánico y electrónico, la integración de prototipos y las pruebas de validación. Cada etapa fue documentada y optimizada, asegurando que el sistema alcanzara un desempeño confiable y eficiente en escenarios reales.

La arquitectura del sistema se organizó en bloques funcionales —monitoreo, control y procesamiento, actuación, interfaz y comunicación— lo que facilitó la comprensión y la implementación modular. El diseño electrónico incluyó la disposición estratégica de circuitos y placas PCB, maximizando el uso del espacio y los recursos disponibles. La selección de componentes y herramientas se realizó bajo criterios de sostenibilidad y funcionalidad, consolidando un sistema robusto, escalable y adaptable a distintos entornos de aplicación.

Finalmente, el diseño mecánico garantizó la correcta ubicación de sensores, actuadores y placas, logrando un ensamble accesible y confiable. La exposición del proyecto permitió mostrar cómo EcoSyncra integra mecánica, electrónica y software en una solución innovadora que combina sostenibilidad con tecnología inteligente. Los resultados evidenciaron su potencial para transformar procesos de automatización y optimizar el aprovechamiento responsable de recursos, posicionando al sistema como una propuesta sólida y visionaria dentro del ámbito biomédico y tecnológico.

2.5 *ALCANCE*

Cabe mencionar que el proyecto se enfoca en el diseño e implementación de un sistema mecatrónico automatizado para el riego y compostaje doméstico, utilizando sensores, algoritmos de control y una interfaz digital para monitoreo en tiempo real. Su propósito se fundamenta en demostrar, mediante un prototipo funcional a escala doméstica, cómo los conocimientos adquiridos pueden contribuir a la sustentabilidad urbana a través del uso eficiente del agua y el aprovechamiento de residuos orgánicos. Este sistema está orientado a hogares en zonas metropolitanas con espacios reducidos, buscando ser replicable y accesible. Incluye sensores de humedad, temperatura, luz y gases conectados a un microcontrolador ESP32, junto con actuadores como bombas, electroválvulas, ventiladores y una interfaz digital (interfaz web y dashboard web).

2.6 *Delimitaciones*

Se plantea como un sistema funcional y sencillo de replicar, pero sin la intención de competir con los sistemas industriales de riego o compostaje que operan a gran escala. En este mismo sentido, tampoco se busca producir ni comercializar el prototipo en esta primera etapa, ya que el enfoque principal es académico y demostrativo. Otra limitación importante se encuentra en el tipo de entradas y datos que el sistema puede manejar. Si bien se integrarán sensores de humedad, temperatura, luz y gases que permiten un monitoreo básico del entorno, la incorporación de un sistema meteorológico más avanzado queda fuera del alcance del proyecto actual. De la misma forma, aunque se plantea el uso de bases de datos sencillas para almacenar información en la nube, no se contempla el desarrollo de infraestructuras complejas ni algoritmos predictivos, los cuales podrían considerarse como mejoras futuras.

2.7 *JUSTIFICACIÓN TÉCNICA*

El uso ineficiente del agua y la gestión deficiente de residuos orgánicos son desafíos críticos en contextos agrícolas, con impactos ambientales, económicos y sociales. El riego mal programado desperdicia recursos y afecta cultivos, mientras que el compostaje tradicional, sin monitoreo técnico, limita su efectividad y sostenibilidad.

EcoSyncra propone una solución mecatrónica integral que automatiza el riego y el compostaje doméstico mediante sensores ambientales, control lógico y visualización remota. Utiliza el microcontrolador ESP32 como núcleo de procesamiento, integrando electrónica

aplicada, programación embebida, diseño mecánico e interfaz digital para toma de decisiones en tiempo real.

Su diseño modular y de más accesibilidad lo hace replicable en hogares urbanos, huertos escolares y comunidades rurales, contribuyendo directamente a los ODS 2 y 6. Desde una perspectiva también académica, permite aplicar conocimientos en modelado físico-biológico, simulación, pensamiento sistémico e IoT y abre un gran camino en futuras mejoras como algoritmos predictivos y conexión con APIs meteorológicas.

2.7.1 Impactos esperados

Una de ellas es la optimización del uso del agua y los residuos orgánicos, también el fomento de las prácticas agrícolas sostenibles en entornos urbanos, agregando la mejora del bienestar ambiental, social y personal y a la vez muy importante una plataforma escalable para mayor innovación técnica y educativa.

2.8 METODOLOGÍA

2.8.1 Fase 1 – Análisis y Requerimientos

En esta primera fase está la identificación del problema como el uso de agua y el compostaje ineficiente, delimitación del entorno de uso como el hogar, huerto urbano y prototipo modular, además de la definición de sensores, actuadores, comunicación y plataforma de monitoreo y la selección del microcontrolador ESP32.

2.8.2 Fase 2 – Diseño del Sistema

En esta segunda fase de diseño del sistema se involucran factores indispensables de diseño como la arquitectura electrónica en el caso de la función sensor-controlador-actuador, también el diseño mecánico y la distribución física, la lógica de control en base a los umbrales y condiciones, el boceto del software embebido y el diseño del dashboard / interfaz web

2.8.3 Fase 3 – Integración y Prototipado

Aquí en esta etapa se revisa la conexión de los sensores como la humedad, temperatura/HR, compostaje, la conexión de los actuadores como el caso de las electroválvulas, bombas, ventilación o compuerta de la composta, también la programación inicial del ESP32, las pruebas requeridas de comunicación mediante bluetooth/puertoserial

2.8.4 Fase 4 – Validación y Pruebas

En esta siguiente fase nos basamos en la calibración de los sensores, la simulación de los escenarios de riego, las pruebas del contenedor de compostaje y los fallos y/o ajustes necesarios junto a documentación.

2.8.5 Fase 5 – Conclusiones y optimización

En esta última fase se fundamenta en la evaluación de desempeño, el análisis del consumo de agua, y la mejora del diseño como el software, electrónica e interfaz.

2.9 DESCRIPCIÓN DE LOS SISTEMAS INVOLUCRADOS

Se divide en cuatro bloques:

- A) Sistema de Monitoreo:** Los componentes que lo conforman son el sensor de humedad del suelo, el sensor DHT11/DHT22, el sensor de temperatura del compostaje y el sensor de gas (MQ135) y el sensor LDR (luz) Cumplen con la función de Adquirir datos para la toma de decisiones.
- B) Sistema de Control y Procesamiento:** Funciona en base al microcontrolador ESP32 que se encarga de apoyar a procesar las lecturas en tiempo real, la conexión vía Bluetooth/puerto serial, la ejecución de la lógica condicional y el envío de datos al dashboard/interfaz. Por otra parte, tenemos el software usando lenguaje micropython y usando Visual Studio Code con la extensión node.js una estructura modular por funciones, y umbrales preconfigurados y ajustables.
- C) Sistema de Actuación:** Donde se concentra en los actuadores (salidas) como la electroválvula y/o bomba de agua de 5V, el motor o aireación de composta y MOSFET driver. Cumplen con la función de Ejecutar acciones físicas según las condiciones detectadas
- D) Sistema de Interfaz y Comunicación:** Se fundamenta en esa interfaz y comunicación que involucra el dashboard web, el canal serial y/o bluetooth y las notificaciones/visualización del estado

2.9.1 Diagrama de bloques

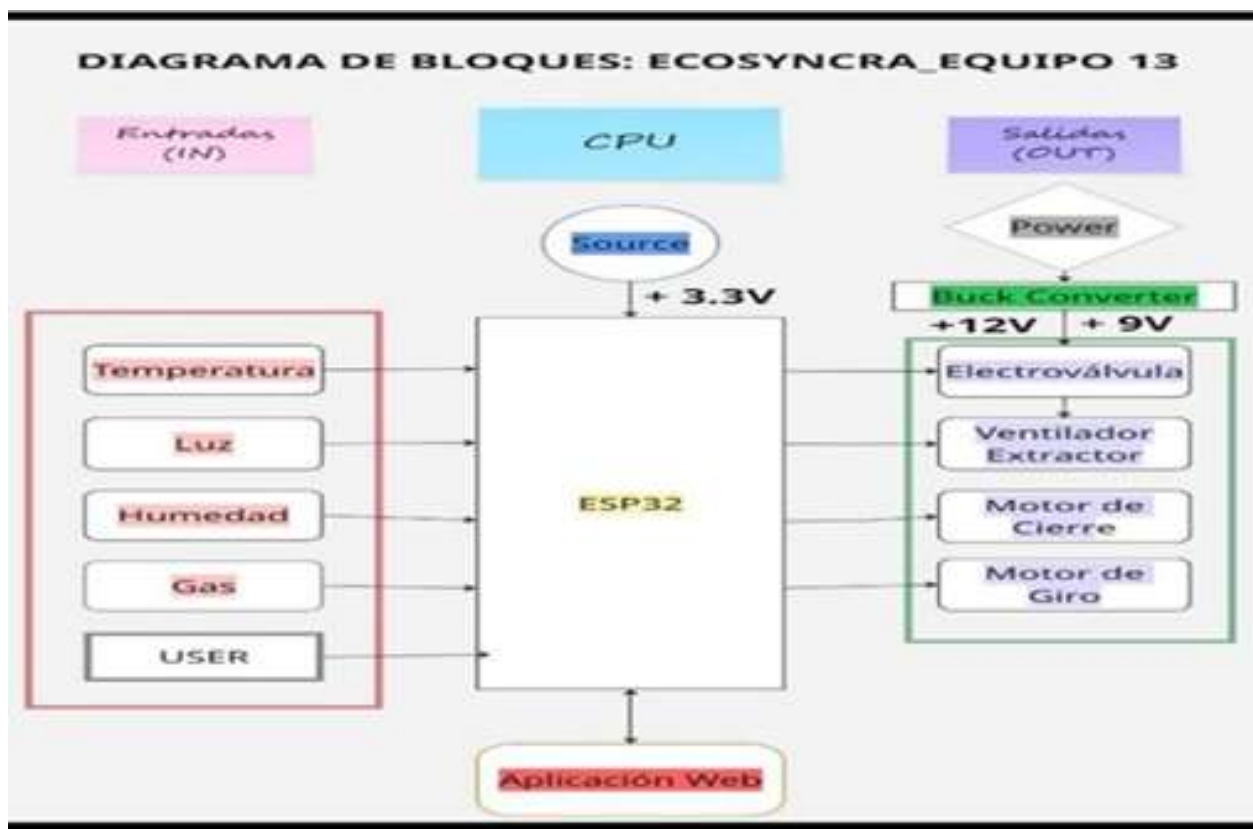


Ilustración 1. DIAGRAMA BLOQUES ECOSYNCR_A

Link: https://miro.com/app/board/uXjVJNWqHwg=/?share_link_id=701817533570

Explicación del diagrama de bloques: El diagrama de bloques elaborado en miro del sistema mecatrónico EcoSyncra utiliza un microcontrolador ESP32 para procesar datos de sensores ambientales (temperatura, luz, humedad y gases) y tomar decisiones automáticas. Controla actuadores como electroválvula, ventilador y motores, alimentados por un buck converter. Una aplicación web permite el monitoreo remoto. El sistema optimiza el riego y compostaje doméstico, contribuyendo a los ODS 2 y 6 mediante una gestión eficiente del agua y los residuos orgánicos.

2.9.3 Diagrama de flujo actualizado



Ilustración 3. DIAGRAMA FLUJO 2 ACTUAL

Link del diagrama de flujo:

<https://drive.google.com/file/d/1xJKB->

[KDwWe38lNqunbjeqHHfrR-HXTk/view?usp=sharing](https://www.kdwe381nqunbjeqHHfrR-HXTk/view?usp=sharing).

El flujo arranca preparando el sistema, lo que implica la configuración inicial del hardware y la activación de los sensores necesarios para que todo funcione correctamente. Una vez completada esta etapa, se despliega un menú interactivo que permite al usuario elegir entre tres alternativas principales, cada una orientada a diferentes acciones dentro del proceso. La primera opción es básicamente para configurar el sistema. Aquí puedes entrar a un submenú donde ajustas parámetros y haces pruebas para asegurarte de que todo esté correcto. La segunda opción es más práctica: te deja ver y ejecutar acciones directamente, como si fuera un modo manual para controlar el sistema.

La tercera opción es la más completa porque activa un proceso automático. Primero tienes que seleccionar la planta que se va a controlar; si no hay ninguna registrada, el sistema te manda

un mensaje de error y regresa al menú. Si sí hay planta, empieza un ciclo donde se revisan varias condiciones. Se checa que los parámetros estén bien; si no, el sistema vuelve a esperar ajustes. Si todo está correcto, analiza la humedad: si está baja, prende las bombas. Luego revisa la luz; si falta, enciende las lámparas. Después verifica la temperatura y, si está en el rango adecuado, activa el ventilador. También pregunta si ya pasó la hora programada para el riego; si sí, lo ejecuta.

Cuando termina todo eso, el sistema manda los datos, imprime un reporte y pregunta si hay más instrucciones. Si el usuario dice que sí, el ciclo se repite; si no, se guarda la configuración y el proceso termina. En resumen, este flujo mezcla configuración manual con control automático para mantener las condiciones ideales, activando bombas, luces, ventiladores y

riego cuando hace falta, y además guarda y transmite la información para tener todo bajo control.

3 DISEÑO DEL PROYECTO

DESCRIPCIÓN

3.1 DISEÑO MECÁNICO

Cabe resaltar que la etapa mecánica constituye el primer paso en la implementación física del sistema de riego y compostaje inteligente, ya que ella se realiza el ensamblaje y la disposición estructural de los componentes que darán soporte al prototipo. Esta fase asegura que los sensores, actuadores y elementos de control cuenten con una base sólida y organizada para su funcionamiento, permitiendo la correcta integración con la parte electrónica y algorítmica. El diseño mecánico no solo garantiza estabilidad y durabilidad, sino que también facilita la distribución eficiente de los subsistemas, como el depósito de agua, la bomba de riego y el contenedor de composta, crenado un entorno ordenado y funcional que respalda la operación automatizada del sistema en condiciones reales.

De esta forma, el diseño correspondiente a EcoSyncra su diseño mecánico corresponde a una carcasa estructural tipo caja desarrollada mediante CAD 3D (Solidworks), destinada a alojar y proteger los componentes que harán el funcionamiento de nuestro sistema de riego y compostaje inteligente. Su geometría rectangular incluye paneles con cortes específicos para botones, pantallas y conectores, lo que facilita la interacción del usuario y el monitoreo del sistema. La estructura incorpora soportes internos y ranuras para el montaje seguro de sensores, actuadores y el microcontrolador ESP32, asegurando estabilidad, orden y accesibilidad durante la operación. Este diseño permite una integración eficiente entre la parte física y la electrónica favoreciendo funcionalidad del prototipo en condiciones reales

3.1.1 PARTE I.: Diseño inicial de EcoSyncra

Los planos técnicos presentados a continuación, corresponden al diseño preliminar de la caja estructural destinada a alojar los componentes electrónicos del sistema EcoSyncra. Este conjunto de vistas —frontal, lateral, superior e isométrica— detalla las dimensiones generales, posiciones de perforaciones y cortes funcionales, necesarios para el montaje de

sensores, conectores y elementos de control. Aunque no representa el diseño final del sistema completo, esta caja cumple una función clave como envolvente protectora y soporte físico, permitiendo una distribución ordenada y accesible de los elementos internos durante la fase de implementación mecánica.

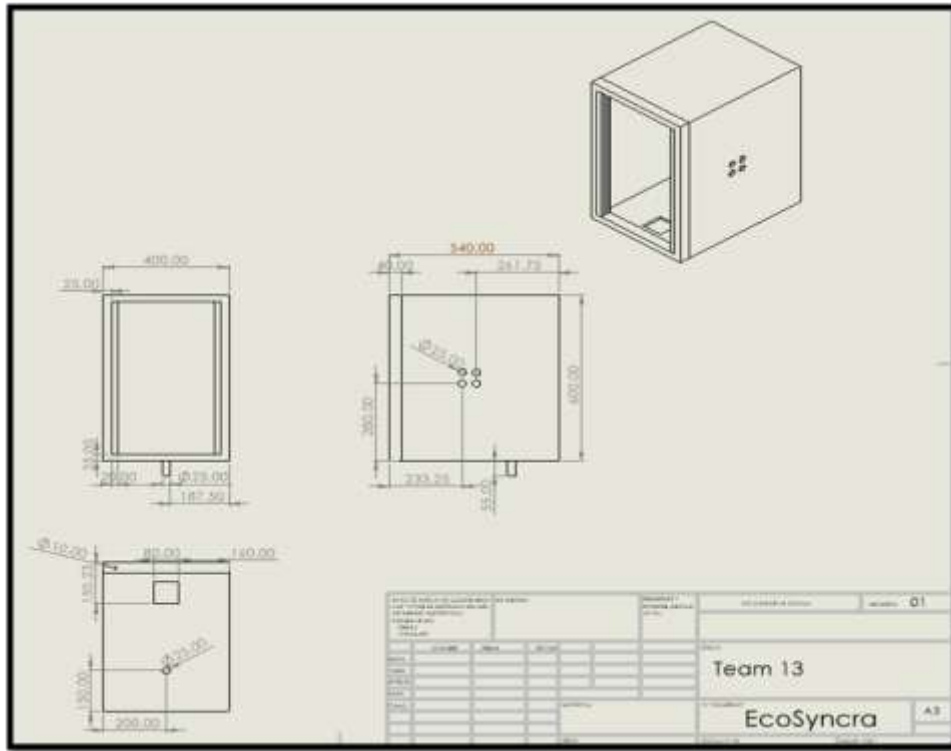
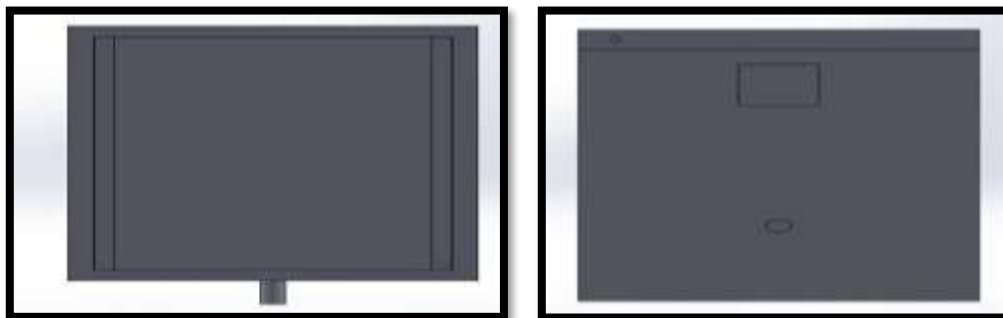


Ilustración 4. PLANOS TÉCNICOS CAJA PRINCIPAL INICIALES

A)Vistas de maceta iniciales



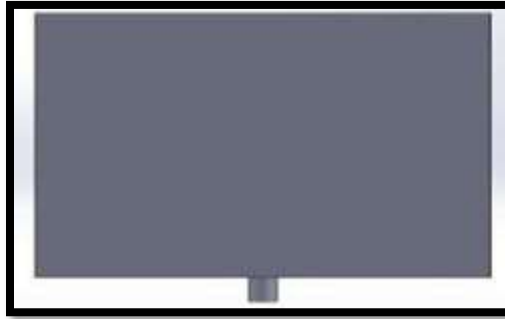


Ilustración 5. Vistas superior, inferior y lateral de ensamble inicial de maceta

a) Ensamblaje iniciales

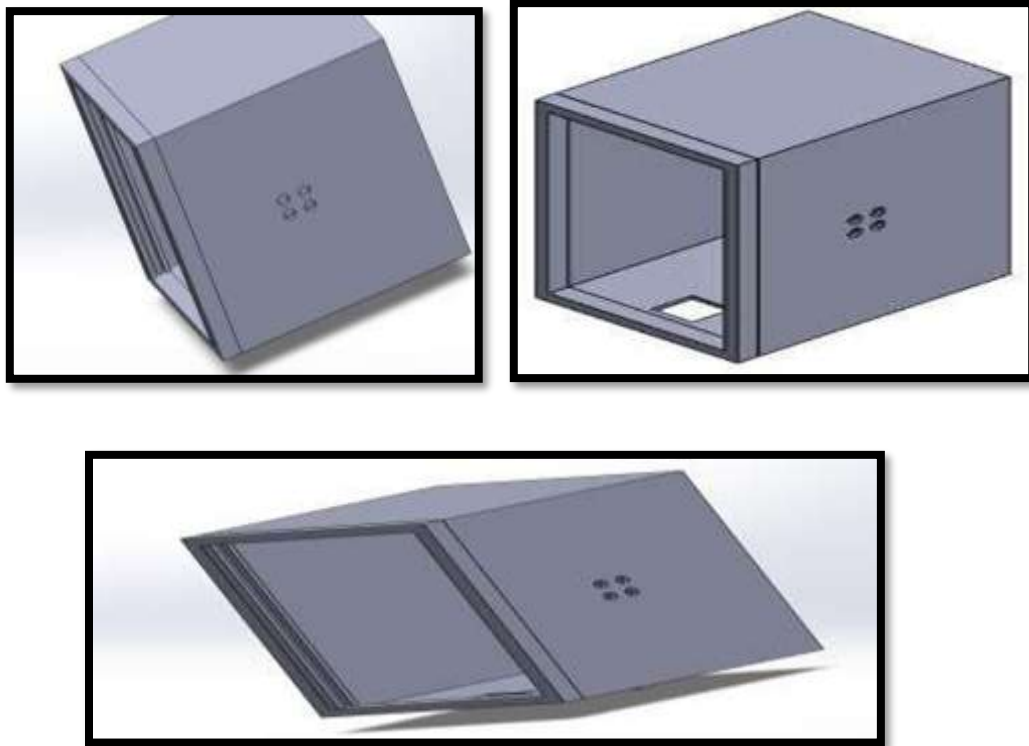


Ilustración 6. Ensamblaje inicial de maceta (vista isométrica)

a) Simulaciones implementadas

A continuación, se muestran las diversas simulaciones que se realizaron en la base de nuestro sistema (maceta) iniciales.

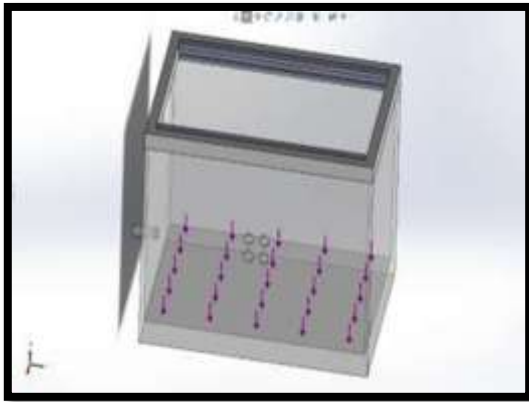


Ilustración 7. Cálculo de Von Mises en maceta (isometrica)

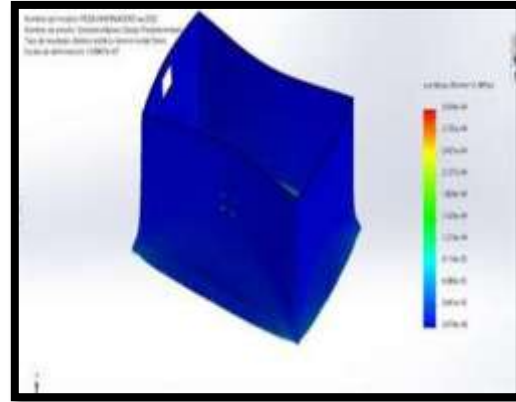


Ilustración 8. Aplicación de carga en maceta

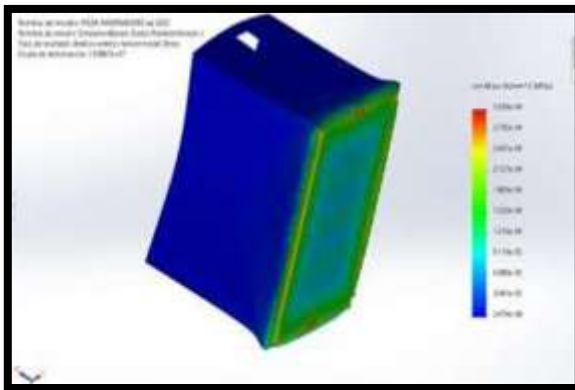


Ilustración 9. Calculo con mises maceta vista inferior

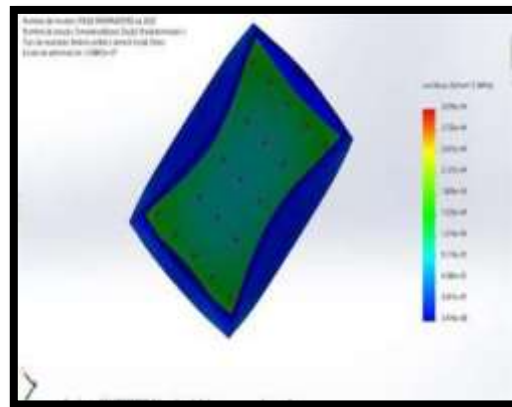


Ilustración 10. Calculo con mises maceta vista superior

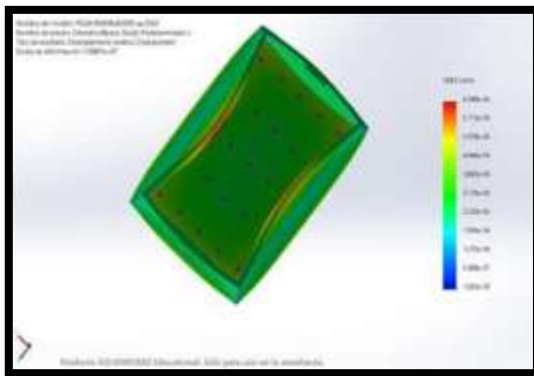


Ilustración 11. Simulación desplazamiento estatico, vista superior

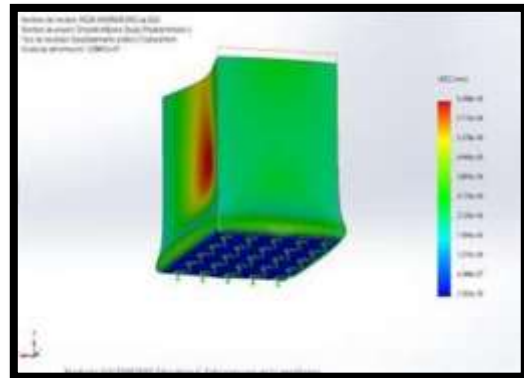


Ilustración 12. Simulación desplazamiento estatico, vista inferior

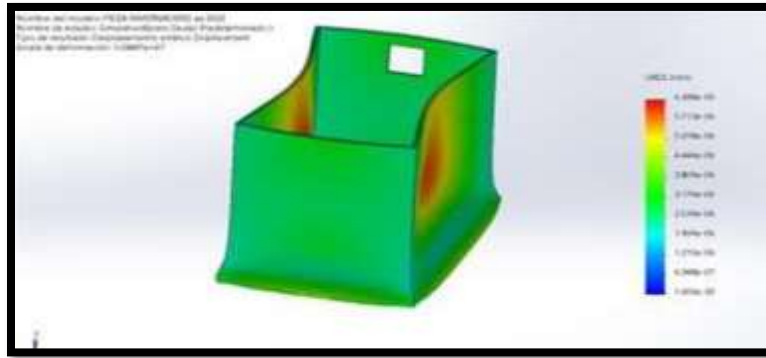


Ilustración 13. Simulación del desplazamiento estático, vista isométrica

Durante las simulaciones realizadas en SolidWorks, se analizaron distintos comportamientos estructurales de la base y paredes de la maceta del sistema EcoSyncra, con el objetivo de comprobar si el diseño mecánico era capaz de soportar el peso del sustrato, la tierra o el compostaje que se colocará en su interior. A continuación, se explican los resultados obtenidos y su interpretación. En primer lugar, la simulación de aplicación de carga permitió observar cómo se distribuye el peso sobre el fondo de la maceta. Al aplicar una presión equivalente al peso de la tierra, se notó que la carga se concentra principalmente en el centro del fondo, lo cual es totalmente Lógico, ya que es la parte que recibe directamente la fuerza de gravedad del material. Este paso fue muy útil para verificar que el modelo estuviera bien definido en sus restricciones (la base fija que simula el apoyo en el suelo) y que las condiciones aplicadas representaran una situación real de uso.

El análisis de desplazamiento estático evidenció una leve flexión en el fondo de la maceta bajo carga simulada, indicando un comportamiento elástico del material. Aunque se observa un hundimiento temporal en el centro, este desaparece al retirar el peso, lo que confirma que no hay deformaciones permanentes. El resultado demuestra que el diseño mantiene un equilibrio funcional entre rigidez y flexibilidad, permitiendo cierto desplazamiento sin comprometer la integridad estructural.

Durante las simulaciones realizadas en SolidWorks, se analizaron distintos comportamientos estructurales de la base y paredes de la maceta del sistema EcoSyncra, con el objetivo de comprobar si el diseño mecánico era capaz de soportar el peso del sustrato, la tierra o el compostaje que se colocará en su interior. A continuación, se explican los resultados obtenidos y su interpretación. En primer lugar, la simulación de aplicación de carga permitió

observar cómo se distribuye el peso sobre el fondo de la maceta. Al aplicar una presión equivalente al peso de la tierra, se notó que la carga se concentra principalmente en el centro del fondo, lo cual es totalmente lógico, ya que es la parte que recibe directamente la fuerza de gravedad del material. Este paso fue muy útil para verificar que el modelo estuviera bien definido en sus restricciones (la base fija que simula el apoyo en el suelo) y que las condiciones aplicadas representaran una situación real de uso.

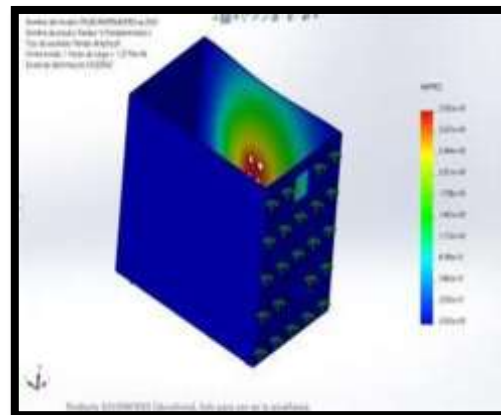
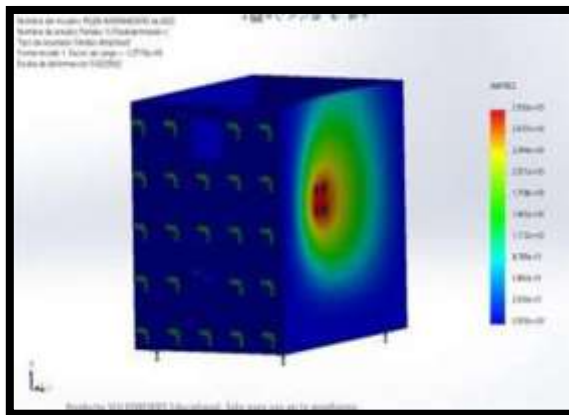


Ilustración 14. Simulación de pandeo, vista isométrica Ilustración 15. Simulación de pandeo, vista lateral

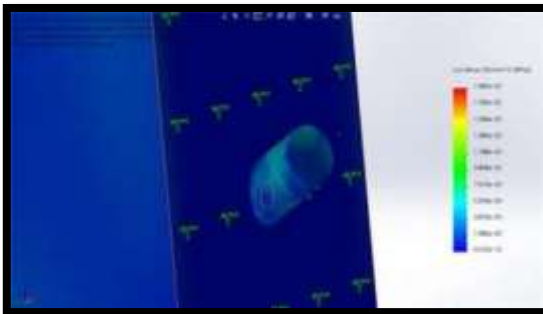
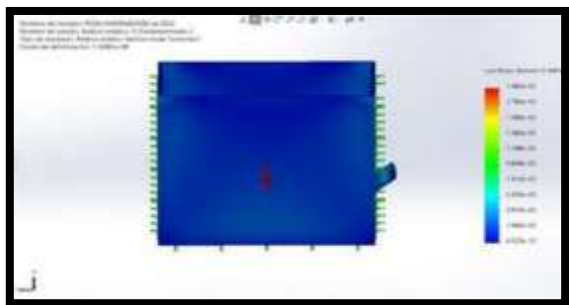


Ilustración 16. Simulación de pandeo vista lateral Ilustración 17. Simulación de pandeo vista lateral (cercanía)

Las simulaciones mecánicas realizadas sobre la maceta del sistema EcoSyncra confirman que el diseño resiste adecuadamente las cargas previstas en condiciones normales de uso. El análisis de pandeo reveló que la zona más vulnerable es el centro del fondo, donde podría ocurrir una deformación si se supera el peso estimado (más de 20 kg de tierra húmeda o composta). Sin embargo, el comportamiento estructural se mantiene estable y el material responde de forma elástica, sin deformaciones permanentes. Se recomienda reforzar esta área

con nervaduras o mayor espesor para aumentar la capacidad de carga. En conjunto, los resultados validan que la maceta es segura, funcional y compatible con la automatización del riego y compostaje.

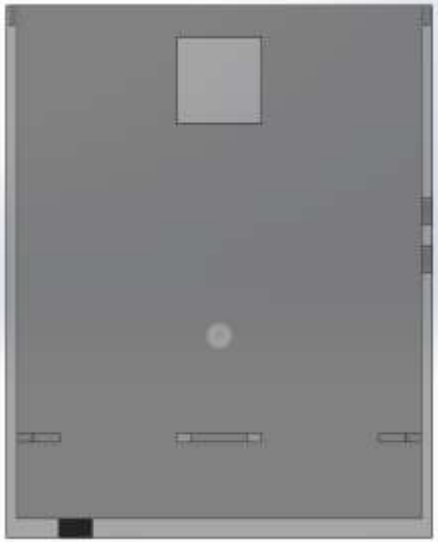
3.1.2 B. DISEÑO ACTUALIZADO (FINAL)

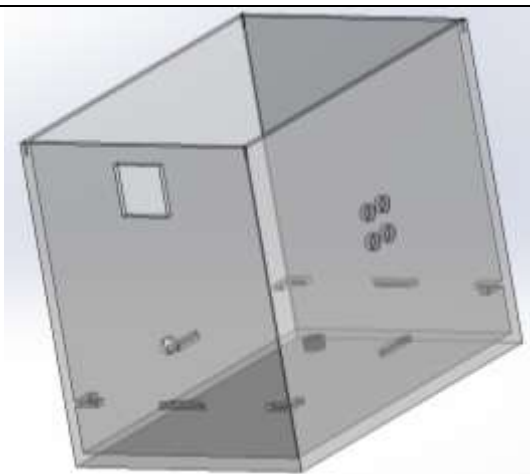
Como parte del proceso de refinamiento del diseño mecánico de la maceta en el sistema EcoSyncra, se realizaron ajustes estructurales orientados a mejorar su funcionalidad e integración con los componentes electrónicos y de control. Estas modificaciones responden a la necesidad de asegurar una interacción eficiente entre el hardware físico y el sistema automatizado, permitiendo una disposición más ordenada, segura y operativa de los elementos internos. Gracias a estas mejoras, el diseño evoluciona hacia una versión más robusta y compatible con los requerimientos del control algorítmico, facilitando la implementación del riego y compostaje inteligente en condiciones reales.

Donde tenemos las siguientes piezas que se fueron creando una por una hasta lograr ensamblar todo en un mismo ensamble para lograr la funcionalidad esperada

➤ PARTES QUE COMPONEN EL DISEÑO MECÁNICO

1. **Marco estructural principal:** Estructura rígida que define el volumen del sistema y sirve como soporte para los demás componentes.
 2. **Soporte del marco:** Refuerzos internos o externos que estabilizan el marco y permiten su fijación segura al entorno.
 3. **Cortina o cubierta móvil:** Elemento de protección y regulación ambiental, que puede actuar como sistema de sombreado o aislamiento.
 4. **Maceta integrada:** Recipiente diseñado para contener el sustrato y la planta, optimizado para soportar peso y facilitar el drenaje.
 5. **Case o carcasa de la PCB:** Envolvente protectora para el microcontrolador ESP32 y sus conexiones, diseñada para facilitar el acceso y disipar calor.
 6. **Tapa de case de pcb:** tapa diseñada para cubrir y proteger la PCB
 7. **Placa de base:** elemento que funciona como separador dentro de la maceta, dividiendo el tanque de agua de la zona destinada a la composta/planta.
- i. Diseño CAD de cada pieza del ensamble del sistema*
- **Maceta**

Vista lateral	Vista Superior
	
Descripción: En esta vista lateral de la maceta se observa la apertura destinada al extractor de aire y/o ventilador. Además, se identifica la extrusión diseñada para alojar el motor encargado de accionar la varilla agitadora. Por otra parte, se aprecian los puntos de sujeción que permitirán separar el tanque de agua de la sección destinada a la composta o a la planta seleccionada. Finalmente en la parte inferior se muestra una figura de mayor oscuridad, en esta se encontrará el sistema que ayudará al drenaje del tanque de agua.	Descripción: Desde la vista superior se pueden observar claramente las dimensiones generales de la maceta, lo que permite entender su escala y distribución. En esta vista también se identifican los puntos de sujeción, ubicados estratégicamente: cuatro en cada esquina, dos en el centro de los lados de 60 cm (a 30 cm de cada extremo), y dos en el centro de los lados de 40 cm (a 20 cm de cada borde). Esta disposición asegura una estructura estable y facilita el ensamblaje con otros componentes. Además, se aprecia el orificio de drenaje en la zona correspondiente al tanque de agua, el cual cumple una función esencial para evitar acumulaciones excesivas de líquido y mantener condiciones óptimas para el sistema de riego.
Vista Isométrica	Vista Inferior



Descripción: Se muestra la maceta en su estructura principal, en esta podemos ver que en cara lateral se muestra una apertura cuadrada, destinada a la instalación del extractor de aire y/o ventilador, lo cual facilitará la circulación interna.

En la misma cara, hacia la parte inferior, se encuentra una extrusión cilíndrica que servirá para la colocación del motor encargado de accionar la varilla agitadora.

Por otro lado, en una de las caras con mas anchura de la maceta, podemos observar cuatro orificios que servirán como punto de ensamble para la PCB. A lo largo de las paredes laterales y la base se observan varios puntos de sujeción distribuidos estratégicamente. Estos elementos permitirán fijar y separar el tanque de agua de la sección superior, donde se ubicará la planta o la composta. La base también incluye un orificio que cumple la función de drenaje, asegurando la correcta evacuación del exceso de agua.

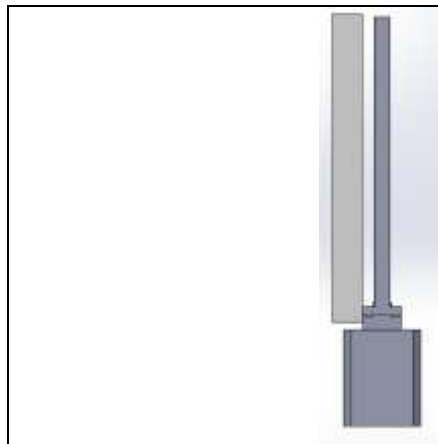


Descripción: En esta vista podemos ver toda la base de la maceta, lo que nos ayuda a entender cómo está distribuida la parte inferior. Se notan los puntos de sujeción en las esquinas y en zonas estratégicas alrededor del borde, que son súper importantes para darle estabilidad y para que todo el sistema se pueda montar bien.

También se ve el orificio circular en una esquina, que sirve como drenaje. Este detalle es clave porque evita que se acumule demasiada agua en el tanque y así mantenemos un buen equilibrio para el riego y para que la planta esté en condiciones óptimas.

- **Cortina o cubierta móvil**

Vista lateral	Vista Frontal
---------------	---------------



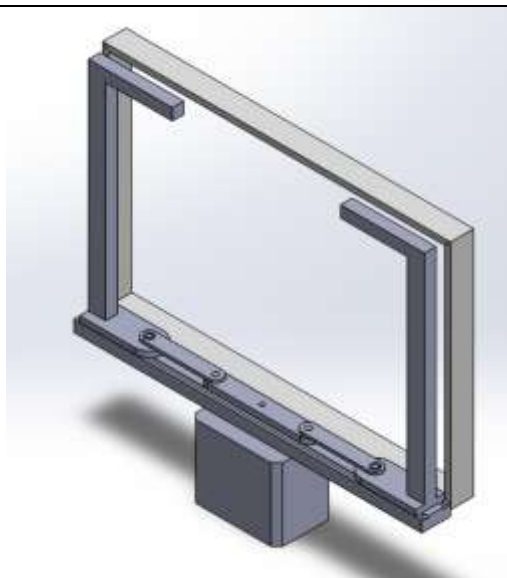
Descripción: En esta vista lateral se aprecia el sistema que permite el movimiento de la cortina. El componente ubicado en la parte derecha corresponde al mecanismo principal, el cual está empotrado en un marco que sirve como soporte estructural. La sección más ancha del dibujo representa el servomotor, encargado de controlar la apertura y el cierre de la cortina mediante movimientos precisos. Este diseño asegura un funcionamiento automatizado y eficiente, permitiendo regular la cubierta según las necesidades del sistema.

Vista Isométrica



Descripción: En esta vista se observa la estructura completa de la cortina. En la parte superior, se distinguen dos separaciones en forma de “L” invertida, una en cada extremo, que serán las encargadas de abrir y cerrar la malla, funcionando como guías para el movimiento. En la parte inferior del dibujo, se encuentra el mecanismo principal que permite el desplazamiento de la cortina. Este sistema está compuesto por varios segmentos conectados mediante uniones tipo deslizable, lo que facilita un movimiento lineal y articulado sin perder estabilidad, de esta manera se realiza el movimiento de apertura y cierre de la malla.

Vista Superior



Descripción: En esta vista podemos ver todo el mecanismo de la cortina, la estructura está montada sobre un marco que le da estabilidad y soporte, evitando que el sistema se deforme o pierda alineación cuando se mueve. En la parte superior se encuentran las piezas en forma de “L” invertida, una a cada lado, que son las encargadas de guiar la apertura y cierre de la malla.. En la parte inferior está el mecanismo principal. Este mecanismo no está conectado directamente al servomotor, sino que los segmentos que permiten el movimiento están unidos mediante uniones tipo deslizable y, además, sujetas a una barra de soporte. Esta barra es importante porque distribuye la fuerza y evita que todo el esfuerzo recaiga en el servomotor, dándole más estabilidad a la cortina. Por último, en la parte más baja se ve el servomotor, que será el encargado de accionar todo el mecanismo para abrir y cerrar la cortina de manera automática



Descripción: En esta vista se observa el mecanismo desde arriba, se pueden ver claramente las piezas en forma de “L” invertida, ubicadas en los extremos, que son las encargadas de guiar la apertura y cierre de la malla. Desde esta perspectiva, se aprecia cómo estas guías están alineadas con el marco para mantener la estabilidad del sistema. En la parte central se distingue la barra de soporte, sobre la cual están montadas las uniones tipo deslizable. Estas uniones conectan los segmentos del mecanismo y permiten que el movimiento sea lineal y controlado. La barra cumple una función estructural importante, ya que distribuye las fuerzas y aporta rigidez al conjunto. También se aprecia la posición del servomotor, que se encuentra alineado con el mecanismo y será el encargado de accionar la apertura y cierre de la cortina.

Finalmente, se incluye el plano general del ensamble de la cortina retráctil. En este plano se muestran las dimensiones completas del diseño, abarcando las vistas previamente descritas (frontal, lateral y superior), así como la vista isométrica. Este documento permite visualizar de manera integral las medidas y proporciones del mecanismo, asegurando una comprensión completa del sistema y su estructura.

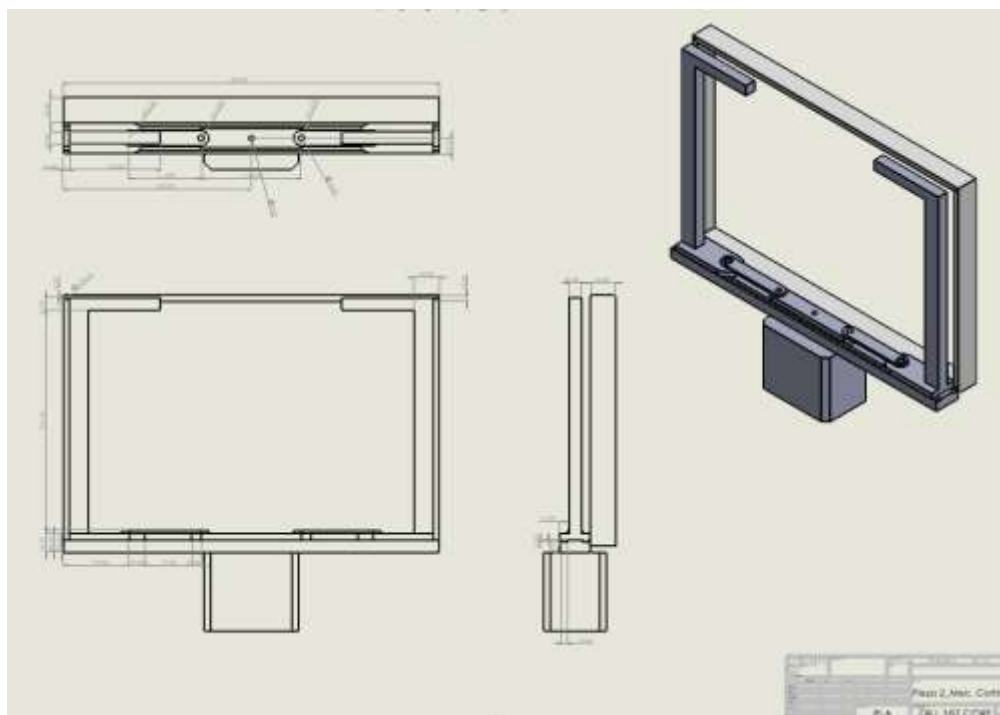
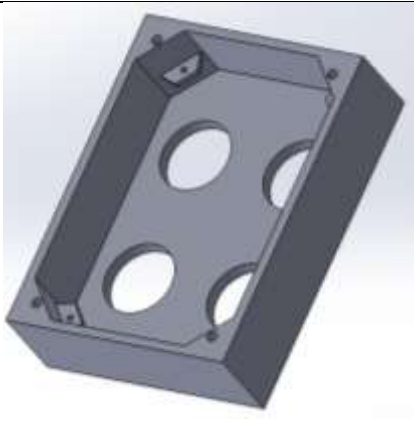
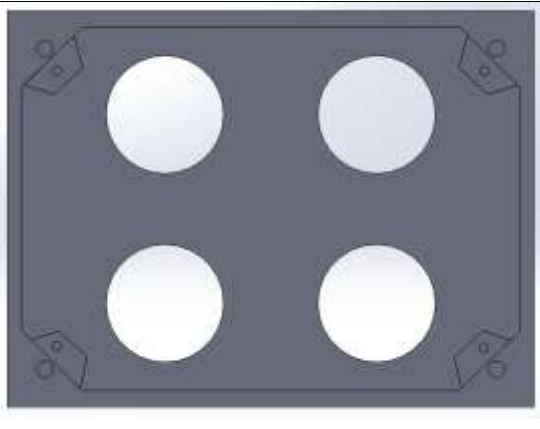


Ilustración 18. PLANOS TÉCNICOS CORTINA

- Case de pcb

Vista Isométrica	Vista Frontal
	
Descripción: En esta vista se aprecia la carcasa diseñada para alojar la tarjeta PCB. La estructura tiene forma rectangular con paredes laterales que proporcionan rigidez y protección al circuito. En la base se observan cuatro orificios circulares que sirven para el paso de cables y/o sensores que se encuentren dentro de la maceta. En cada esquina interna se distinguen puntos de sujeción, pensados para asegurar la	Descripción: En esta vista se observa la base interna del case desde una perspectiva frontal. Se distinguen claramente los cuatro orificios circulares distribuidos de manera simétrica, los cuales están pensados para permitir el paso de cables, ventilación o la integración de otros componentes del sistema.

PCB mediante tornillos, evitando movimientos dentro del case. Además, las esquinas presentan un diseño inclinado que facilita el montaje y aporta estabilidad al conjunto y que serán útiles para atornillar con la tapa del case.

En cada esquina se aprecian los puntos de sujeción, diseñados para fijar la PCB mediante tornillos o anclajes, asegurando que la tarjeta quede estable dentro de la carcasa. Además, se nota el diseño inclinado en las esquinas, que contribuye a la rigidez y facilita el montaje y servirán para montar la tapa del case para mayor seguridad de la placa así como la estética del proyecto.

Finalmente, se incluye el plano general del case de la PCB. En este plano se muestran las dimensiones completas del diseño, abarcando las vistas previamente descritas, así como la vista isométrica. Este documento permite visualizar de manera integral las medidas y proporciones del mecanismo, asegurando una comprensión completa de su estructura.

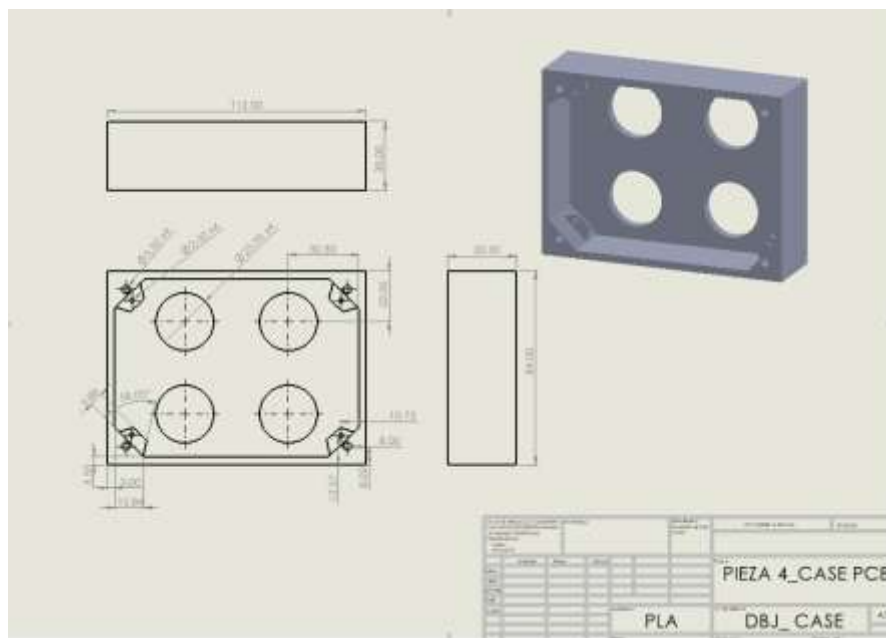


Ilustración 19.PLANOS TÉCNICOS CASE PCB

- **Tapa de case pcb**

Vista Isométrica	Vista Frontal
------------------	---------------



Descripción: En esta vista se muestra la tapa diseñada para cubrir y proteger el case del PCB. La pieza cuenta con cuatro orificios en las esquinas, que permiten fijarla mediante tornillos al cuerpo principal, asegurando un cierre firme y estable. Además, se incorporó un diseño decorativo en forma de señal Wi-Fi, que hace alusión al método de control del sistema, el cual será mediante una interfaz inalámbrica. Este detalle no solo aporta estética, sino que también comunica la funcionalidad del proyecto. Por ultimo se agregó el nombre de nuestro proyecto para darle mayor identidad a nuestro sistema.

Descripción: En esta vista se aprecia la tapa desde una perspectiva frontal, mostrando los elementos principales del diseño. Se observan cuatro orificios ubicados en las esquinas, cuya función es permitir la fijación mediante tornillos al cuerpo del case, garantizando un cierre seguro y estable. En el lado derecho se encuentra un diseño en forma de señal Wi-Fi, que hace referencia al método de control del sistema, el cual se realizará mediante una interfaz inalámbrica. En la parte central inferior se aprecia el nombre del proyecto “EcoSyncra”, grabado en la superficie, lo que aporta identidad.

Finalmente, se incluye el plano general de la tapa del case de la PCB. En este plano se muestran las dimensiones completas del diseño, abarcando la vista frontal, lateral y superior, así como la vista isométrica. Este documento permite visualizar de manera integral las medidas y proporciones del mecanismo, asegurando una comprensión completa de su estructura.

fijar la placa a la estructura de la maceta, asegurando estabilidad y evitando desplazamiento.

estructural de la placa. Además, esta pieza será fijada en los puntos de sujeción previamente mencionados dentro de la maceta, garantizando estabilidad.

Finalmente, se incluye el plano general de la placa de la base. En este plano se muestran las dimensiones completas del diseño, abarcando diferentes vistas, así como la vista isométrica. Este documento permite visualizar de manera integral las medidas y proporciones del mecanismo, asegurando una comprensión completa de su estructura.

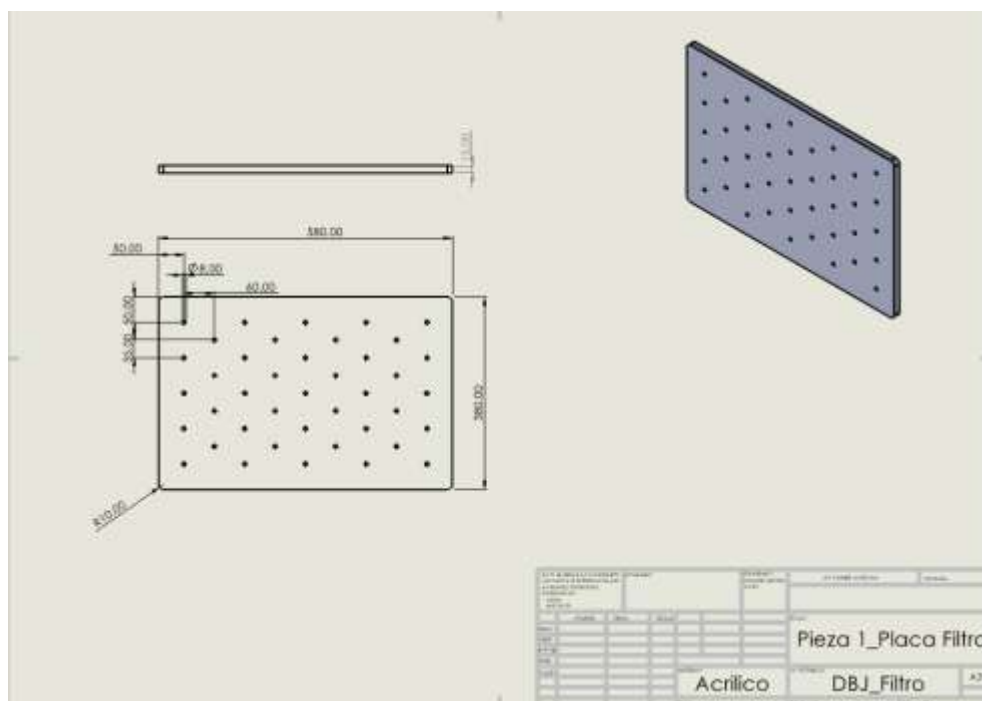


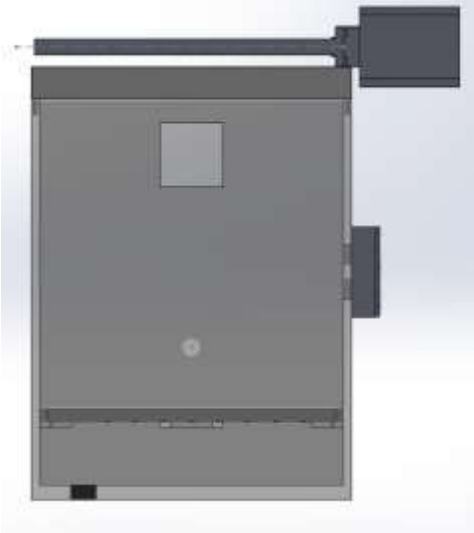
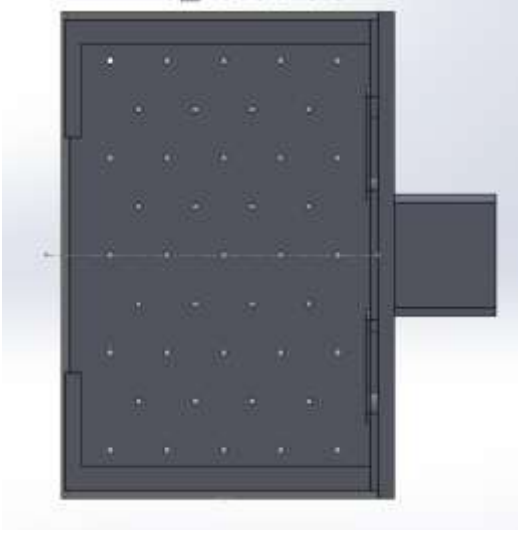
Ilustración 21. PLANOS TÉCNICOS PLACA BASE

ii. Ensamblaje CAD de mecanismo completo con todas las piezas

Para visualizar la integración total del sistema EcoSyncra, se presenta a continuación el modelo CAD del ensamble completo. Esta representación permite observar cómo se articulan todos los componentes mecánicos —desde el marco estructural y la maceta, hasta los elementos de soporte, cubierta y carcasa electrónica— en una configuración funcional y coherente. El ensamble refleja la disposición real del prototipo, facilitando el análisis de compatibilidad entre partes, rutas de cableado, accesibilidad para mantenimiento y puntos de

interacción con el sistema de control. Esta vista global es clave para validar la viabilidad técnica del diseño y su preparación para la automatización del riego y compostaje.

▪ **Ensamble de Ecosyncra final**

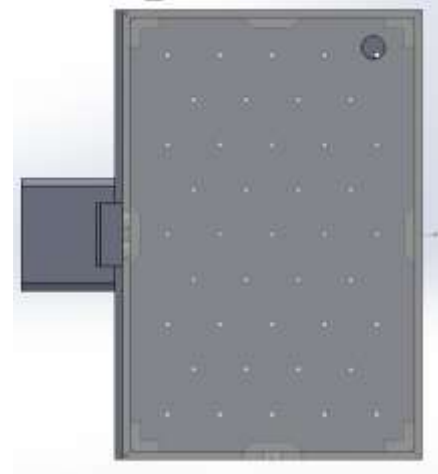
Vista lateral	Vista Superior
	
Descripción: En esta vista se aprecia la integración completa del sistema desde una perspectiva lateral, mostrando cómo se ensamblan los diferentes componentes en la estructura principal. La maceta constituye el cuerpo central del diseño, dentro del cual se encuentra la placa de base, encargada de separar el tanque de agua de la zona destinada a la planta o composta. Esta placa está fijada en los puntos de sujeción previamente mencionados, garantizando estabilidad y funcionalidad. En la parte superior se observa el mecanismo de la cortina retráctil, montado sobre un marco que proporciona soporte estructural. Este mecanismo incluye la barra con uniones tipo deslizable, que permiten el movimiento lineal de la cortina, y el servomotor, ubicado en el extremo derecho, encargado de accionar la apertura y cierre de manera automática. En el costado se aprecia el espacio destinado para el extractor de aire, el cual permitirá la circulación interna y el control de humedad. Asimismo, se identifica la extrusión para el motor de la varilla agitadora, que será el encargado de mezclar la composta y mantener la aireación adecuada en la zona	Descripción: En esta vista se aprecia la parte superior del sistema completamente ensamblado. Se observa la placa de base instalada dentro de la maceta, la cual separa el tanque de agua de la zona destinada a la planta o composta. La placa presenta múltiples perforaciones distribuidas uniformemente, permitiendo el paso controlado de agua y aire para mantener un adecuado drenaje y oxigenación. En el costado derecho se distingue el mecanismo de la cortina retráctil, compuesto por la barra con uniones tipo deslizable y el servomotor, que acciona la apertura y cierre de la cubierta. Este mecanismo está montado sobre un marco que proporciona estabilidad y soporte estructural.

superior. Finalmente, en la parte inferior de la maceta se encuentra el sistema de drenaje, diseñado para evacuar el exceso de agua del tanque, evitando acumulaciones que puedan afectar el funcionamiento del sistema.

Vista Isométrica



Vista Inferior



Descripción: En esta vista se aprecia el diseño completo del sistema desde una perspectiva tridimensional, lo que permite entender cómo se integran todos los componentes en la estructura principal. La maceta constituye el cuerpo central del proyecto y está diseñada para ser sólida y resistente. En la parte inferior se encuentra la placa de base, encargada de separar el tanque de agua de la zona destinada a la planta o composta. Esta placa está fijada en los puntos de sujeción previamente mencionados y cuenta con perforaciones distribuidas uniformemente para permitir el drenaje y la oxigenación del sustrato. En uno de los laterales se observa la parte donde se encuentra para el motor de la varilla agitadora, que será el encargado de mantener la mezcla homogénea y aireada en la zona superior. También se aprecia el espacio destinado para el extractor de aire, el cual permitirá la circulación interna y el control de humedad. En la parte superior se encuentra el mecanismo de la cortina retráctil, montado sobre un marco que aporta estabilidad. Este mecanismo incluye la barra con uniones tipo deslizable, que permiten el movimiento lineal de la cortina, y el servomotor, ubicado en el extremo, encargado de accionar la apertura y cierre de manera automática.

Descripción: En esta vista se observa la parte inferior del sistema completamente ensamblado. Se aprecia la placa de base, que ocupa la mayor parte del área y presenta perforaciones distribuidas uniformemente, permitiendo el paso controlado de agua y aire entre el tanque inferior y la zona superior destinada a la planta o composta. En la esquina superior derecha se distingue el orificio de drenaje, elemento clave para la evacuación del exceso de agua acumulada en el tanque. También se aprecia la integración del mecanismo lateral, donde se encuentra el servomotor que acciona la cortina retráctil.

Finalmente, en la zona lateral interna se aprecia el case del PCB, que alojará los componentes electrónicos para el control del sistema, incluyendo la interfaz inalámbrica que permitirá la gestión del proyecto.	
--	--

i. Planos técnicos de ensamble

Para complementar la visualización del diseño completo del sistema EcoSyncra, se presentaron los planos técnicos del modelo CAD del ensamble. Estos planos incluyen vistas detalladas — acompañadas de acotaciones precisas que especifican dimensiones, posiciones de perforaciones, cortes funcionales y tolerancias. La documentación técnica permite validar la factibilidad de fabricación, asegurar el correcto ensamblaje de los componentes y facilitar la interpretación por parte de equipos de ingeniería, manufactura o montaje. Esta etapa es clave para garantizar que el prototipo físico se construya conforme a los parámetros definidos en el diseño digital.

iii. Simulación del ensamble final para validación por estudios

Como parte del proceso de validación estructural del diseño completo del sistema EcoSyncra, se realizaron simulaciones mecánicas en SolidWorks que incluyen estudios de pandeo, análisis de esfuerzos de Von Mises y desplazamientos estáticos. Estas simulaciones permiten evaluar el comportamiento del ensamble bajo condiciones de carga realistas, identificando zonas críticas, posibles deformaciones y límites de estabilidad. Gracias a estos estudios, se garantiza que el diseño no solo cumple con los requisitos funcionales, sino que también posee la resistencia y rigidez necesarias para operar de forma segura y confiable en entornos agrícolas urbanos automatizados.

Simulación de ensamble final por Von Mises

Maceta:

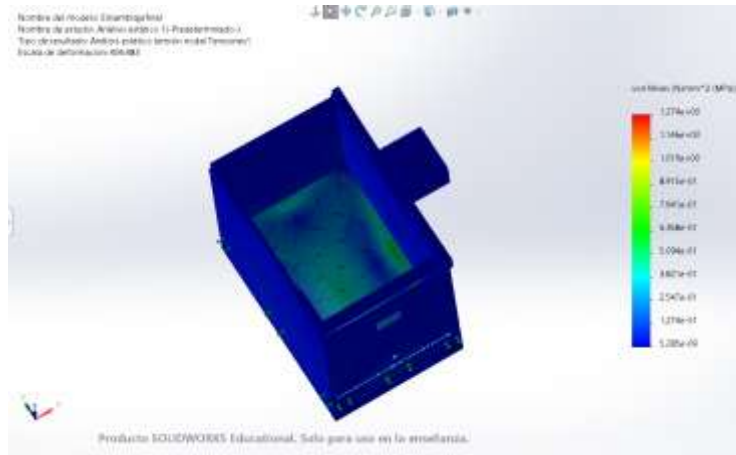


Ilustración 22. SIMULACIÓN CAD VON MISES ENSAMBLE FINAL

En este estudio, las cargas se aplicaron sobre la parte superior del ensamble, simulando el peso que soportará la estructura (por ejemplo, la planta, la composta y el mecanismo superior). Las restricciones se colocaron en la base y puntos de sujeción, representadas por los marcadores verdes, para simular la fijación del sistema.

Resultados obtenidos:

- La escala de colores indica la magnitud del esfuerzo en MPa:
 - Rojo = mayor esfuerzo (hasta 1.27 MPa).
 - Azul oscuro = menor esfuerzo (casi nulo).
- La mayor concentración de esfuerzos se encuentra en la zona superior interna, donde se distribuye la carga, mientras que las paredes y la base permanecen en azul, indicando tensiones muy bajas.

El valor máximo obtenido (≈ 1.27 MPa) es significativamente menor al límite elástico típico de materiales plásticos o metálicos (> 20 MPa), lo que confirma que el diseño es seguro y estable para las condiciones simuladas.

El análisis demuestra que el ensamble final de Ecosyncra puede soportar las cargas previstas sin riesgo de falla estructural. La distribución uniforme de esfuerzos y la baja magnitud de tensiones validan que el diseño es adecuado para su uso.

✓ **Simulación de ensamble final por Pandeo**

Maceta:

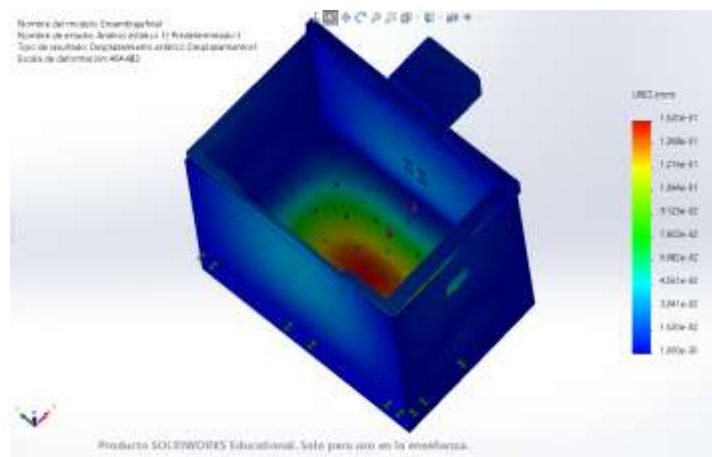


Ilustración 23. SIMULACIÓN CAD ENSAMBLE FINAL MACETA POR PANDEO

La carga se aplicó sobre la parte superior del ensamble, simulando el peso que soportará la maceta (planta, composta y mecanismo superior). Las restricciones se colocaron en la base y puntos de sujeción, representadas por los marcadores verdes, para simular la fijación del sistema.

Resultados obtenidos:

- La escala de colores indica el desplazamiento en milímetros (mm):
 - Rojo (≈ 0.152 mm) = mayor desplazamiento.
 - Azul (≈ 0 mm) = desplazamiento mínimo.
 - Las zonas con mayor deformación se concentran en el centro de la placa superior, mientras que las paredes y esquinas permanecen en azul, indicando alta rigidez.
 - El desplazamiento máximo es 0.152 mm, lo cual es extremadamente bajo y no compromete la integridad del diseño.

El análisis demuestra que la maceta es estructuralmente estable ante cargas de compresión. El pandeo no representa un riesgo, ya que las deformaciones son mínimas y el diseño mantiene su forma y funcionalidad. Esto confirma que el ensamble es seguro para soportar las condiciones previstas.

✓ **Simulación de ensamble final por desplazamiento estático**

Maceta:

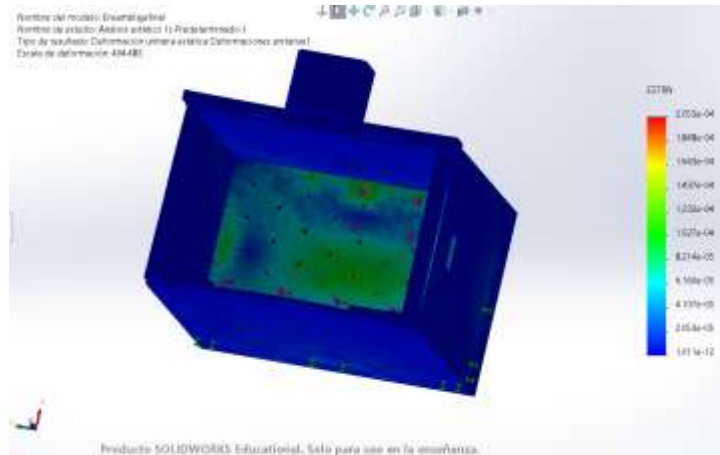


Ilustración 24. SIMULACIÓN CAD ENSAMBLE FINAL DESPLAZAMIENTO ESTÁTICO MACETA

La carga se aplicó sobre la parte superior del ensamble, simulando el peso que soportará la maceta (planta, composta y mecanismo superior). Las restricciones se colocaron en la base y puntos de sujeción (indicados en verde), para representar la fijación del sistema.

Resultados obtenidos

- La escala de colores indica la deformación unitaria (strain):
 - Rojo ($\approx 2.05e-4$) = mayor deformación.
 - Azul ($\approx 1.01e-12$) = deformación prácticamente nula.
- Las zonas con mayor deformación se concentran en el centro de la placa superior, mientras que las paredes y esquinas permanecen en azul, lo que indica alta rigidez.
- El valor máximo de deformación ($\approx 2.05e-4$) es extremadamente bajo, lo que significa que la estructura no sufrirá cambios significativos en su forma.

El análisis confirma que la maceta es estructuralmente estable ante cargas estáticas. Las deformaciones son mínimas y no comprometen la funcionalidad del diseño, por lo que el ensamble es seguro para las condiciones previstas.

3.2 ETAPA ELECTRÓNICA

La etapa electrónica del sistema EcoSyncra marca el punto en el que el prototipo integra sus componentes de control y automatización. En esta fase se incorporan la PCB, los sensores, actuadores y el microcontrolador, asegurando la comunicación entre el diseño físico y el sistema inteligente de riego y compostaje. El objetivo es validar que la electrónica se acopla

correctamente al ensamble mecánico, garantizando tanto la protección de los circuitos como la funcionalidad del control algorítmico. Esta integración permite que el prototipo evolucione de una estructura mecánica a un sistema completo, capaz de operar de manera autónoma y eficiente.

3.2.1 Diagrama de bloques

Para esta etapa primero hay que considerar el funcionamiento de nuestro sistema para definir que entradas, salidas, control y procesamiento tendremos, por lo cual se implementa un diagrama de bloques que nos permite visualizar con claridad esta parte, como se muestra a continuación:

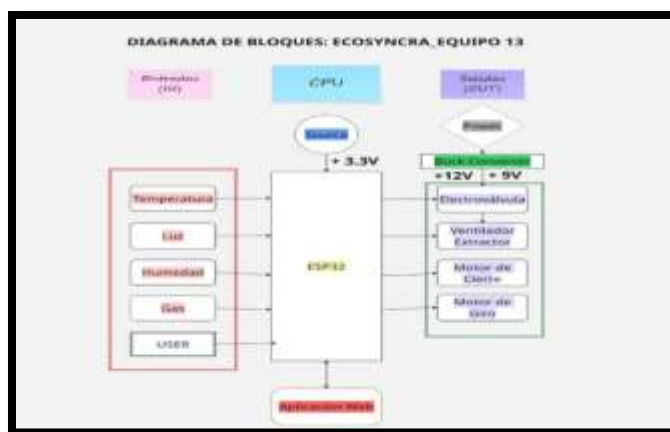


Ilustración 25. DIAGRAMA DE BLOQUES ECOSYNCR A

Este diagrama de bloques del sistema EcoSyncra representa una arquitectura de control ambiental automatizado, que está diseñada para monitorear y regular las condiciones óptimas de cultivo de cuatro plantas base que son el romero, cactus, menta y albahaca. Cada una de estas especies tiene requerimientos específicos de temperatura, humedad, luz y gases por lo que el sistema integra sensores dedicados a que capturan esos parámetros en tiempo real. Estos datos se envían al microcontrolador ESP32, que actúa como un núcleo de procesamiento, tomando decisiones sobre los actuadores conectados como electroválvulas, ventiladores y motores para mantener el entorno ideal según la planta seleccionada.

La interfaz de usuario (USER), representada en el bloque de entradas, permite al operador seleccionar entre las plantas preconfiguradas o ingresar una nueva especie con requerimientos personalizados. Esta flexibilidad amplía el alcance del sistema, permitiendo

adaptar el control ambiental a cualquier planta fuera del trío base, al registrar una nueva planta, el sistema comienza a recopilar datos de sus condiciones reales, almacenándolos y ajustando progresivamente los parámetros de control. Esta función de aprendizaje y adaptación convierte a EcoSyncra en una solución escalable y replicable para cultivos urbanos diversos.

Además, el ESP32 está conectado a una aplicación web, lo que permite al usuario visualizar los datos históricos de temperatura, humedad, luz y gases, así como controlar remotamente los actuadores. La fuente de alimentación está dividida en dos niveles: una entrada de +12V regulada a +9V mediante un Buck converter para los motores y una línea de +3.3V dedicada al ESP32. Esta separación asegura la eficiencia energética y protección del sistema. De esta forma, el diagrama refleja una solución integrada que combina monitoreo ambiental, control automatizado y expansión personalizada, alineada con los principios de sostenibilidad y democratización tecnológica que guían el enfoque.

3.2.2 Diagramas esquemáticos

Los diagramas esquemáticos representan la etapa en la que el diseño electrónico pasa de una visión general a una descripción técnica detallada. A diferencia del diagrama de bloques, que muestra únicamente la relación entre módulos, el esquema eléctrico especifica cómo se conectan cada uno de los componentes: sensores, actuadores, microcontrolador, fuentes de alimentación y elementos de protección. Esta documentación es fundamental para validar la lógica del circuito, evitar errores de conexión y preparar el diseño de la PCB. Gracias a los diagramas esquemáticos, se asegura que la integración electrónica del sistema EcoSyncra sea clara, precisa y confiable para su fabricación y montaje.

A continuación, se presentarán los diagramas esquemáticos correspondientes a la etapa electrónica del sistema, en los cuales se integran los principales componentes de control y sensado. Estos incluyen el servomotor, el ventilador extractor, los motores de compostaje de corriente directa, así como los sensores de ambiente y suelo: DHT11, sensor de temperatura del sustrato y sensor de gas. La representación esquemática permitirá visualizar de manera clara las conexiones eléctricas y la interacción entre cada módulo, asegurando la correcta integración del hardware dentro del ensamble final.

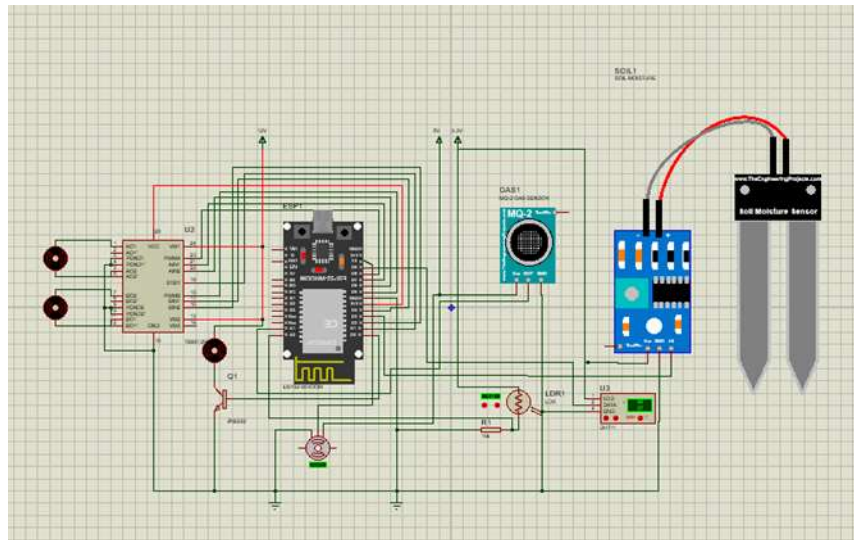


Ilustración 26. Esquemático en Proteus

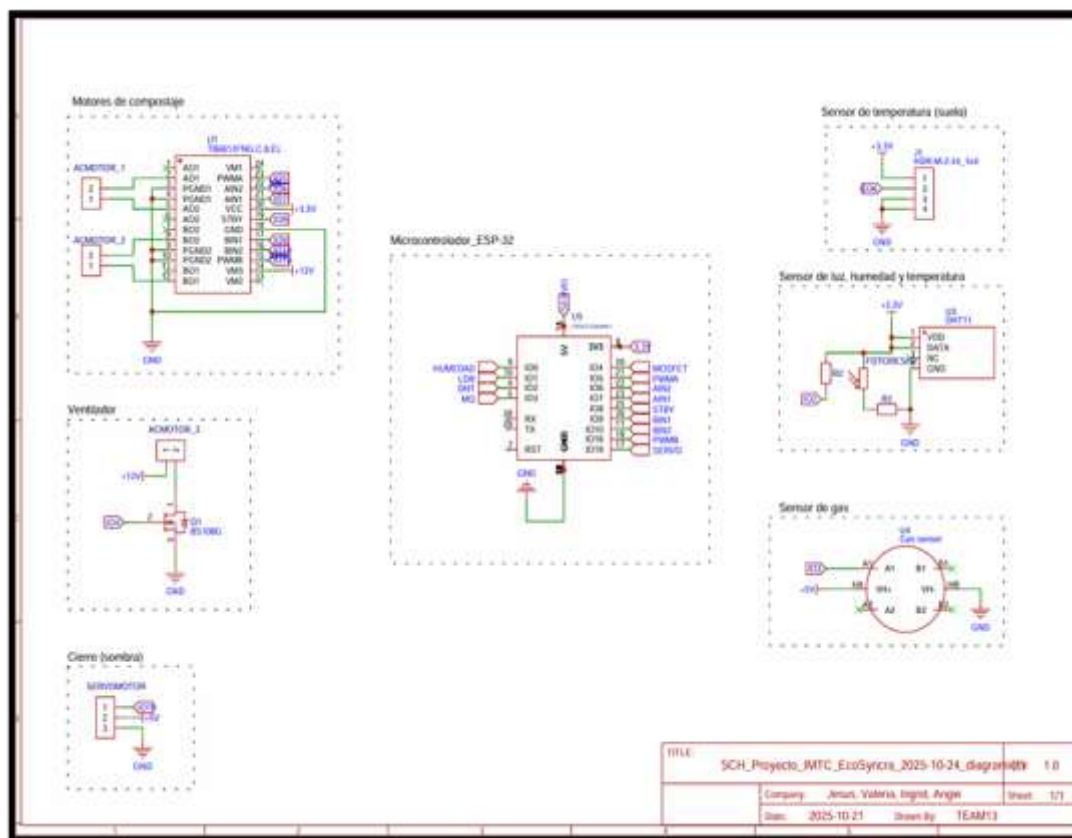
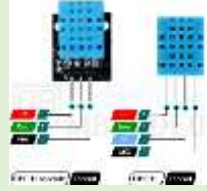


Ilustración 27. ESQUEMÁTICOS PCB

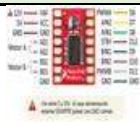
3.2.3 Lista de componentes electrónicos involucrados

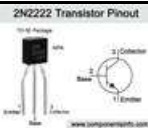
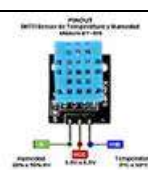
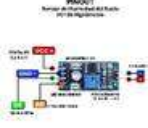
A continuación, se presenta una tabla para enlistar los componentes electrónicos que se requieren para esta etapa.

COMPONENTE	TIPO/MODELO	FUNCIÓN PRINCIPAL	OBSERVACIONES	IMAGEN
Microcontrolador	ESP32 DevKit	Unidad central de control y comunicación	Soporta WiFi/Bluetooth, gestiona sensores y actuadores	
Servomotor	SG90	Movimiento preciso para la garra o mecanismos	Movimiento preciso para la garra o mecanismos	
Motor DC	9V / baja potencia	Accionamiento del sistema de compostaje	Controlado mediante driver TB6612	
Ventilador extractor	12 V	Regulación de flujo de aire y gases	Se recomienda protección contra polvo	
Sensor ambiental	DHT11	Medición de temperatura y humedad del aire	Fácil integración con ESP32	
Sensor de humedad del suelo	FC-28	Monitoreo del sustrato	Preciso para compostaje y raíces	
Sensor de gas	MQ-135 /	Detección de gases generados en compostaje	Necesita calibración inicial	

Driver de motores	TB6612	Control de motores DC con dirección y velocidad	Más eficiente que un puente H tradicional	
Fuente de alimentación		Suministro eléctrico para motores y electrónica	Debe incluir reguladores de voltaje	
PCB + Case	Diseño propio en PLA	Montaje y protección de la electrónica	Facilita mantenimiento y disipación térmica	
Cables y conectores	Dupont / terminales	Interconexión de todos los módulos	Organizados para evitar interferencias	
Tarjeta perforada	7.5 cm x 4.5 cm	Conexión de motores	Facilita el orden mediante	
Tarjeta adaptadora de Esp32		Ayuda de base de esp32	Permite un mejor prueba de los sensores	

Lista de especificaciones técnicas de los componentes usados

Componente	Modelo	Voltaje/Corriente	Pinout relevante	Cantidad	Notas de uso
Microcontrolador	ESP32-DEVKITC-32D pdf , ESP32-DEVKITC-32D Descripción , ESP32-DEVKITC-32D Hoja de datos , ESP32-DEVKITC-32D vista , ALLDATASHEET	3.3 V / 500 mA máx.		1	Control central, PWM y comunicación WiFi/Bluetooth
Driver de motores	TB6612FNG Hoja de datos	2.5–13.5 V / 1.2 A por canal		1	Control de 2 motores DC, requiere

	PDF - Alldatasheet				disipación mínima
Transistor	2N2222 2N2222 Hoja de datos, PDF - Alldatasheet	40 V / 800 mA máx.		1	Conmutación de cargas pequeñas, requiere resistencia en base
Servomotor	SG90 SG90 Datasheet(PDF) - List of Unclassified Manufacturers	4.8–6 V / 500 mA		1	Movimiento angular preciso, controlado por señal PWM
Motor DC	MOT-155 MOT- Datasheet, PDF - Alldatasheet	9V/54mA		1	Motor de alto torque
Ventilador	12 V VENTILADOR AXIAL DE 12V Datasheet, PDF - Alldatasheet	0.2–0.5 A		1	Extracción de aire y gases
Sensor ambiental	DHT11 DHT11 Datasheet, PDF - Alldatasheet	3.3V - 5V		1	Detectar temperatura y humedad en el ambiente
Sensor de suelo	FC-28 FC-28 Datasheet, PDF - Alldatasheet	3.3V - 5V		1	Detección de hidratación del sustrato
Sensor de gas	MQ-135 MQ135 Datasheet, PDF - Alldatasheet	5V		1	Necesita calibración inicial,

					sensible a CO ₂ y NH ₃
Fuente de poder	Cargador de 12V				

3.2.4 Simulación electrónica

La simulación electrónica en plataformas como Proteus constituye una etapa fundamental para validar el diseño antes de la implementación física. A través de este entorno se pueden representar los circuitos esquemáticos con todos los componentes —microcontrolador, sensores, actuadores, drivers y transistores— y observar su comportamiento en condiciones controladas. Esta fase permite comprobar la lógica de conexión, verificar señales de entrada y salida, analizar consumos de corriente y detectar posibles errores sin necesidad de fabricar la PCB de inmediato. Además, la simulación facilita la optimización del diseño, ya que se pueden realizar ajustes en parámetros, probar diferentes configuraciones y anticipar el desempeño del sistema completo. De esta manera, Proteus se convierte en una herramienta clave para asegurar que la integración electrónica del proyecto sea confiable, eficiente y lista para su montaje físico.

3.2.5 Diseño de PCB

El diseño de la PCB del sistema EcoSyncra inició en KiCad, donde se trazaron los primeros esquemas y se definió la distribución básica de los componentes electrónicos. Esta etapa permitió establecer la lógica de conexión y validar la coherencia entre el diagrama esquemático y el layout inicial. Sin embargo, durante el proceso se identificaron ajustes necesarios en la disposición de pistas, la organización de los módulos y la compatibilidad con el ensamble físico. Por ello, el proyecto se migró a EasyEDA, una plataforma que ofreció mayor flexibilidad para optimizar el ruteo, integrar librerías específicas y generar archivos de fabricación más accesibles. Finalmente, el diseño se concluyó en EasyEDA, logrando una PCB funcional, clara y lista para su producción, asegurando la correcta integración de sensores, actuadores y el microcontrolador dentro del prototipo.

♣ **Consideraciones que se tuvieron al diseñar la pcb**

Al manejar 4 actuadores en una misma placa se decidió separar en 3 secciones, el área central como el área lógica, la sección lateral derecha como las entradas de señales y la sección lateral izquierda como las salidas y el área reguladora de potencia.

♣ **Esquemáticos de componentes de pcb**

A continuación, se describen los componentes integrados a la pcb del sistema EcoSyncra

❖ **Control central**

- **ESP32 DEVKIT:** Microcontrolador principal que gestiona sensores, actuadores y comunicación inalámbrica.

❖ **Actuadores conectados**

- **Driver L298N:** Módulo para control de motores DC (compostaje), conectado a pines D23, D22, D21 y D19 del ESP32.
- **Ventilador extractor (DC):** Controlado desde el pin D18 del ESP32, alimentado por +5V.

❖ **Sensores ambientales y de suelo:**

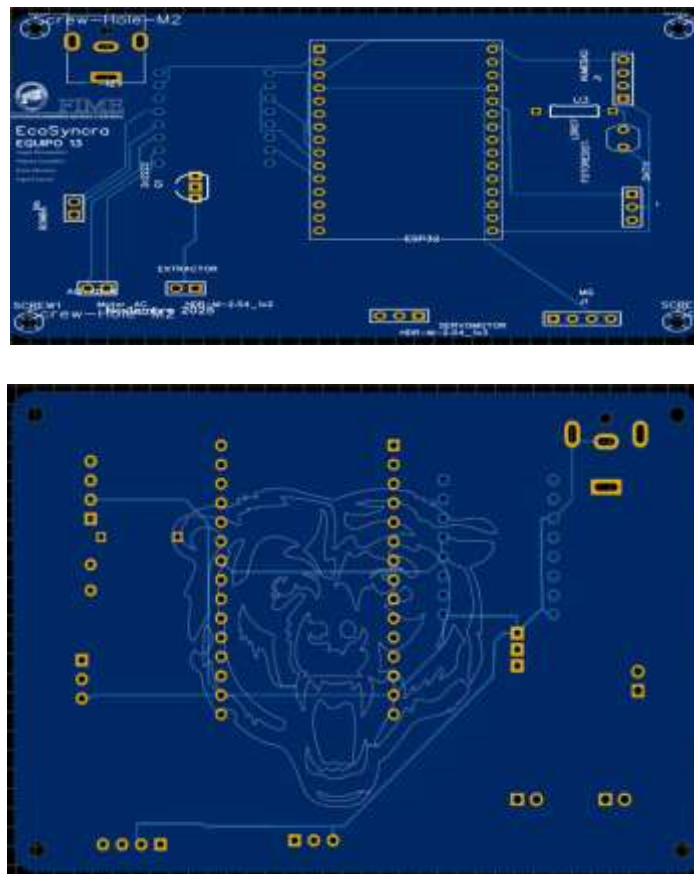
- **DHT11:** Sensor de temperatura y humedad del aire, conectado al pin D5.
- **DS18B20:** Sensor de temperatura del sustrato, conectado al pin D4
- **MQ-135:** Sensor de gases, típicamente conectado a entrada analógica o digital
- **LDR (resistor dependiente de luz):** Sensor de luz ambiental, conectado al pin D34 mediante divisor de voltaje.

Donde su esquemáticos para el diseño de la pcb son los siguientes:

El esquemático electrónico del sistema EcoSyncra representa una arquitectura integrada basada en el microcontrolador ESP32, que actúa como núcleo de control para una red de sensores y actuadores orientados al monitoreo ambiental y la automatización de procesos. A través de sus pines GPIO, el ESP32 se conecta a módulos como el driver L298N para controlar motores de compostaje, un ventilador extractor, sensores de temperatura y humedad (DHT11 y DS18B20), un sensor de luz mediante LDR, . La alimentación se distribuye mediante rieles de +5V y +3.3V, asegurando compatibilidad con los distintos componentes. El diseño refleja una lógica modular que permite gestionar tanto variables ambientales como acciones mecánicas, consolidando una plataforma versátil para aplicaciones de agricultura inteligente y compostaje automatizado.

♣ Diseño de pcb final para electrónica

El diseño de la PCB representa la consolidación física del sistema electrónico, donde se integran todos los componentes previamente definidos en el esquemático. Esta etapa permite organizar de forma compacta y funcional los sensores, actuadores, el microcontrolador y los módulos de potencia, asegurando conexiones seguras y una distribución eficiente de señales y energía. A continuación, se muestra el diseño final de la placa, resultado de un proceso iterativo que inició en KiCad y se optimizó en EasyEDA para garantizar compatibilidad con el ensamble físico y facilitar su fabricación. Como se muestra a continuación:



Integración del ensamble físico con los componentes electrónicos

La etapa física de la electrónica representa la transición del diseño digital al montaje tangible de los componentes sobre la PCB. En esta fase se lleva a cabo la colocación, soldadura y conexión de cada módulo: microcontrolador, sensores, actuadores, drivers y elementos de alimentación, siguiendo el layout previamente definido. Este proceso permite validar la compatibilidad entre el diseño estructural y la electrónica, asegurando que cada componente

esté correctamente orientado, alimentado y comunicado. Además, se verifica la accesibilidad de los conectores, la disipación térmica y la protección del sistema, consolidando una base robusta para las pruebas funcionales y la integración final con el ensamble mecánico del sistema EcoSyncra.

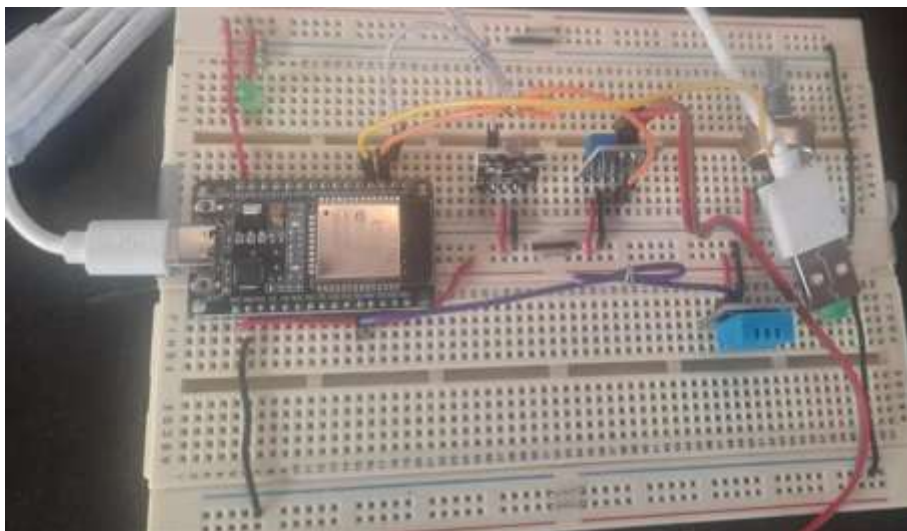


Ilustración 17: Prueba de electrónica del circuito

▪ Montaje en pcb

1. Preparación del área de trabajo
 - Mantener la mesa limpia, bien iluminada y con ventilación adecuada.
2. Revisión de la PCB y los componentes
 - Verificar que el diseño impreso corresponda al esquema final.
 - Confirmar que todos los componentes estén disponibles según la lista BOM (ESP32, TB6612, 2N2222, sensores, conectores, etc.).
3. Colocación de los componentes principales
 - Insertar el ESP32 en su socket o header.
 - Montar el driver TB6612 y los transistores 2N2222 en sus posiciones designadas.
 - Colocar los conectores correspondientes para sensores y actuadores.
4. Soldadura de componentes
 - Utilizar un cautín de punta fina y estaño con resina.
 - Soldar primero los pines de control (GPIO, VCC, GND).
 - Asegurar que no existan puentes accidentales entre pistas.

5. Montaje de conectores y terminales
 - Instalar headers para sensores (DHT11, DS18B20, MQ-135, LDR).
 - Colocar borneras o conectores para motores y ventilador.
 - Verificar la firmeza mecánica y la continuidad eléctrica.
 6. Instalación de elementos de potencia
 - Conectar la fuente de alimentación regulada (12 V $\rightarrow$ 5 V y 3.3 V).
 - Revisar las rutas de alimentación y confirmar que cada módulo reciba el voltaje adecuado.
 7. Inspección visual y pruebas de continuidad
 - Comprobar la ausencia de soldaduras frías o puentes.
 - Usar un multímetro para verificar continuidad y niveles de voltaje.
 8. Prueba funcional inicial
 - Encender la fuente regulada.
 - Confirmar el arranque del ESP32 y la respuesta básica de sensores y actuadores.
 9. Integración con el prototipo físico
 - Asegurar que la PCB encaje correctamente en su case o soporte.
 - Revisar la accesibilidad de los conectores y la correcta disipación térmica.
- **Herramientas necesarias adicionales**
1. Cautín de punta fina (30–60 W)
 2. Estaño con núcleo de resina (0.5–1 mm)
 3. Pinzas de precisión y cortadores de patillas
 4. Multímetro digital
 5. Alcohol isopropílico y cepillo para limpieza
 6. Destornilladores y brocas (para fijación en case)
 7. Fuente de alimentación regulada (5 V y 12 V)

A continuación, se muestra la pcb en físico

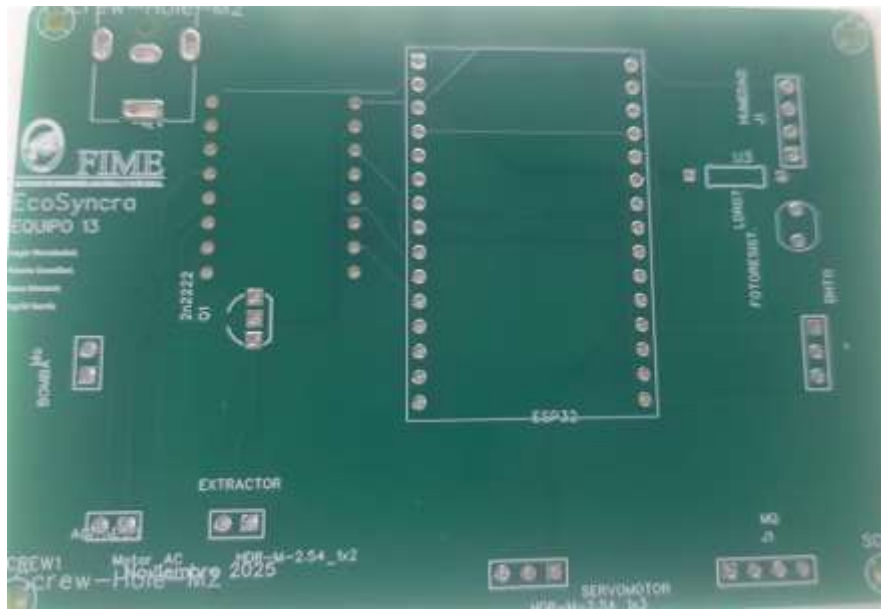


Ilustración 28. PCB FÍSICA VISTA DELANTERA

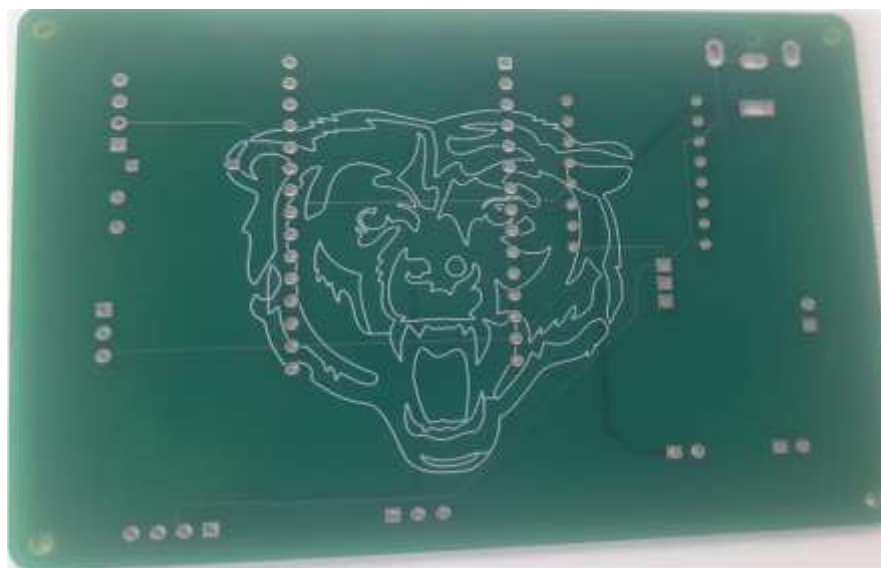


Ilustración 29. PCB FÍSICA VISTA TRASERA

3.3 ETAPA DE CONTROL/ALGORITMICA

El algoritmo de control no solo define la lógica de actuación de los sensores y actuadores, sino que también se complementa con un sistema de monitoreo en tiempo real mediante una interfaz y un dashboard web. Gracias a las capacidades de comunicación del ESP32, el sistema puede transmitir datos vía Bluetooth o WiFi, permitiendo que las variables ambientales (temperatura, humedad, luz, gases) y los estados de los actuadores (motores, ventilador, servomotor) se visualicen de forma inmediata en una plataforma gráfica.

Este dashboard web ofrece indicadores dinámicos, gráficas de evolución y controles interactivos que facilitan la supervisión y ajuste del sistema desde cualquier dispositivo conectado. De esta manera, el algoritmo de control se integra con la capa de comunicación y visualización, logrando un ecosistema completo: captura de datos → procesamiento lógico → acción física → monitoreo remoto en tiempo real.

3.3.1 Pseudocódigo

El pseudocódigo es importante porque permite expresar la lógica del algoritmo de control de manera simple y estructurada, sin necesidad de usar un lenguaje de programación específico. Funciona como un puente entre el diagrama de flujo y la implementación real en el microcontrolador, facilitando la comprensión de cada paso y reduciendo errores en la codificación.

En la sección de anexos de insertara el código (9.4.1 Pseudocodigo)

3.3.2 Simulaciones de control

3.3.2.1 Pruebas iniciales

El control utilizado fue PID donde aplica de la siguiente manera:

El control PID en este simulador se aplica exclusivamente para regular la velocidad del ventilador (extractor), con el objetivo de mantener la temperatura del compostaje dentro de un rango óptimo que favorezca una descomposición gradual y eficiente. La temperatura es el parámetro clave porque influye directamente en la actividad microbiana: si sube demasiado, la descomposición se acelera de forma abrupta, lo que puede degradar nutrientes esenciales; si baja demasiado, el proceso se ralentiza. El PID actúa como un moderador inteligente: cuando detecta que la temperatura se acerca o supera los 60°C, incrementa progresivamente la potencia del ventilador para disipar el exceso de calor. En cambio, si la temperatura está por debajo del setpoint, reduce o elimina la acción del ventilador, permitiendo que el compost retenga el calor necesario para continuar el proceso biológico.

Este enfoque permite que el sistema mantenga un equilibrio térmico sin oscilaciones bruscas, asegurando que la descomposición se mantenga controlada y que los nutrientes se conserven. El ventilador no se enciende simplemente por estar “caliente”, sino que su velocidad se ajusta proporcionalmente a cuánto se desvía la temperatura del valor deseado. Así, cuando hay poco

calor, el PID apenas activa el ventilador; cuando hay mucho calor, lo acelera más. Esta modulación continua evita que el compost pierda humedad o se enfríe innecesariamente, y permite validar que el sistema responde de forma precisa y adaptativa, tal como se observa en el plotter de Thonny: la línea de temperatura se estabiliza cerca del setpoint, mientras la línea del esfuerzo PID sube y baja suavemente según lo requiere el sistema.

3.3.2.2 Parámetros de diseño y ajustes realizados

Cabe señalar que los parámetros se dividen principalmente en dos grupos donde uno tiene que ver con el diseño que serían como las decisiones antes de iniciar que se basan en la biología del composteo o en el hardware, siendo como los objetivos mientras que los de ajuste son los diales de forma que hagan que el sistema se comporte más hacia como se desea y alcance esos objetivos.

Ese es el sistema de control más complejo, su objetivo se basa en mantener la temperatura estable en el setpoint donde vemos lo siguiente:

Parámetros principales de diseño

SETPOINT_TEMPERATURA viene a ser la temperatura ideal que se desea alcanzar y mantener el compostaje, siendo un parámetro biológico, revisando cual sería la temperatura óptima para la fase de composteo que se desea que es fase mesofílica o termofílica, resultando que es la que higieniza el compostaje.

PID_LIMITS vienen a ser los límites de salida que es el rango de salida del PID. Donde si se usa un relé (on/off), este valor de 0-100 es conceptual, en cambio si se usa un controlador de velocidad PWM para el ventilador este valor de 0-100 se mapea directamente al ciclo de trabajo (Duty Cycle) del PWM. Para en el caso de la simulación, se dejó en valores de (0,100) como si fuera un porcentaje de esfuerzo.

Parámetros principales de ajuste

Aquí cabe resaltar que es donde aplica el plotter de Thonny o la gráfica de MATLAB. El objetivo se basa en el ajuste de estos 3 valores mencionados a continuación para que la línea de la temperatura (roja) llegue a la línea del setpoint (azul) de una forma rápida, estable y sin pasarse (overshoot).

KP (Constante proporcional): se trata del “presente”, es decir, reacciona inmediatamente al error actual, si consideramos su ajuste considera que cuando si el KP resulta muy alto el sistema reaccionará muy bruscamente, causando grandes oscilaciones donde la temperatura subirá y bajará pasando el setpoint, en cambio si resulta muy bajo, el sistema será muy “lento” y tardará demasiado en reaccionar. Por lo que ante esto se recomienda empezar con un KP pequeño y ve subiéndolo hasta que el sistema reaccione rápido, justo antes de que empiece a soldar.

KI (Constante Integral) se podría considerar como el “pasado”, debido a que corrige errores pequeños pero que son persistentes donde debemos verlo del siguiente modo en el cual si la temperatura se queda “atascada” en 58°C y no llega a 60 °C (error de estado estacionario), donde el KI acumula ese pequeño error con el tiempo y de esta forma empuja la salida hasta corregirlo. Por lo que ante esto considera que si KI es muy alto, acumulará error demasiado rápido y causará oscilaciones lentas, si es muy bajo (o 0), resulta muy posible que nunca se alcance el setpoint exacto. Ante ello se recomienda empezar con $KI = 0$, de forma que se ajuste KP primero, después si se observa que la temperatura no llega exactamente al setpoint, empieza a subir KI de forma lenta.

KD (Constante Derivativa) se podría considerar como el “futuro” debido a que actúa como un “freno” que predice hacia dónde va el error. En el cual, si la temperatura está subiendo muy rápido, el KD frena la acción para evitar que se pase del setpoint. Además, su ajuste considera que amortigua el sistema y previene el overshoot, donde si KD es muy alto, el sistema será demasiado “cauteloso” y lento, en cambio, si es muy bajo, la temperatura se pasará de 60 °C antes de empezar a bajar. Ante ello se recomienda que a menudo sea el último en ajustarse, y usarse si se cuenta con mucho overshoot (donde la temperatura sobrepase de 60) inclusive con un KP bien ajustado.

También tenemos los parámetros del control por Umbral (ON/OFF)

UMBRAL_HUMEDAD_BAJA: Aquí considera que, si el porcentaje de humedad como por ejemplo 30% por debajo del cual se activará la bomba de agua.

UMBRAL_MQ135_GASES: En este punto lo vemos como el nivel del PPM (Partes por Millón) de los gases, por ejemplo 400.0 por encima del cual se activan los rodillos y el

ventilador para airear, considerando un ajuste más difícil de calibrar debido a que el sensor MQ135 es algo genérico, el mejor ajuste es empírico por lo cual el compostaje deja de funcionar, entonces cuando huele a amoníaco o a podrido (siendo una señal de anaerobiosis) se mira la lectura del sensor, siendo ese valor para el umbral de gases.

UMBRAL_LUZ_ALTA: se puede observar como el valor del LDR, por ejemplo 900 que se considera para “luz solar directa”, entonces se consideraría un ajuste de poner el LDR en la sombra y anotar el valor, ponerlo bajo luz solar directa y anotar el valor, por lo cual el umbral será un número alto, cercano al de la luz directa, por ejemplo, si la sombra es de 200 y el sol es de 1000, un umbral de 900 suena razonable).

Otro ajuste importante razona la histéresis

Donde para evitar que los motores (bomba, rodillos, etc) se enciendan y apaguen rápidamente (“chattering”) justo en el límite del umbral se usa una histéresis.

Por ejemplo, en la humedad sin histéresis vemos que el umbral es 30, la humedad baja a 29.9 (Bomba ON), riega, sube a 30.1 (Bomba OFF), se seca a 29.9 (Bomba ON). Esto es malo.

En cambio, si lo vemos con histéresis donde tiene los siguientes parámetros

b) UMBRAL_BAJO = 30 (Encender bomba)

b) UMBRAL_ALTO = 40 (Apagar bomba)

Esto su lógica nos dice que si la humedad baja de 30, ENCIENDE la bomba. No debe apagarse hasta que la humedad haya subido por encima de 40. “Esto da un ancho de banda y protege al motor.

También dentro de los parámetros de temporización y actuación, consideramos lo siguiente:

LOOP_DELAY_SEGUNDOS (Tiempo de ciclo): que es el tiempo de espera al final de cada bucle, donde considera un ajuste siendo diseño crítico debido a que los procesos de composteo son lentos, entonces, la temperatura no cambia en 2 segundos, la lectura de los sensores y toma de decisiones cada 30 seg, 1 minuto o inclusive 5 minutos es mucho más razonable. Aquí es clave que las constantes de PID (KI Y KD) dependen directamente de este tiempo, además si se cambia el LOOP_DELAY_SEGUNDOS (ejemplo de 2 s a 30 s), será necesario volver a ajustar el PID.

Y con respecto a la duración de los actuadores de riego/movimiento, básicamente es cuando los actuadores de motores se activan, pero por ¿cuánto tiempo?, donde considera un ajuste en el cual no deben quedarse encendidos indefinidamente. Por ejemplo, en el caso de la bomba si usa una lógica donde la humedad es baja, enciende la bomba durante 15 seg y luego apagarse, no volver a comprobar hasta el próximo ciclo, este es un ajuste 100% empírico que dependerá de cuanta agua eche la bomba. Y así con el control de los demás motores

Parámetros de diseño del código completo

En este código completo esta el abandono del PID, donde el cambio más grande es reemplazar el PID por un control ON/OFF que es mucho más simple de programar y entender, sin embargo, el control es un poco menos preciso, la temperatura de la composta no se estabilizará en 50°C, sino que oscilará alrededor de 50 °C ((subirá a 50.1, se encenderá el ventilador, bajará a 49.9, se apagará, volverá a subir, etc.). Además, el sistema depende ahora de un reloj interno y del conteo de horas de luz. Significa que el ESP32 debe funcionar 24/7 sin reiniciarse. Donde si se reinicia a mediodía, el contador `planta_horas_luz_acumuladas` se irá a cero y la planta recibirá más luz de la debida. Y también cabe destacar que el diseño de “perfiles” es bueno. Es escalable, es decir, que se pueden añadir más plantas al diccionario) y permite al usuario final cambiar el comportamiento del sistema fácilmente, desde una app.

Por otro lado, el hecho de que ambos sistemas controlen un solo actuador (la cubierta) es un punto donde si la composta pide sombra (por luz intensa) pero el helecho aún no ha cumplido sus 4 horas de luz, la composta “gana” y el helecho se queda sin luz. La lógica OR actual prioriza siempre el “cerrar”

Dentro de sus ajustes críticos esta el hardware real de la calibración, tomando las siguientes consideraciones:

- **Ajuste 1: Umbrales de sensores:** Donde todos los valores deben ser calibrados experimentalmente, tomando ¿COMPOSTA_UMBRAL_GAS_ALTO = 400 es el valor correcto? ¿UMBRAL_LUZ_DIA = 300 es "luz de día" o "sombra"? Esto requiere mediciones en el sitio real.

- **Ajuste 2: Histéresis (Muy relevante):** Donde cabe mencionar que con el control ON/OFF, si el umbral es 50°C, el ventilador puede encenderse y apagarse rápidamente 50.1° -> ON, 49.9° -> OFF, 50.1° -> ON), llamándose como “aleteo” (flapping) y destruirá los reles y motores, por lo cual su solución es agregar la histéresis, donde en lugar de un solo umbral, se usen dos, por ejemplo: Encender el ventilador si la temperatura sube de 50°C, pero *no apagarlo* hasta que baje de 48°C. Debiéndose aplicar a los controles de humedad, gas y temperatura.
- **Ajuste 3: Persistencia de datos (Horas de luz):** Que podría llegar a suceder cuando el ESP32 se reinicia, el contador de horas de luz se pierde por lo cual su solución sería modificar el código para que guarde la variable `planta_horas_luz_acumuladas` en la memoria Flash (memoria no volátil) del ESP32, por ejemplo, cada 10 o 15 minutos. De esa forma al arrancar el ESP32 debe primero leer este valor de la memoria para saber dónde se quedó.
- **Ajuste 4: Tiempos de actuación:** Que es cuando la bomba de suelo se enciende, tarda un tiempo en el que el agua llegue al sensor HL-69. Entonces el código actual podría encender la bomba, leer de nuevo 1 segundo después, ver que sigue seco y mantenerla encendida, inundando el sistema. Por lo cual su ajuste de solución sería que cuando se active un actuador como la bomba, el código debe ignorar ese sensor por un tiempo (ej:5 minutos) para darle tiempo al sistema a reaccionar físicamente.

3.3.3 ALGORITMO

Los pseudocódigos detallados para composta, composta más plantas, se podrán encontrar en la parte de anexos (Títulos 9.4.2 – 9.2.3). A continuación se redacta descripción de cada uno.

3) Paso 1: Simulación de la “Física de la Planta”

Primero se tiene un pequeño “motor de física” (actualiza el estado simulado). Esta función es la que se encarga de fingir lo que está pasando dentro del compostador.

Así pues, simula que el compostaje genera calor por si solo (la temperatura sube 0.5 °C en cada ciclo, además simula que se seca lentamente (la humedad baja 0.1% en cada ciclo) y también simula que, si está caliente y no se mueve, se acumulan gases (MQ135 sube).

De esta forma, si nuestros actuadores como en el caso del ventilador o la bomba están encendidos, esta física reacciona. Por ejemplo, si el ventilador se enciende, la temperatura baja y el compostaje se seca más rápido, si la bomba se enciende, la humedad sube de golpe.

3) Paso 2: “Leer” los sensores (simulado)

Aquí cabe resaltar que una vez que la física se ha calculado estado real, por ejemplo, donde la temperatura es 45 °C, el código finge leer los sensores (simular_lecturas_sensores).

Destacando que, para hacerlo más realista, no lee ‘45’ exacto, sino que le añade un pequeño ruido aleatorio donde pudiera leer ‘44.9’ o ‘45.1’. de forma que simula cómo los sensores reales nunca son perfectos y/o ideales.

3) Paso 3: Tomar decisiones (“El cerebro”)

Aquí se encuentra el corazón del programa. Con esas lecturas simuladas, el código ejecuta cuatro lógicas de control independientes al mismo tiempo de la siguiente forma:

- A. **Control de humedad (Bomba):** Esta es una lógica un poco más simple de ‘ON/OFF’. Básicamente comprueba ¿la humedad promedio está por debajo de nuestro umbral (30%)?. Si la respuesta es que ‘Sí’, decide encender la bomba. Si es ‘No’, la mantiene apagada.
- B. **Control solar (Servo):** Aquí comprueba ¿El sensor de luz (LDR) detecta luz solar muy intensa (más de 900)?. Si es ‘Sí’, decide mover el servo a 90 grados para dar sombra. Si es ‘No’, lo regresa a 0.
- C. **Control de gases (rodillos):** Otra lógica de tipo ‘ON/OFF’, busca comprobar ¿El sensor de gases (MQ135) detecta una acumulación alta (¿más de 400 ppm?) Sí es ‘Sí’, significa que la composta necesita aire. Entonces, decide encender los rodillos para mover la tierra.
- D. **Control de Temperatura (PID):** Es el control más avanzado. Su objetivo es mantener la temperatura exactamente en 60 °C.
 - Para esto se usa un controlador PID. Un PID es como un conductor inteligente que no solo pisa el freno (el ventilador) cuando pasa de 60 °C., sino que calcula la fuerza exacta para frenar justo a tiempo y mantenerse estable en 60°C sin pasarse ni quedarse corto. Este PID nos da un valor de 0 a 100 que viene a representar “la potencia del ventilador” que considera que se requiere.

3) Paso 4: Combinar decisiones y Actuar

Finalmente, el código toma todas esas decisiones y las aplica. Se consideran un punto clave basado en que el ventilador tiene dos jefes principales.

- b) **PID de Temperatura** que puede pedir que se encienda (si la salida, PID es mayor a 10, por ejemplo).
- b) **Control de gases:** También puede pedir que se encienda (si los gases son altos, para ventilarlos).

Por lo que la lógica final es si el ventilador encenderá si el PID lo pide O si el control de gases lo pide.

Explicación de lógica de código dual de composta + plantas

El código simula el paso del tiempo, considerando factores clave como los siguientes:

- c) **Aceleración del tiempo:** Donde cada ciclo del bucle (que dura 1 segundo real) simula el paso de 10 minutos (MINUTOS_SIM_POR_CICLO). Esto nos permite probar la lógica de un día completo en pocas horas.
- c) **Ciclo día/noche:** Se simula una hora del día (sim_hora_día) y basándose en ella, se calcula una sim_luminosidad_global. Haciendo que el LDR “vea” luz durante el día y oscuridad durante la noche.
- c) **Contador de horas de luz:** El sistema lleva la cuenta de cuántas horas de luz solar ha recibido la planta (planta_horas_luz_acumuladas), un dato clave para controlar las plantas.

2. Simulación de la Física

Aquí en lugar de solo leer los sensores, el código actualiza el estado de la composta y las plantas basándose en la física y los actuadores. Por ejemplo:

- Si el estado_ventilador_compost está encendido, la sim_compost_temp bajará.
- Si el estado_bomba_planta está encendida, la sim_planta_hum_suelo bajará.
- Si no se hace nada, la composta genera calor y gases por si sola.

3. Lógica de control (el cerebro)

En esta sección es donde se toman las decisiones, divididas en tres partes principales:

A. Control de la Composta (Lógica Simple):

Se revisan 4 reglas de umbral directas:

1. **Humedad:** Si la humedad del suelo baja de 50%, se enciende la estado_bomba_compost.
2. **Gases:** Si los gases (MQ135) suben de 400 ppm, se enciende el estado_agitador_compost.
3. **Temperatura:** Si la temperatura sube de 50 °C, se enciende el estado_ventilador_compost.
4. **Luz:** Si la luz es muy intensa ($LDR > 700$), pide que se cierre la cubierta de sombra.

B. Control de las plantas (lógica de perfiles):

Esta sección es más avanzada, ya que primero debe seleccionarse un perfil (“helecho”, “tomate”, etc.), luego aplica las reglas de ese perfil:

1. **Humedad suelo:** si baja del mínimo del perfil (ejemplo: 65% del helecho), se enciende la estado_bomba_planta.
2. **Ambiente:** Si la temperatura O la humedad del aire superan los máximos del perfil, se enciende el estado_ventilador_planta.
3. **Luz Acumulada:** Si las planta_horas_luz_Acumuladas superan el máximo del perfil (ej: 4 horas para el helecho), pide que se cierre la cubierta.

C. Control del actuador compartido (la cubierta)

La cubierta (que el diseño físico es el servomotor) es un recurso compartido, la decisión final se basa en una lógica OR de la siguiente forma:

La cubierta se cerrará (estado_cubierta_cerrada=True) SI la composta pide sombra O SI la planta pide sombra.

4. Salida para el Plotter

Al final del bucle, se imprimen 5 valores numéricos clave (temperaturas y humedades de ambos sistemas) en una sola línea, para que el plotter de thonny pueda graficarlos en tiempo real, ignorando cualquier otro texto de diagnóstico.

3.3.3.1 Lenguaje de programación propuesto

El lenguaje de programación usado fue Python utilizando el entorno de Thonny. Donde Usar Python junto con el entorno Thonny para el proyecto EcoSyncra es una excelente elección porque permite desarrollar, simular y validar sistemas de control de forma accesible, visual y modular. Python ofrece una sintaxis clara y potente para implementar algoritmos como el PID, mientras que Thonny facilita la depuración paso a paso, la visualización en tiempo real mediante su plotter integrado y la conexión directa con microcontroladores como el ESP32. Esta combinación permite que el cerebro del sistema —incluyendo lógica de sensores, actuadores y simulaciones ambientales— se pruebe de manera segura y ajustable antes de pasar al hardware físico, acelerando el desarrollo iterativo y asegurando que cada decisión esté fundamentada en datos y comportamiento observable.

4 RESULTADOS

Para validar su funcionamiento, nos basamos en dos acciones principales mostradas a continuación, se imprime en la consola, donde en cada ciclo se ve un resumen de texto que enuncia la temperatura es X, la humedad es Y, se decide encender el ventilador y los rodillos. También se usa el plotter de thonny, donde este código imprime una línea especial de números, entonces, el plotter de thonny la lee y la convierte en una gráfica de tiempo real.

En esa gráfica se puede observar ver cómo la línea de Temperatura (simulada) sube desde los 25°C y cuando se acerca a los 60 °C, vemos como la línea del PID (el esfuerzo del ventilador) empieza a subir para ‘frenarla’, logrando que la temperatura es clave exactamente en nuestro objetivo de 60 °C. Así este script permite ajustar los umbrales y las constantes del PID en un entorno seguro antes de construir el sistema real.

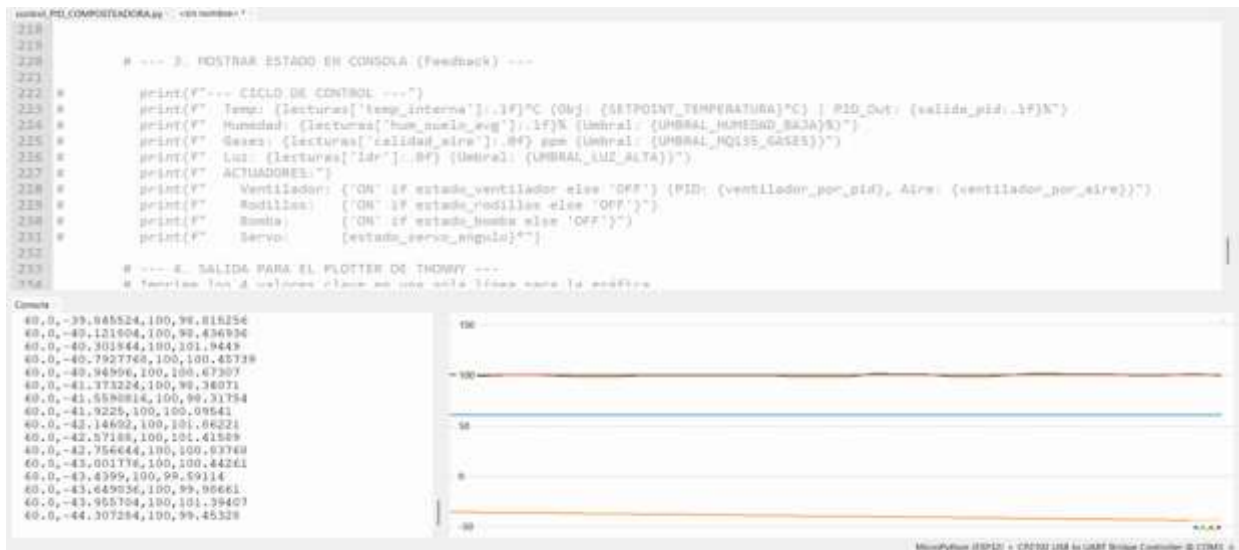


Ilustración 30. ANALISIS DE VALIDACIÓN DE ALGORITMOS DE COMPOSTA

A continuación, se muestra el resultado del código de la composta + las plantas ejecutado en el entorno de Thonny para Python.

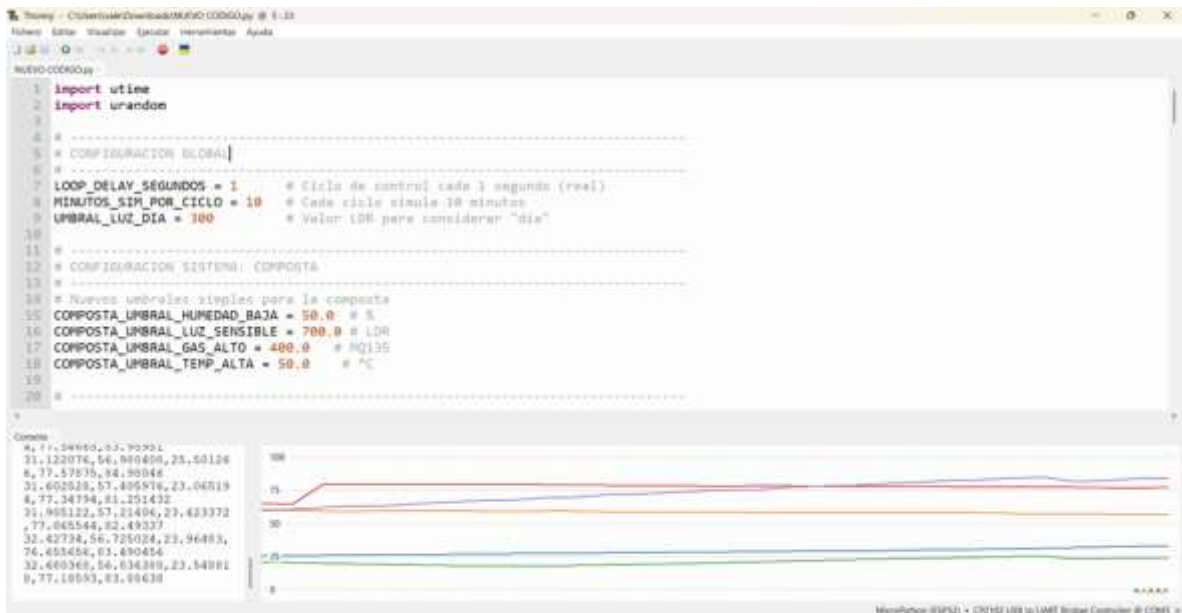


Ilustración 31. ANÁLISIS DE VALIDACIÓN DE CÓDIGO COMPLETO

La gráfica mostrada valida correctamente el funcionamiento del sistema de simulación, evidenciando cómo las variables de temperatura, humedad y luminosidad evolucionan con el paso del tiempo según las condiciones establecidas en el código. Se observa una tendencia gradual en las líneas que representan los sensores de la composta y de las plantas, lo cual confirma que las funciones de actualización física (actualizar_estado_compost()) y

actualizar_estado_plantas()) responden a los umbrales definidos y a la lógica de control de los actuadores. En particular, los incrementos y estabilizaciones en las curvas de temperatura y humedad reflejan el equilibrio dinámico entre los procesos de riego, evaporación y ventilación, validando así que los algoritmos de simulación reproducen comportamientos ambientales coherentes y que los actuadores se activan de forma adecuada frente a los cambios de las condiciones simuladas

a) Configuraciones necesarias para la operación del sistema

Para la operación del sistema ECOSYNCR v1.0, es fundamental realizar primero la configuración correcta de los pines de hardware definidos en el programa. Cada componente del sistema tiene un pin asignado: los sensores de humedad de suelo, temperatura y humedad (DHT11), gas CO₂ (MQ) y luminosidad (LDR) están conectados a entradas analógicas o digitales, mientras que los actuadores —servo, extractor, bomba de agua y motor mezclador— emplean salidas PWM o digitales. Durante la inicialización, los sensores se configuran para operar dentro del rango completo de lectura (0–3.3V) y los actuadores son preparados para responder a los comandos de control. Esta fase de configuración asegura que todas las señales físicas se interpreten correctamente y que las órdenes de salida se apliquen con precisión sobre los dispositivos conectados.

En la siguiente etapa, el sistema requiere establecer la calibración y los parámetros de operación. A través del menú de configuración (Set Up), el usuario puede acceder a funciones de prueba de sensores y motores, así como ajustar los valores de referencia ambientales según las necesidades de cada planta. Estos valores incluyen la humedad mínima del suelo, la luminosidad mínima, la temperatura máxima y el límite de CO₂, los cuales se almacenan en un archivo externo config.json. Este archivo actúa como una pequeña base de datos local que el programa principal lee y escribe dinámicamente, permitiendo modificar las condiciones de control sin reprogramar el sistema. La calibración previa garantiza que las mediciones sean fiables y que las respuestas del sistema (como activar la bomba o abrir el servo) sean oportunas y precisas.

Finalmente, el sistema se integra con un entorno de comunicación y monitoreo en tiempo real. La transmisión de datos de los sensores hacia el dashboard web se realiza mediante conexión serial, Wi-Fi o Bluetooth, enviando los valores en formato JSON para su

visualización en una interfaz HTML. Desde este panel, el usuario puede observar en tiempo real el comportamiento del entorno (humedad, temperatura, CO₂ y luz) y supervisar la activación automática de los actuadores. Este esquema de comunicación permite la gestión remota del sistema y facilita el análisis del rendimiento de las plantas, combinando la automatización física con la interacción digital en un ecosistema inteligente, modular y adaptable.

Para la prueba funcional del sistema EcoSyncra, se realizaron distintas verificaciones en el entorno de desarrollo Thonny, utilizando la conexión del ESP32 por medio de Wi-Fi y Bluetooth, con el fin de comprobar la correcta interacción entre sensores, actuadores y la interfaz de monitoreo tipo dashboard. Primero, se validó la lectura individual de sensores (DHT11, LDR, MQ-135 y sensor de humedad de suelo) mediante el submenú de calibración, observando que los valores en el terminal de Thonny respondieran a estímulos reales (como cambios de luz o humedad). Luego, se probó la activación de actuadores desde el submenú de prueba de motores, verificando el funcionamiento de la bomba de agua, el motor mezclador, el extractor y el servomotor en diferentes ángulos. Cada una de estas acciones se acompañó de mensajes en consola para confirmar el estado de los actuadores y su respuesta dentro de los tiempos programados en el pseudocódigo.

Posteriormente, se llevó a cabo la prueba integrada del bucle principal, en la que se seleccionó una planta configurada en el archivo config.json y se ejecutó la lógica automática de control. Durante esta etapa, el sistema reguló la humedad del suelo mediante la bomba, ajustó la iluminación con el servomotor, activó el extractor al detectar temperaturas o concentraciones elevadas de CO₂ y ejecutó la mezcla de composta cada hora simulada. En paralelo, se conectó el ESP32 a la red local para transmitir los datos en formato JSON hacia la interfaz de dashboard, adaptada en Thonny para mostrar gráficas en tiempo real de humedad, temperatura, luz y gases. Finalmente, se registró un video de funcionamiento mostrando el inicio del sistema, las lecturas dinámicas en pantalla y la respuesta física de los actuadores, demostrando que la comunicación, el control y la estabilidad del sistema operan correctamente en conjunto.

En lo que es el control, se realizó un código principal ejecutado en Micropython para un entorno de Thonny, en el cual se demostró arriba anteriormente para el funcionamiento de

nuestra interfaz, donde se utilizó la librería de json que es un script que se manda a llamar como se muestra a continuación.



Ilustración 32. BASE DE DATOS EN JSON DE PLANTAS

Donde se encuentran las bases de las plantas que el usuario puede seleccionar para plantar en el sistema de EcoSyncra, o en bien, puede agregar una nueva planta que no esté registrada añadiendo parámetros como nombre, humedad del suelo, luz mínima, temperatura mínima y dióxido.

Entonces de esta forma, las fallas que quizás podrían considerarse dentro del entorno de control es que no se caracterizaron los motores ya que se recomienda para menor posibilidad de error el caracterizar los motores donde al ser de DC, llevan ciertas especificaciones muy precisas como número de torque, rpm, peso, entre otras. Como área de mejora dentro del algoritmo de control podrían considerarse en vez de implementar un control PID con k_p , k_i y k_d como se implementó en el algoritmo de este sistema, se podría considerar usar un sistema difuso ya que se adapta mejor a variaciones, tolera imprecisiones y ofrece mayor estabilidad sin necesidad de un modelo matemático exacto pero cuidando justamente la caracterización de los motores para que funcione adecuadamente y puedan funcionar de forma dependientes unas de otras.

Y además al ejecutar el código si podemos ver como va variando los valores de los sensores en reconocer según la condición del umbral referente a la temperatura, humedad, CO2 y suelo y luz. Donde se puede apreciar como en el entorno de Thonny al regular el potenciómetro ungiendo como el MQ135 de gas, adecua el sistema

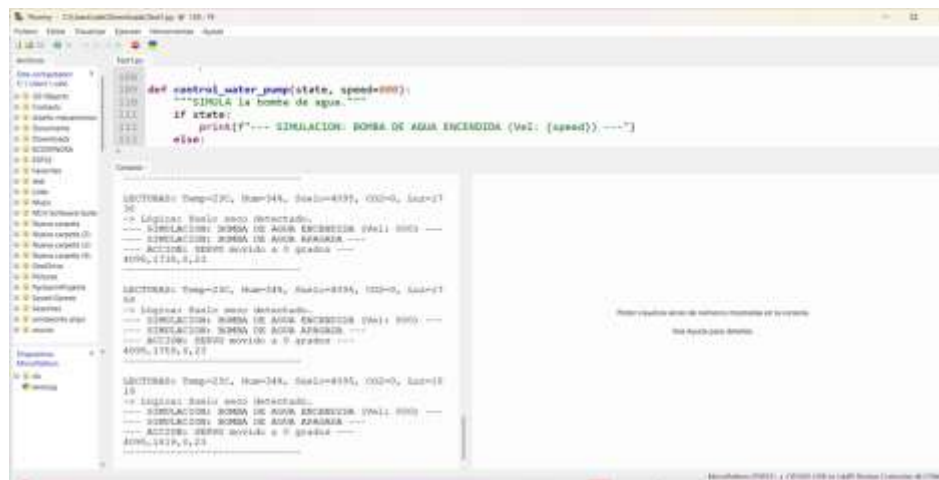
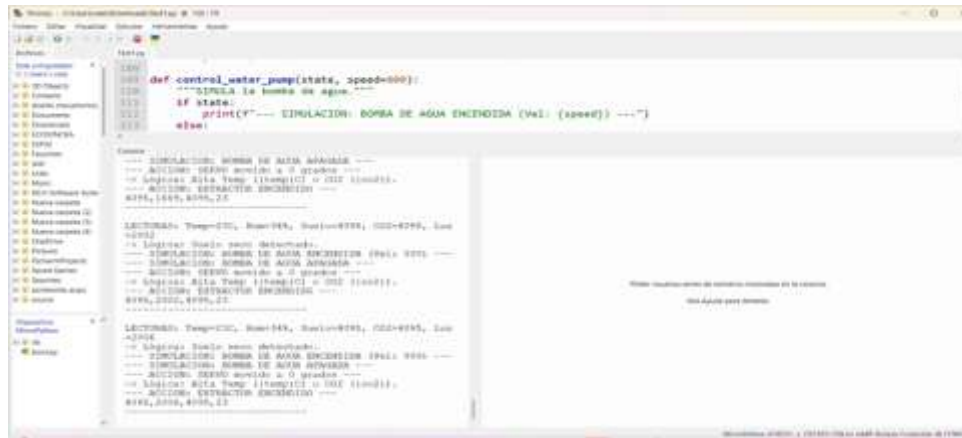


Ilustración 33. Prueba del extractor

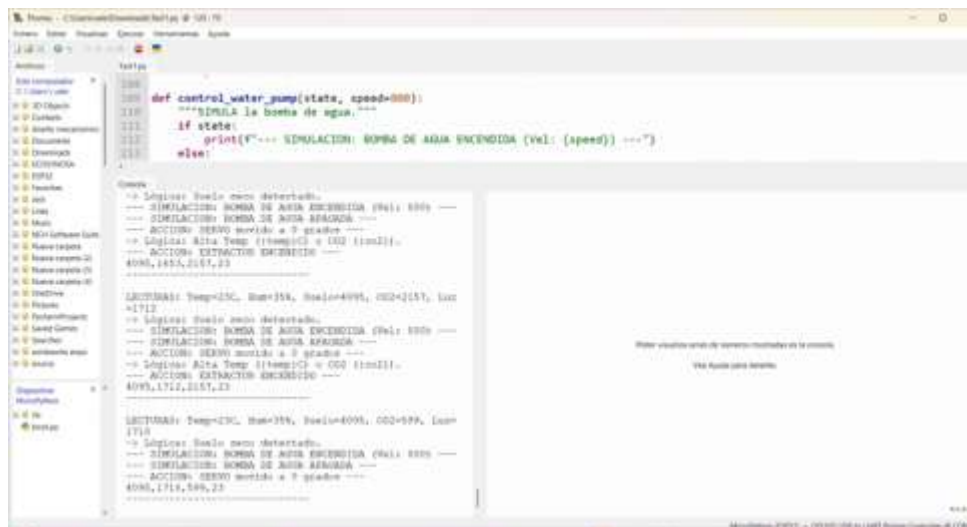


Ilustración 34. Apagado el extractor

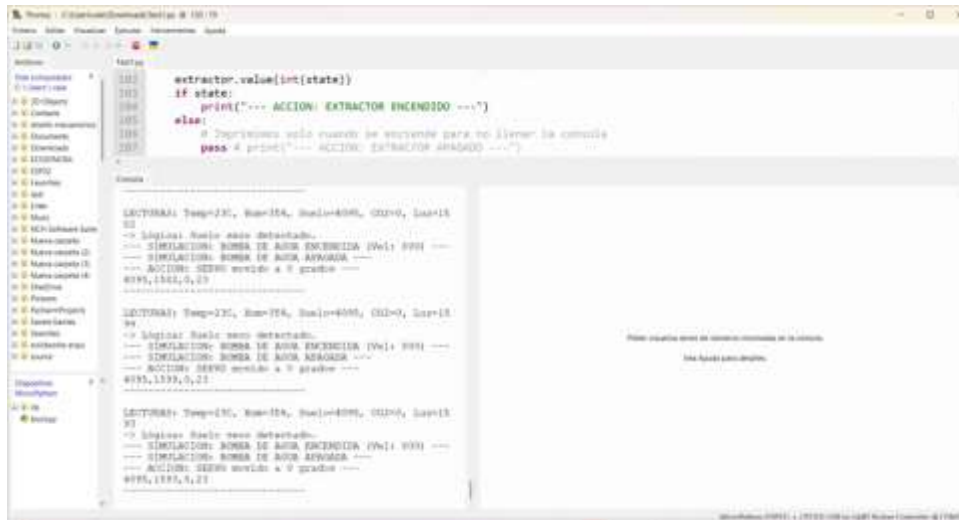


Ilustración 35. VALIDACIÓN DE PRUEBAS DE ACTUADORES

LINK DE VIDEO DE EVIDENCIA:

<https://youtu.be/pmmkZ5BTnYE?si=FCTj7hASXdjbJzXk>

4.1.1 PRUEBAS FINALES DE CONTROL

Este código fue creado como parte del dashboard web de EcoSyncra, diseñado para conectar el sistema físico con una interfaz digital accesible desde cualquier navegador. La idea principal es que los datos generados por el ESP32 —como lecturas de sensores o estados de actuadores— puedan visualizarse en tiempo real en una página web, ofreciendo a los usuarios una herramienta clara y dinámica para monitoreo y control.

Para lograrlo, se integraron cuatro módulos fundamentales: Express para levantar el servidor web y servir la interfaz, Socket.io para habilitar comunicación en tiempo real entre el navegador y el backend, SerialPort para establecer la conexión física con el microcontrolador, y ReadlineParser para organizar los datos entrantes y transmitirlos en formato estructurado. En conjunto, estos componentes permiten que el dashboard funcione como un puente directo entre el hardware y la visualización web, asegurando que la información fluya de manera rápida, confiable y comprensible.

La sección de configuración establece el puerto serial (COM3) y la velocidad de transmisión (115200 baudios). Al abrir el puerto, se utiliza un parser de líneas (ReadlineParser) que interpreta los datos entrantes del ESP32 línea por línea, facilitando su procesamiento.

Además, se imprime un mensaje en consola confirmando que la conexión física fue exitosa, lo que indica que el sistema está listo para recibir datos desde el hardware.

Este enfoque es ideal para sistemas de automatización como EcoSyncra, donde se requiere monitoreo en tiempo real, visualización de variables técnicas y respuesta inmediata ante eventos físicos. El código puede expandirse fácilmente para graficar datos, activar controles desde la interfaz o registrar eventos críticos, integrando el flujo físico-digital en un entorno accesible y escalable. Código en anexo (9.4.4)

El diseño incluye un panel de control estilizado, campos de entrada personalizados, botones interactivos y badges de estado que indican si un componente está encendido, apagado, abierto o cerrado. También se prepara un contenedor para medidores visuales (gauges-container) que pueden mostrar presión, temperatura u otras variables críticas. Este archivo es el punto de contacto visual entre el usuario y el sistema físico, y está pensado para ser intuitivo, responsivo y técnicamente robusto. (Código en anexo 9.4.5)

Dashboard en interfaz web

El dashboard web de EcoSyncra es una interfaz visual diseñada para monitorear y controlar en tiempo real los procesos físicos del sistema, como el estado de bombas, extractores, mezcladores, sensores de humedad, temperatura, luz y CO₂. Desde cualquier navegador, el usuario puede interactuar con el sistema mediante botones, menús desplegables y medidores gráficos, todo presentado en un entorno limpio, responsivo y técnicamente robusto. La interfaz permite seleccionar perfiles de cultivo (como tomate, menta o cactus), activar o desactivar componentes, y visualizar el estado actual de cada parte del sistema con indicadores tipo badge y gauges radiales.

Este dashboard se conecta directamente al microcontrolador (como el ESP32) mediante una arquitectura basada en Node.js.. Utiliza Express para servir la interfaz web, Socket.io para transmitir datos en tiempo real, y SerialPort para recibir información desde el hardware físico. Los datos estructurados que llegan por el puerto serial se interpretan y se reflejan instantáneamente en la interfaz, permitiendo al usuario tomar decisiones informadas y validar el comportamiento del sistema sin necesidad de herramientas externas.

Código principal de interfaz web para funcionamiento

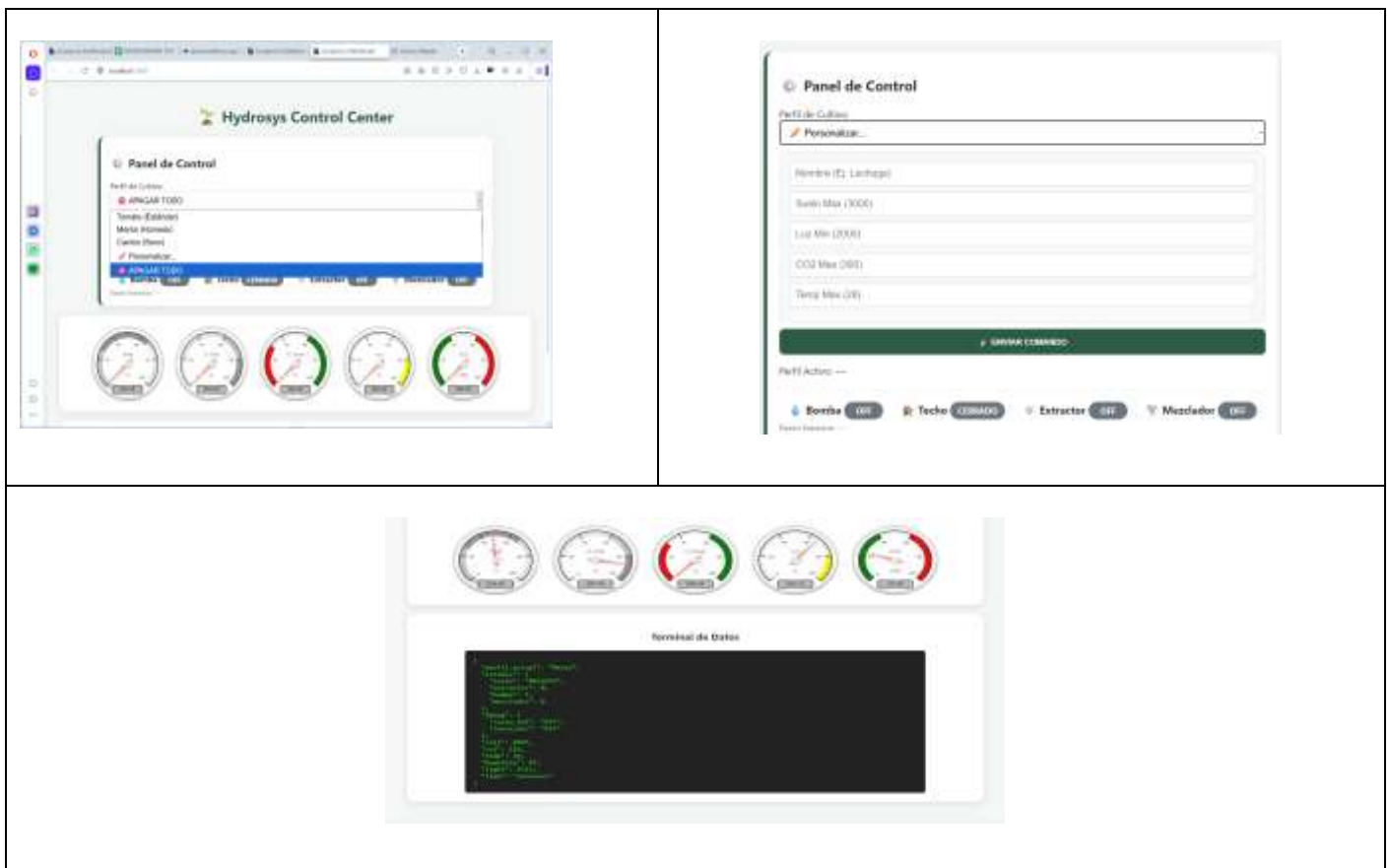
Este código escrito en **MicroPython** dentro de Thonny está diseñado para controlar y monitorear un sistema automatizado que integra sensores y actuadores. La primera parte define los pines de conexión de cada componente: sensores de humedad del suelo, temperatura y humedad (DHT11), gas/CO₂ (MQ), luz (LDR), además de actuadores como un servo para el techo, un extractor de aire y motores para bomba y mezclador. También se establecen constantes de calibración que permiten ajustar el comportamiento del servo y los tiempos de operación de cada dispositivo, asegurando que el sistema responda de manera estable y controlada.

En la sección de hardware se implementa la clase **MotorDriver**, que abstrae el control de los motores A y B mediante señales PWM y pines de dirección. Esto facilita la activación de la bomba de agua y del mezclador, permitiendo modular la velocidad y el sentido de giro. Además, se inicializan los sensores y actuadores, incluyendo el servo en posición de frenado y el extractor apagado, lo que garantiza que el sistema arranque en un estado seguro antes de comenzar la lógica principal.

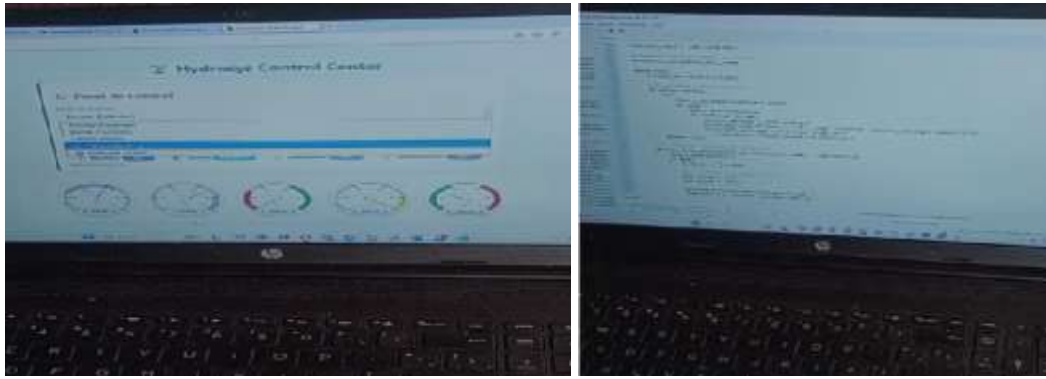
La función **read_sensors()** es clave porque centraliza la lectura de todos los sensores y devuelve valores de temperatura, humedad, humedad del suelo, concentración de CO₂ y nivel de luz. Estos datos se convierten en la base para la toma de decisiones del sistema. El código también incluye la función **move_servo_duty()**, que ajusta directamente la posición del servo según el valor de duty cycle, controlando la apertura o cierre del techo.

En el bucle principal (**main_loop()**) se gestionan tres aspectos: la recepción de comandos por entrada serial (lo que permite cambiar perfiles de operación desde la interfaz), la lectura periódica de sensores y el control de actuadores según condiciones definidas. Por ejemplo, si el CO₂ excede el límite, se activa el extractor y el mezclador; si la humedad del suelo es baja, se enciende la bomba de riego; y si la luz supera un umbral, el servo abre el techo. Todo esto se reporta en formato **JSON**, lo que facilita la integración con un dashboard en Visual Studio Code ya que la interfaz puede interpretar estos mensajes y mostrar estados en tiempo real.

Finalmente, el sistema incluye temporizadores que aseguran que los actuadores no permanezcan activos indefinidamente: la bomba se apaga tras unos segundos, el techo se detiene después de completar su movimiento, y el mezclador se activa periódicamente o por niveles altos de CO₂. Esta lógica garantiza seguridad, eficiencia energética y un control autónomo del prototipo. Al ejecutar este código en Thonny se obtiene la lectura y reconocimiento de sensores y actuadores, y al conectarlo con Visual Studio se puede construir un **dashboard interactivo** que muestre gráficamente los datos y estados, permitiendo al usuario supervisar y ajustar el sistema de manera intuitiva.



Evidencias fotográficas de funcionamiento



Video: [2Funcionamiento_ECOSYNCR_A F0U3_010_N4_ProyectoIMTC.mp4 - Google Drive](#)

[3Funcionamiento_ECOSYNCR_A F0U3_010_N4_ProyectoIMTC.mp4 - Google Drive](#)

5 IMPLEMENTACIÓN

5.1.1 PROCESO DE ENSAMBLAJE FÍSICO

Para la construcción física del prototipo, se emplearon principalmente dos materiales que responden tanto a criterios funcionales como de sostenibilidad. La maceta fue fabricada en madera, aprovechando su resistencia estructural y compatibilidad con el ambiente agrícola. Por otro lado, las demás piezas del sistema —incluyendo soportes, carcasa electrónica, mecanismos móviles y componentes de fijación— fueron producidas mediante impresión 3D utilizando filamento PLA, un material que permite alta precisión en el ensamblaje y facilita la iteración del diseño. Esta combinación de materiales asegura un equilibrio entre robustez, adaptabilidad y compromiso ambiental.

A continuación, se describen los pasos que se llevaron a cabo para realizar el ensamblaje físico del sistema EcoSynkra. Este proceso incluyó la integración secuencial de cada componente estructural, desde la base y la maceta hasta los elementos impresos en 3D, asegurando su correcta disposición y fijación. La documentación del montaje permite visualizar cómo se consolidó el prototipo completo, validando su funcionalidad mecánica y su compatibilidad con el sistema electrónico y de control.

1. PASO 1: Adquisición de materiales

A continuación, se presenta una tabla con los materiales que se usaron para llevar a cabo este proceso:

CATEGORÍA	ELEMENTO	MATERIAL/TIPO	FUNCIÓN PRINCIPAL
Estructura	Marco principal	PLA (impresión 3D)	Soporte general del mecanismo de la malla y fijación de piezas
Contenedor	Maceta	Madera	Recipiente para sustrato y planta
Contenedor	Placa de la base	Acrílico	Funciona como placa de la base que crea la separación entre el tanque y la planta.
Contenedor	Malla de filtro	Tela	Previene que la tierra y/o composta no se filtre hacia el tanque de agua
Contenedor	Tornillos	Acero	Soporte para los ensambles de cada parte de la maceta.
Mecanismo	Mecanismo de cortina	PLA (impresión 3D)	Apertura y cierre de malla que ayuda y/o impide el paso de luz solar
Sistema de riego	Contenedor	Plástico	Sirve como almacenamiento para el agua destinada al riego del sustrato y/o planta
Sistema de riego	Manguera de plástico	Plástico	Transporta el agua desde el depósito hasta los puntos de riego, asegurando que llegue de manera eficiente a la planta y/o sustrato.
Herramientas	Cortadora	Elementos varios	Realizar cortes precisos en las piezas de madera
	Segueta y regla		Medir y marcar las dimensiones exactas antes de cortar y cortar piezas pequeñas o realizar ajustes finos en la madera
	Taladro		Realizar perforaciones en la madera
	Destornillador		Fijar los tornillos en las uniones y en las escuadras metálicas

2. PASO 2: Corte de madera para la maceta (Dimensiones: 60 cm x 40 cm x 50 cm)



Ilustración 36. CORTE DE MADERA, PASO 2 IMPLEMENTACIÓN

Trazado: Se dibujan sobre la madera las dimensiones necesarias para cada parte:

- Tres rectángulos de 60 cm de ancho x 50 cm de alto (corresponden a la parte frontal, trasera y base).
- Dos rectángulos de 40 cm de ancho x 50 cm de alto (corresponden a los laterales).

Corte: Con las medidas ya trazadas, se utiliza la herramienta adecuada (sierra eléctrica) para realizar cortes precisos, asegurando que las piezas queden rectas y con bordes limpios. Esto es fundamental para que el armado posterior sea estable.

3. Paso 3: Inicio del armado y verificación del ajuste



Ilustración 37. INICIO DE ARMADO Y AJUSTES

Con las piezas ya cortadas, se comienza a presentar y unir las partes para dar forma a la maceta, tal como se observa en la imagen:

- **Ensamble inicial**

Se colocan la base y dos laterales, verificando que las piezas encajen correctamente y que los ángulos estén alineados.

- **Comprobación**

Antes de fijar con tornillos o clavos, se revisa que la estructura sea estable y que las dimensiones coincidan con el diseño.

- **Ajustes**

Si alguna pieza no encaja bien, se realizan pequeños rebajes o correcciones en los bordes.

4. Paso 4: Mediciones, trazado y corte de adecuaciones para la maceta



Ilustración 38. MEDICIONES, TRAZADO Y CORTES DE MACETA *Ilustración 39. ADECUACIONES MECÁNICAS DE MACETA*

En este paso se realizan las adaptaciones necesarias para que la maceta cumpla con su función y tenga un acabado adecuado. Las acciones incluyen:

- **Medición y trazado de perforaciones para drenaje**

En la base de la maceta, se marcan los puntos donde se realizarán los orificios para permitir la salida del agua.

- **Ajustes para uniones y refuerzos**

Se señalan las áreas donde se colocarán tornillos, clavos o escuadras metálicas para reforzar la estructura. Esto asegura que la maceta sea resistente y estable.

- **Trazado de case de PCB**

Se realizan el trazado de la case de PCB, se marca en la madera de acuerdo a las dimensiones.



Ilustración 40. ADECUACIONES DE LA MACETA

En este paso se llevan a cabo las adecuaciones finales que darán funcionalidad y estética a la maceta. El proceso incluye:

- **Perforaciones para el sistema de drenaje**

Se marcan y perforan varios orificios en la base de la maceta utilizando un taladro con broca para madera. Estos agujeros permiten la salida del exceso de agua, evitando encharcamientos y protegiendo las raíces.

- **Aperturas para el extractor de aire (PCB)**

En la superficie designada para la instalación del extractor, se trazan las medidas exactas y se perfora o recorta la madera para crear el espacio donde se colocará el componente. Esto asegura una correcta ventilación interna.

- **Adecuaciones adicionales**

- Ranuras o rebajes para pasar componentes.
- Orificios para fijar soportes, escuadras o refuerzos internos.

5. Paso 6: Colocación de las sujeciones para la placa de la base

En este paso se instalan los soportes que permitirán fijar la placa inferior de la maceta de manera segura. El procedimiento consiste en:

- **Ubicación de las escuadras metálicas**

Se colocan varias escuadras en las paredes internas de la estructura, distribuidas estratégicamente para sostener la base.



Generalmente se instalan dos escuadras por lado para garantizar estabilidad.

- **Fijación**

Cada escuadra se asegura con tornillos, utilizando un taladro o destornillador, procurando que queden firmes y alineadas. Esto permitirá que la placa de la base se apoye correctamente sobre ellas.

6. PASO 7: ENSAMBLE DE LA MACETA



Para completar la estructura principal, se procede a colocar la última cara de la maceta, asegurando que todo el sistema quede cerrado y estable. Este panel cumple varias funciones importantes:

- **Refuerzo interno**

En la parte interior, la cara final incluye dos soportes adicionales que ayudan a dar firmeza a la placa de la base, garantizando que pueda sostener el peso del sustrato y el agua.

- **Integración del sistema electrónico**

En el exterior, esta pieza ya cuenta con la caja para la PCB (placa de circuito), la cual será utilizada para controlar el sistema de riego automatizado.

- **Abertura funcional**

Se incorpora una ventana o abertura diseñada para colocar el contenedor que almacenará el agua, elemento esencial para el funcionamiento del riego.

Este paso marca la finalización de la estructura base de la maceta, dejando lista la instalación para los componentes internos y el sistema de riego.

Paso 8: Colocación de la cortina impresa en 3D para el mecanismo de la luz.



Ilustración 41. COLOCACIÓN DE LA CORTINA

- **Ensamble de unión de piezas de cortina**

Aquí se imprimieron dos uniones más grandes que se tuvieron que encajar para unir las y formar el marco de la cortina que posteriormente con tornillos y ensambles de acrílico se ajustó a la maceta, como se muestra a continuación.

PASO 9: COLOCACIÓN DE LA MANGUERA DE AGUA DE 5V



Ilustración 42. COLOCACIÓN DE LA MANGUERA

Aquí se hizo una perforación y con un sostén se colocó la manguera como se muestra para que pasara por la parte superior de la maceta y se le colocó una llave para abrirla y cerrarla.



PASO 10: COLOCACIÓN DE LA MALLA SOBRE LA CORTINA

Aquí se colocó la malla que corresponde a la cortina pegándose con silicón.

PASO 11: COLOCACIÓN DEL TUBO DE COMPOSTA DEL MOTOR MEZCLADOR

En este paso va en cierto modo conectado con la electrónica porque se hace el incrustado del tubo el cual se le colocan tornillos para que pueda girar más fácilmente en la zona inferior.



Ilustración 43. COLOCACIÓN DEL TUBO DE COMPOSTA

Este paso marca la finalización de la estructura base de la maceta, dejando lista la instalación para los componentes internos y el sistema de riego.

PASO 12: COLOCACIÓN DE LA ELECTRÓNICA

En este paso se procede a colocar los sensores y actuadores, donde primero una vez ya todo conectado en la tarjeta que en este caso se usó un adaptador de tarjeta de ESP32 y una tarjeta perforada debido a que se tuvo un inconveniente de conexión, por lo cual se implementó con ayuda de esta alternativa, donde una vez conectado el esp32 en la tarjeta de forma similar a esta.



Ilustración 44.COLOCACIÓN DE LA ELECTRÓNICA DE LA PCB DEL CEREBRO PRINCIPAL DEL ESP32 Y LOS SENSORES



Ilustración 45.COLOCACIÓN DE LOS ACTUADORES

La pcb se colocó en la case previamente diseñada e impresa en PLA, además los sensores se colocaron cerca de la planta y los actuadores en el lugar designado en el diseño CAD.

PASO 13: PLANTAR LA PLANTA (MENTA)



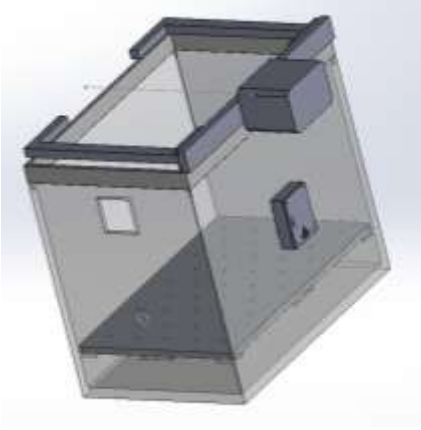
Ilustración 46.COLOCACIÓN DE PLANTA

En este último paso procedimos a plantar la planta que como primera muestra es la menta con la tierra necesaria en nuestra maceta.

PASO 14: PRUEBAS DE FUNCIONAMIENTO EN CONJUNTO

Aquí se hacen las verificaciones del funcionamiento de nuestro sistema.

- **COMPARATIVA ENTRE ENSAMBLE CAD VS FÍSICO**

Diseño CAD	Ensamble físico
	
El nuestro diseño en CAD presenta una estructura precisa y bien planificada, lo que permite visualizar la distribución interna antes de la fabricación. Se observa una base perforada para filtrado, soportes internos estratégicamente ubicados para sostener la placa base, y una caja exterior	La estructura física está elaborada en madera contrachapada, lo que aporta resistencia, durante el armado se realizaron ajustes para compensar tolerancias, se agregaron los soportes internos tal como en el CAD para

destinada a la PCB, que será el centro del sistema electrónico. Además, incluye una abertura específica para el contenedor de agua, asegurando la funcionalidad del sistema de riego. Este modelo digital ofrece ventajas como la posibilidad de prever la ubicación de cada elemento, optimizar el espacio y reducir errores en cortes y ensamblaje, sirviendo como guía fundamental para la construcción física

asegurar la placa de la base que será colocada. La abertura para el contenedor se ejecutó manualmente, lo que generó diferencias respecto al diseño CAD. Este proceso práctico permitió identificar necesidades reales que no se perciben en el diseño digital, como refuerzos adicionales y ajustes en las dimensiones.

5.1.2 COSTO TOTAL DEL PROTOTIPO

Componente	Costo Unitario	Cantidad	Total
Microcontrolador ESP32 DeVKIT	119	1	119
Servomotor MG995-RC	180	1	180
Motor DC 9V	85.5	1	85.5
Ventilador extractor 12v	107	1	107
Sensor ambiental DHT11	55	1	55
Sensor de humedad del suelo FC-28	40	1	40
Sensor de gas MQ-135	50	1	50
Driver de motores TB6612	175	1	175
Transistor de señal 2N222	50	1	50
PCB	25.76	5	128.8
Fuente de alimentación	250	1	250
Tablas de madera	66.66	6	399.96
Acrílico para placa de base	200	1	200
CasePCB en impresión 3D	150	1	150
Malla de filtro	100	1	100
Mecanismo de cortina en impresión 3D	1400	1	1400
Contenedor de plástico para tanque de agua	68	1	68

Manguera de plástico	20	1	20
Albahaca	100	1	100
Tierra fértil	160	1	160
Malla de filtro solar	200	1	200
Tarjetas perforadas	50	2	100
Tarjeta adaptadora de ESP32	600	1	600
Jumpers	60	1	60
Terminal block	10	8	80
Cautín	150	1	150
Base para cautín	120	1	120
Rollo de soldadura	150	1	150
Pin headers macho	30	2	60
Pin headers hembra	30	2	60
Resistencias	10 (1 paquete)	3	30
Tubo del movimiento del motor reductor con tornillos	120	1	120
Total estimado			\$ 5,568.26

5.1.3 VIDEO DEL PROTOTIPO FUNCIONAL

[Funcionamiento_ECOSYNCRA_AF0U3_010_N4_ProyectoIMTC.mp4 - Google Drive](#)

[2Funcionamiento_ECOSYNCRA_AF0U3_010_N4_ProyectoIMTC.mp4 - Google Drive](#)

[3Funcionamiento_ECOSYNCRA_AF0U3_010_N4_ProyectoIMTC.mp4 - Google Drive](#)

[4Funcionamiento_ECOSYNCRA_AF0U3_010_N4_ProyectoIMTC](#)

[5Funcionamiento_ECOSYNCRA_AF0U3_010_N4_ProyectoIMTC.mp4 - Google Drive](#)

6 CONCLUSIONES

1. Valeria Abigail González Sada 1963075

En el desarrollo de EcoSyncra logré consolidar un sistema que conecta directamente el hardware con un dashboard web, permitiendo personalizar perfiles de cultivo y visualizar en

tiempo real las variables críticas del entorno. Uno de los mayores logros fue que el usuario puede crear y adaptar una nueva planta dentro de la interfaz, ajustando parámetros según sus necesidades específicas. Esto me permitió comprobar que la arquitectura planteada funciona y que la personalización es posible de manera práctica y accesible.

Durante el proceso encontré limitaciones importantes, principalmente relacionadas con la necesidad de contar con un servicio de hosting para que el dashboard pueda estar disponible en la web abierta y no solo en red local. Esta restricción me mostró que, aunque la solución técnica es sólida, aún requiere infraestructura adicional para escalar hacia un uso más amplio y colaborativo. También identifiqué que la seguridad y la persistencia de datos serán aspectos clave a resolver en la siguiente etapa.

En el desarrollo de EcoSyncra logré combinar de manera efectiva el modelado CAD con la integración electrónica, ya que los diseños tridimensionales y diagramas de proceso me permitieron estructurar el prototipo con precisión y anticipar ajustes mecánicos, mientras que los sensores, controladores y actuadores se integraron en un esquema modular que garantizó la operatividad autónoma del sistema; esta sinergia entre la representación estructural y la dinámica electrónica fue clave para validar el desempeño estable del prototipo en condiciones reales y demostrar su potencial como solución doméstica sostenible.

En cuanto al cumplimiento de los objetivos, considero que se alcanzaron las metas principales: conectar el sistema físico con la interfaz digital, habilitar la personalización de cultivos y validar la comunicación en tiempo real. Sin embargo, reconozco que el despliegue completo en un entorno abierto aún está pendiente. Mi conclusión es que el proyecto sentó bases firmes y demostró su viabilidad, y ahora el siguiente paso será ampliar su alcance mediante hosting y servicios en la nube para que EcoSyncra pueda crecer y ser utilizado en escenarios reales de producción y validación.

2. Angie Sofia Hernández Tristán.

El proyecto EcoSyncra logró cumplir satisfactoriamente con el diseño e implementación de un sistema automatizado de autorriego y compostaje doméstico, validando la integración entre sensores, controladores y actuadores dentro de una arquitectura mecánica y electrónica funcional. Se alcanzaron los objetivos planteados mediante la correcta elaboración de

diagramas de flujo, el diseño y validación del modelo CAD, y la construcción del prototipo físico, demostrando un desempeño confiable en escenarios reales. Entre los logros destacan la organización estructural del sistema y su capacidad para optimizar el uso de recursos, mientras que las principales limitaciones se relacionaron con la necesidad de infraestructura adicional para ampliar el acceso remoto y mejorar la escalabilidad. En conjunto, EcoSyncra demuestra ser una solución viable y prometedora que integra sostenibilidad y tecnología inteligente, quedando preparada para futuras mejoras como servicios en la nube y funciones avanzadas de monitoreo.

3. Jesús Adrián Moreno Contreras

El desarrollo de EcoSyncra evidenció la posibilidad de crear un sistema doméstico capaz de gestionar de manera autónoma tanto el riego como el proceso de compostaje. A lo largo del proyecto se integraron correctamente los componentes electrónicos —sensores, controladores y actuadores— dentro de una propuesta mecánica funcional, lo que permitió validar su operatividad en pruebas prácticas. El trabajo incluyó la elaboración de modelos CAD y diagramas de proceso que guiaron la construcción del prototipo y contribuyeron a obtener un desempeño estable en condiciones reales.

El sistema mostró fortalezas notables, como una estructura bien organizada y un uso eficiente de recursos, características que refuerzan su aporte a prácticas sostenibles. No obstante, también se identificaron áreas susceptibles de mejora, particularmente en lo referente a la infraestructura necesaria para ampliar la conectividad remota y facilitar una futura escalabilidad.

En síntesis, EcoSyncra se perfila como una propuesta tecnológica viable y adaptable, que combina automatización y sostenibilidad, y que deja abierta la puerta a nuevas etapas de perfeccionamiento, especialmente mediante funciones avanzadas de monitoreo y la integración con plataformas en la nube.

4. Ingrid del Rosario García Sánchez

En base al desarrollo de nuestro proyecto, podemos afirmar que cumplimos con la idea de poder consolidar la automatización y la sustentabilidad en un solo sistema que aporte

beneficios reales tanto al ser humano como al medio ambiente. Participar en el desarrollo de EcoSyncra fue un reto importante para mí, especialmente porque gran parte de mi trabajo se centró en el diseño mecánico, un área que no era mi fuerte al inicio. A pesar de eso, pude cumplir con esta responsabilidad, aprendiendo y aplicando conceptos que permitieron dar forma a la estructura física del sistema y asegurar que los componentes electrónicos se integraran correctamente. Este proceso no solo me ayudó a crecer profesionalmente, sino que también me demostró que con dedicación es posible superar limitaciones y aportar valor real al proyecto.

EcoSyncra representa exactamente lo que queríamos mostrar, una base sólida para implementar tecnología en algo tan cotidiano como el cuidado de una planta. La idea es que, mediante la automatización de procesos como el riego y el compostaje, se logre un sistema más sustentable que ayude a optimizar recursos y facilite la vida de las personas. El proyecto no solo permite personalizar parámetros y monitorear variables en tiempo real, sino que también abre la puerta a prácticas más responsables y eficientes. Aunque todavía hay aspectos por mejorar, como la integración con servicios en la nube y la ampliación de la conectividad, los resultados obtenidos demuestran que EcoSyncra es una opción viable y adaptable para quienes buscan combinar tecnología y sostenibilidad en su día a día.

En conclusión, este proyecto sienta bases firmes para evolucionar hacia una solución completa, capaz de operar en entornos reales y transformar la manera en que se gestionan los cultivos domésticos, haciendo que algo tan básico como cuidar una planta se convierta en una práctica inteligente y sustentable.

7 SUGERENCIAS FUTURAS

Una propuesta clara de mejora para EcoSyncra es implementar un servicio de hosting en la nube que permita abrir el dashboard a usuarios externos, garantizando acceso remoto y multiusuario. Esto no solo resolvería la limitación actual de operar únicamente en red local, sino que también facilitaría la integración de funciones avanzadas como almacenamiento histórico de datos, análisis comparativo de cultivos y alertas automáticas vía correo o aplicaciones móviles. Con ello, el sistema podría escalar hacia un entorno colaborativo y profesional, útil tanto en producción agrícola como en investigación académica.

Una posibilidad es integrar módulos de energía renovable, como paneles solares o microturbinas, conectados al dashboard. Esto permitiría que el sistema no solo controle el entorno de cultivo, sino que también gestione su propio consumo energético y muestre métricas de sostenibilidad en tiempo real. Así, el usuario podría visualizar cuánta energía se produce y se utiliza, alineando el proyecto con objetivos de eficiencia y reducción de huella ambiental.

Otra extensión sería la incorporación de funciones de trazabilidad y blockchain, donde cada ajuste o dato del cultivo quede registrado en una cadena segura e inmutable. Esto abriría la puerta a aplicaciones en certificación agrícola, exportación y transparencia de procesos, garantizando que los parámetros de producción cumplen con estándares de calidad. Con ello, EcoSyncra no solo sería un sistema de control, sino también una herramienta de confianza para validación y auditoría en entornos productivos.

8 REVISIÓN BIBLIOGRÁFICA

- Veizaga Balta, W., Vega Torrejón, G., & Colque Ayaviri, J. C. (2025). Monitoreo y control de compostaje automatizado basados en microcontroladores Raspberry Pi, sensores de gases, temperatura, pH, humedad y placas de desarrollo. *Journal Boliviano de Ciencias*, 21(57), 75–93. <https://doi.org/10.52428/20758944.v21i57.1326>
- [2]. Morales Romero, R. G., Ruiz Aguilar, G., & Jacinto Alejo López, S. (2020). Impacto de automatización de riego: Caso de estudio. *Computación y Sistemas*, 24(2), 917–930. <https://doi.org/10.13053/cys-24-2-3412>
- [3]. Castro C., N. D., Chamorro F., L. E., & Viteri M., C. A. (2021). Una red de sensores inalámbricos para la automatización y control del riego localizado. *Revista de Ciencias Agrícolas*, 33(2), 106–116. <https://doi.org/10.22267/rcia.163302.57>
- [4]. López, R., & Rodríguez, J. (2019). Diseño de sistema de riego automatizado con sensores de humedad y temperatura para agricultura de precisión. *Revista Iberoamericana de Automática e Informática Industrial*, 16(1), 45–52. <https://doi.org/10.1016/j.riai.2018.10.004>

- [5]. Martínez, A., & Gómez, P. (2022). Evaluación de sensores ambientales en sistemas de compostaje inteligente. *Ingeniería y Desarrollo*, 40(1), 33–42. <https://doi.org/10.14482/inde.40.1.12345>
- [6]. Hayim De Vries, A. (2022). Integrated Composting And Edible Plant Growing Facility. ((WO 2022/150882 A1). Wasteplant Pty Ltd. <https://link.lens.org/r2wa07tanh>.
- [7]. Ferreiro Pazos, A. J. (2025). Plant feeding device (EP 4512235 A1). <https://link.lens.org/xwZWSFSz92>.
- [8]. Decarli. B. Thompson, S. (2019). CONTAINER SYSTEM AND METHOD OF USE FOR FARMING. (US10219447B1). <https://patentimages.storage.googleapis.com/f6/20/0b/974aa52968a660/US10219447.pdf>.
- [9]. Lu Zhengxiong. (2023). Automatic soil underground irrigation system and application thereof. (CN111587716B). <https://patentimages.storage.googleapis.com/e8/a5/ea/6bcb3b377838b7/CN111587716B.pdf>.
- [10]. Ugwuanyi, Christian & Amoke, Douglas & Ezike, Morris & Ogbuka, Cosmas. (2025). AUTOMATIC IRRIGATION SYSTEM. https://www.researchgate.net/publication/390844472_AUTOMATIC_IRRIGATION_SYSTEM.
- [11]. Sihombing, Ruspita & Azizah, Amiril & Arifin, Zainal & Sari, Wahyuni & Oscar, Agus & Putra, Pandhu. (2025). Design of Automatic Irrigation System For Post-Mining Land Reclamation. *JOIV : International Journal on Informatics Visualization*. 9. 65. 10.62527/joiv.9.1.2950. https://www.researchgate.net/publication/389602985_Design_of_Automatic_Irrigation_System_For_Post-Mining_Land_Reclamation.
- [12]. Adegboye, M. A., Ajao, L. A., Folorunso, T. A., Mamudu, A. Y., & Isah, A. S. (2017). Automatic fertilized-irrigation control and management system. *FUW Trends in Science and Technology Journal* (FTSTJ), <http://irepo.futminna.edu.ng:8080/jspui/handle/123456789/9530>.

- [13]. 2(2), 801-805. Z. FANGDONG, W. ZHIGUANG, J. SHAN, Z. XIAOFEI, D. YUHAO, Z. ZENGRUI, P. JIN, Y. JIALONG, S. HUIXIN, “Multi-terrain intelligent planter”, CN115843648A, Mar 28, 2023
- [14]. A. Dobrinsky, “Method for determining water stress in plants”, US 11,985,925 B2, Mar 12, 2024.
- [15]. [16]. G. Hansen, “Smart irrigation system”, US 12,144,295 B1, Nov 19, 2024 S. Chamorro, R. Asto, “Dispositivo de riego inteligente para plantas mediante el uso de sensores para determinar la necesidad de agua en las plantas”, PE2025-0857, Nov 11, 2024 [17]. H. SEUNG, L. SEOK, L. DONG L. HEE SU, K. MIN, “Smart planter system”, KR1020240006117, Jul 05, 2022.

ANTEPROYECTO Y EXAMEN MEDIO CURSO

- Carreon, R. (2024). Falta de áreas verdes en Zona Metropolitana de Nuevo León daña al medio ambiente. Reporte Indigo. <https://www.reporteindigo.com/nacional/Falta-de-areas-verdes-en-Zona-Metropolitana-de-Nuevo-Leon-dana-al-medio-ambiente-20240201-0014.html>
- National Institutes of Health. (2016). Las plantas: ¿nuestras socias en la salud? National Institutes of Health. <https://salud.nih.gov/recursos-de-salud/nih-noticias-de-salud/las-plantas-nuestras-socias-en-la-salud>

DISEÑO MECÁNICO Y/O ELECTRÓNICO

- Pombal, M.M. P.M.M. Á. (s.f.). Órganos vegetales. Raíz. Atlas de Histología Vegetal y Animal. https://mmmegias.webs.uvigo.es/2-organos-v/guiada_o_v_raiz.php
- Pombal, M.M. & Álvarez, P.M.M. (s.f.). Órganos vegetales. Tallo. Atlas de Histología Vegetal y Animal. https://www.mmmegias.webs.uvigo.es/2-organos-v/guiada_o_v_tallo.php.
- Pombal, M.M. & Álvarez, P.M.M. (s.f.). Órganos vegetales. Hoja. Atlas de Histología Vegetal y Animal. https://www.mmmegias.webs.uvigo.es/2-organos-v/guiada_o_v_hoja.php

DISEÑO CONTROL Y/O ALGORITMICO

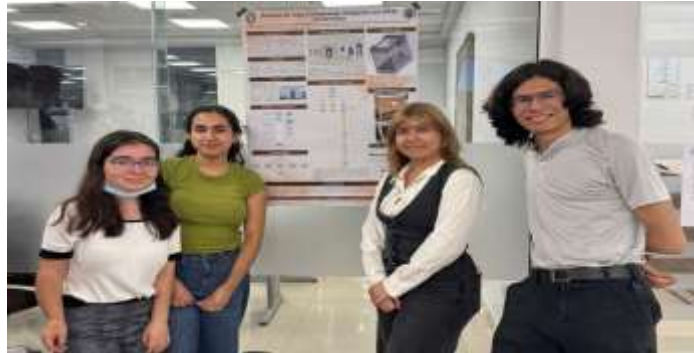
- Pombal, M.M. P.M.M. Á. (s.f.). Órganos vegetales. Raíz. Atlas de Histología Vegetal y Animal. https://mmmegias.webs.uvigo.es/2-organos v/guiada_o_v_raiz.php
- Pombal, M.M. & Álvarez, P.M.M. (s.f.). Órganos vegetales. Tallo. Atlas de Histología Vegetal y Animal. https://www.mmmegias.webs.uvigo.es/2-organos v/guiada_o_v_tallo.php
- Pombal, M.M. & Álvarez, P.M.M. (s.f.). Órganos vegetales. Hoja. Atlas de Histología Vegetal y Animal. https://www.mmmegias.webs.uvigo.es/2-organos v/guiada_o_v_hoja.php
- CACheME. (2014, 28 enero). Control PID con Python e interfaz web. CACheME. <https://www.cacheme.org/control-pid-con-interfaz-web-y-python>

IMPLEMENTACIÓN FÍSICA E INSTRUMENTACIÓN

- Sociedad Mexicana de Ingeniería Mecánica (SOMIM). (2022). Criterios de sustentabilidad en diseño preliminar y manufactura. Memorias del Congreso SOMIM 2022. Recuperado de https://somim.org.mx/memorias/memorias2022/articulos/A5_165.pdf
- Universidad Nacional Autónoma de México (UNAM). (2018). Propuesta y aplicación de un método de diseño mecánico sostenible. Tesis de Ingeniería. Recuperado de <http://www.ptolomeo.unam.mx:8080/jspui/bitstream/132.248.52.100/7629/3/Tesis.pdf>
- Universidad Politécnica Salesiana (UPS). (2023). Desarrollo de un banco de pruebas utilizando el chip ESP32 y sensores ambientales. Trabajo de titulación. Recuperado de <https://dspace.ups.edu.ec/bitstream/123456789/24026/1/UPS-GT004158.pdf>
- Instituto Politécnico Nacional (IPN). (2022). Implementación del módulo ESP32 como herramienta para el desarrollo de prácticas IoT. Revista de Ciencias de la Computación. Recuperado de https://rcs.cic.ipn.mx/2022_151_12/Implementacion%20del%20modulo%20ESP32%20como%20

9 ANEXOS

9.1 *Fotografía del equipo completo*



9.2 *Fotografías de los integrantes individuales*

1. Valeria Abigail González Sada (líder)



2. Angie Sofia Hernández Tristán



3. Jesús Adrián Moreno



4. Ingrid del Rosario García Sánchez



9.3 *Repartición de tareas del equipo*

1. Valeria Abigail González Sada: Se encargó del desarrollo de la parte de control para desarrollar y validar con las pruebas necesarias el sistema de control generando los códigos para el accionamiento de los sensores y actuadores, así como de la comunicación vía en tiempo real de la interfaz y dashboard web para monitoreo del sistema y sección de personalización que el usuario desee. También de implementación de la electrónica en cuanto a esquemáticos, pero mayormente al implementarlo ya vía física en el sistema para hacer las correcciones necesarias para el correcto funcionamiento. También estuvo a cargo del análisis de costos y de la cotización de materiales, además de la documentación de presentación de resultados y revisión de actividades. Además realizó y revisión de documentos y de documento de PIA.

2. Angie Sofia Hernández Tristán: Contribuyo al desarrollo del sistema mediante la implementación mecánica del diseño, la simulación y la validación del modelo CAD. Además, estuvo a cargo del análisis de costos y la cotización de materiales. Realizo la implementación mecánica del modelo físico, incluyendo la planificación de la ubicación de cada componente para establecer la base estructural física de la maceta, así como la definición, organización y validación final del documento.

3. Jesús Adrián Moreno: Contribuyó al diseño de la parte electrónica del proyecto, específicamente en el diseño funcional de la PCB, su cotización y adquisición, así como en la validación de los circuitos mediante pruebas simuladas. Además, participó en la implementación física, especialmente en el mecanismo del extractor.

3. Ingrid del Rosario García Sánchez: Colaboró en el diseño, validación y simulación de los modelos CAD del proyecto. Estuvo encargada de la redacción del proceso de implementación mecánica seguido. A cargo de la documentación para la presentación del proyecto (Poster) También brindó apoyo en una parte del montaje final del sistema.

9.4 Códigos y pseudocódigos utilizados.

9.4.1 Pseudocodigo

```
INICIO_PROGRAMA

// --- 1. Definición de Constantes y Pines ---
DEFINIR PIN_HUMEDAD_SUELO = 39
DEFINIR PIN_DHT11 = 21
DEFINIR PIN_MQ_CO2 = 34
DEFINIR PIN_LDR = 36
DEFINIR PIN_SERVO = 20
DEFINIR PIN_EXTRACTOR = 23
DEFINIR PIN_PWMA = 5
DEFINIR PIN_AIN1 = 7
DEFINIR PIN_AIN2 = 6
DEFINIR PIN_PWMB = 18
DEFINIR PIN_BIN1 = 9
DEFINIR PIN_BIN2 = 10
DEFINIR PIN_STBY = 8
DEFINIR INTERVALO_MEZCLA_MS = 3600000
DEFINIR DELAY_BUCLE_S = 5

// --- 2. Inicialización de Hardware (Objetos Globales) ---
// (Estos se inicializan una vez al arrancar)
DECLARAR dht_sensor COMO ObjetoDHT(PIN_DHT11)
DECLARAR soil_sensor COMO ObjetoADC(PIN_HUMEDAD_SUELO)
DECLARAR mq_sensor COMO ObjetoADC(PIN_MQ_CO2)
DECLARAR ldr_sensor COMO ObjetoADC(PIN_LDR)
DECLARAR servo COMO ObjetoPWM(PIN_SERVO)
DECLARAR extractor COMO ObjetoPinSalida(PIN_EXTRACTOR)
DECLARAR driver COMO ObjetoMotorDriver(PIN_PWMA, AIN1, AIN2, PWMB, BIN1, BIN2, STBY)
// Configurar ADC para rango completo (0-3.3V)
soil_sensor.configurar_atenuacion()
mq_sensor.configurar_atenuacion()
ldr_sensor.configurar_atenuacion()
ESCRIBIR "Hardware inicializado."

// --- 3. Definición de Clases (Abstraído) ---
CLASE PID
    PROPIEDADES: Kp, Ki, Kd, setpoint, _integral, _last_error
    FUNCIÓN update(valor_actual):
        // ... Lógica de cálculo PID ...
        RETORNAR salida_control
    FIN_FUNCIÓN
FIN_CLASE

CLASE MotorDriver
    PROPIEDADES: pines_ain, pines_bin, pines_pwm
    PROCEDIMIENTO control_motor_a(direccion, velocidad):
        // ... Lógica para pines AIN y PWMA ...
```

```

FIN_PROCEDIMIENTO
PROCEDIMIENTO control_motor_b(direccion, velocidad):
    // ... Lógica para pines BIN y PWMB ...
FIN_PROCEDIMIENTO
PROCEDIMIENTO stop_all():
    // ... Detener ambos motores ...
FIN_PROCEDIMIENTO
FIN_CLASE
// --- 4. Funciones de Lectura de Sensores ---
FUNCIÓN leer_dht11():
    INICIO-INTENTAR
        medir_dht()
        RETORNAR (temperatura, humedad)
    CAPTURAR-ERROR
        ESCRIBIR "Error DHT11"
        RETORNAR (NULO, NULO)
    FIN-INTENTAR
FIN_FUNCIÓN
FUNCIÓN leer_humedad_suelo():
    INICIO-INTENTAR
        RETORNAR soil_sensor.leer_valor_adc()
    CAPTURAR-ERROR
        ESCRIBIR "Error Suelo"
        RETORNAR NULO
    FIN-INTENTAR
FIN_FUNCIÓN
// (Funciones similares para leer_co2() y leer_luz())
// --- 5. Funciones de Control de Actuadores ---
PROCEDIMIENTO control_servo(angulo):
    duty_cycle = mapear(angulo, 0, 180, MIN_DUTY, MAX_DUTY)
    servo.set_duty(duty_cycle)
FIN_PROCEDIMIENTO
PROCEDIMIENTO control_extractor(estado):
    extractor.set_valor(estado) // (VERDADERO o FALSO)
FIN_PROCEDIMIENTO
PROCEDIMIENTO control_bomba_agua(estado, velocidad):
    SI estado ES VERDADERO ENTONCES
        driver.control_motor_a(1, velocidad) // 1 = adelante
    SINO
        driver.control_motor_a(0, 0) // 0 = frenar
    FIN_SI
FIN_PROCEDIMIENTO
PROCEDIMIENTO control_motor_mezclador(estado, velocidad):
    SI estado ES VERDADERO ENTONCES
        driver.control_motor_b(1, velocidad)
    SINO
        driver.control_motor_b(0, 0)

```

```

FIN_SI
FIN_PROCEDIMIENTO
// --- 6. Funciones de Configuración (JSON) ---
FUNCIÓN cargar_config():
    INICIO-INTENTAR
        archivo = abrir("config.json", "leer")
        datos = decodificar_json(archivo)
        cerrar(archivo)
        RETORNAR datos
    CAPTURAR-ERROR
        ESCRIBIR "No se encontró config.json, creando nuevo."
        RETORNAR {} // Retorna diccionario vacío
    FIN-INTENTAR
FIN_FUNCIÓN
PROCEDIMIENTO guardar_config(datos_config):
    INICIO-INTENTAR
        archivo = abrir("config.json", "escribir")
        texto_json = codificar_json(datos_config)
        escribir_en_archivo(archivo, texto_json)
        cerrar(archivo)
        ESCRIBIR "Configuración guardada."
    CAPTURAR-ERROR
        ESCRIBIR "Error al guardar config.json."
    FIN-INTENTAR
FIN_PROCEDIMIENTO
// --- 7. Bucle de Control Principal ---
PROCEDIMIENTO bucle_principal(ajustes_planta):
    ESCRIBIR "Iniciando bucle de control para: " + ajustes_planta.nombre
    DECLARAR ultimo_tiempo_mezcla = tiempo_actual_ms()
    MIENTRAS VERDADERO:
        INICIO-INTENTAR
            // 1. Leer Sensores
            (temp, hum) = leer_dht11()
            suelo = leer_humedad_suelo()
            co2 = leer_co2()
            luz = leer_luz()
            // 2. Lógica de Control
            SI suelo > ajustes_planta.humedad_suelo_min ENTONCES
                ESCRIBIR "Regando..."
                control_bomba_agua(VERDADERO, 800)
                ESPERAR 2 segundos
                control_bomba_agua(FALSO, 0)
            FIN_SI
            SI luz < ajustes_planta.luz_min ENTONCES
                control_servo(90) // Abrir
            SINO
                control_servo(0) // Cerrar

```

```

FIN_SI
SI temp > ajustes_planta.temp_max O co2 > ajustes_planta.co2_max ENTONCES
    control_extractor(VERDADERO)
SINO
    control_extractor(FALSO)
FIN_SI
// 3. Control Periódico
DECLARAR ahora = tiempo_actual_ms()
SI (ahora - ultimo_tiempo_mezcla) > INTERVALO_MEZCLA_MS ENTONCES
    ESCRIBIR "Mezclando composta..."
    control_motor_mezclador(VERDADERO, 600)
    ESPERAR 15 segundos
    control_motor_mezclador(FALSO, 0)
    ultimo_tiempo_mezcla = ahora
FIN_SI
// 4. Transmitir Datos
ESCRIBIR (suelo, luz, co2, temp) // Para Plotter Thonny
// datos_json = crear_json(temp, hum, suelo, co2, luz)
// ESCRIBIR datos_json // Para Interfaz Web
ESPERAR DELAY_BUCLE_S segundos
CAPTURAR-INTERRUPCION_TECLADO
    ESCRIBIR "Bucle detenido por usuario."
    SALIR BUCLE // Sale del MIENTRAS VERDADERO
CAPTURAR-ERROR (e)
    ESCRIBIR "Error en bucle principal: " + e
    ESPERAR 10 segundos
FIN-INTENTAR
FIN_MIENTRAS
FIN_PROCEDIMIENTO
// --- 8. Sistema de Menús ---
PROCEDIMIENTO menu_calibracion_sensores():
    MIENTRAS VERDADERO:
        ESCRIBIR "--- Sub-Menú: Calibración ---"
        ESCRIBIR "1. Leer Humedad Suelo"
        ESCRIBIR "2. Leer DHT11"
        ESCRIBIR "3. Leer CO2"
        ESCRIBIR "4. Leer Luz"
        ESCRIBIR "5. Leer TODOS (Bucle 10s)"
        ESCRIBIR "R. Regresar"
        opcion = LEER_ENTRADA_USUARIO()
        SI opcion == "1" ENTONCES ESCRIBIR "Valor Suelo: " + leer_humedad_suelo()
        SINO SI opcion == "2" ENTONCES ESCRIBIR "DHT: " + leer_dht11()
        SINO SI opcion == "3" ENTONCES ESCRIBIR "Valor CO2: " + leer_co2()
        SINO SI opcion == "4" ENTONCES ESCRIBIR "Valor Luz: " + leer_luz()
        SINO SI opcion == "5" ENTONCES
            ESCRIBIR "Leyendo sensores... (Ctrl+C para parar)"
            PARA i DESDE 1 HASTA 5:

```

```

        ESCRIBIR (leer_humedad_suelo(), leer_dht11(), leer_co2(), leer_luz())
        ESPERAR 2 segundos
    FIN_PARA
    SINO SI opcion == "R" ENTONCES
        SALIR BUCLE
    FIN_SI
FIN_MIENTRAS
FIN_PROCEDIMIENTO
PROCEDIMIENTO menu_prueba_motores():
    MIENTRAS VERDADERO:
        ESCRIBIR "--- Sub-Menú: Prueba Motores ---"
        ESCRIBIR "1. Probar Bomba Agua (3s)"
        ESCRIBIR "2. Probar Mezclador (3s)"
        ESCRIBIR "3. Probar Extractor (3s)"
        ESCRIBIR "4. Probar Servo (0-90-180-0)"
        ESCRIBIR "R. Regresar"
        opcion = LEER_ENTRADA_USUARIO()
        SI opcion == "1" ENTONCES
            control_bomba_agua(VERDADERO, 800); ESPERAR 3s; control_bomba_agua(FALSO, 0)
        SINO SI opcion == "2" ENTONCES
            control_motor_mezclador(VERDADERO, 600); ESPERAR 3s; control_motor_mezclador(FALSO, 0)
        SINO SI opcion == "3" ENTONCES
            control_extractor(VERDADERO); ESPERAR 3s; control_extractor(FALSO)
        SINO SI opcion == "4" ENTONCES
            control_servo(0); ESPERAR 1s; control_servo(90); ESPERAR 1s; control_servo(180); ESPERAR 1s;
control_servo(0)
        SINO SI opcion == "R" ENTONCES
            SALIR BUCLE
        FIN_SI
    FIN_MIENTRAS
FIN_PROCEDIMIENTO
PROCEDIMIENTO menu_setup():
    MIENTRAS VERDADERO:
        ESCRIBIR "--- Menú Set UP ---"
        ESCRIBIR "1. Calibración de sensores"
        ESCRIBIR "2. Prueba de motores"
        ESCRIBIR "3. Regresar a menú Principal"
        opcion = LEER_ENTRADA_USUARIO()

        SI opcion == "1" ENTONCES LLAMAR menu_calibracion_sensores()
        SINO SI opcion == "2" ENTONCES LLAMAR menu_prueba_motores()
        SINO SI opcion == "3" ENTONCES SALIR BUCLE
    FIN_SI
    FIN_MIENTRAS
FIN_PROCEDIMIENTO
PROCEDIMIENTO menu_manejo_plantas(referencia_config_data):
    MIENTRAS VERDADERO:

```

```

ESCRIBIR "--- Menú Manejo de Plantas ---"
ESCRIBIR "1. Ver plantas actuales"
ESCRIBIR "2. Guardar nueva planta"
ESCRIBIR "3. Regresar al menú principal"
opcion = LEER_ENTRADA_USUARIO()
SI opcion == "1" ENTONCES
    PARA CADA (id_planta, datos_planta) EN referencia_config_data:
        ESCRIBIR id_planta + ": " + datos_planta.nombre
        ESCRIBIR "  Setpoints: " + datos_planta
    FIN_PARA
SINO SI opcion == "2" ENTONCES
    ESCRIBIR "--- Agregar Nueva Planta ---"
    id_unico = LEER_ENTRADA_USUARIO("ID (ej: 'tomate'):" )
    nombre = LEER_ENTRADA_USUARIO("Nombre: ")
    hum_min = LEER_ENTRADA_NUMERICA("Hum. Suelo Min (ADC): ")
    luz_min = LEER_ENTRADA_NUMERICA("Luz Min (ADC): ")
    temp_max = LEER_ENTRADA_NUMERICA("Temp Max (C): ")
    co2_max = LEER_ENTRADA_NUMERICA("CO2 Max (ADC): ")
    nueva_planta = {nombre, hum_min, luz_min, temp_max, co2_max}
    referencia_config_data[id_unico] = nueva_planta
    LLAMAR guardar_config(referencia_config_data)
SINO SI opcion == "3" ENTONCES
    SALIR BUCLE
FIN_SI
FIN_MIENTRAS
FIN_PROCEDIMIENTO
FUNCIÓN seleccionar_planta_para_iniciar(config_data):
    SI config_data ESTÁ VACÍO ENTONCES
        ESCRIBIR "Error: No hay plantas. Agregue una."
        RETORNAR NULO
    FIN_SI
    lista_keys_plantas = obtener_keys(config_data)
    MIENTRAS VERDADERO:
        ESCRIBIR "--- Seleccionar Planta ---"
        PARA i DESDE 0 HASTA longitud(lista_keys_plantas) - 1:
            key = lista_keys_plantas[i]
            ESCRIBIR (i+1) + ": " + config_data[key].nombre
        FIN_PARA
        ESCRIBIR "R. Regresar"
        opcion = LEER_ENTRADA_USUARIO()
        SI opcion == "R" ENTONCES RETORNAR NULO
    INICIO-INTENTAR
        indice_num = convertir_a_numero(opcion) - 1
        SI 0 <= indice_num < longitud(lista_keys_plantas) ENTONCES
            key_seleccionada = lista_keys_plantas[indice_num]
            RETORNAR config_data[key_seleccionada]
        SINO

```

```

        ESCRIBIR "Opción inválida."
    FIN_SI
    CAPTURAR-ERROR
        ESCRIBIR "Opción inválida."
    FIN-INTENTAR
    FIN_MIENTRAS
    FIN_FUNCIÓN
// --- 9. Punto de Entrada Principal ---
PROCEDIMIENTO menu_principal():
    DECLARAR config_data = cargar_config()
    MIENTRAS VERDADERO:
        ESCRIBIR "===== MENÚ PRINCIPAL ECOSYNCR v1.0 ====="
        ESCRIBIR "1. Set UP (Calibrar, Probar)"
        ESCRIBIR "2. Manejo de plantas (Ver, Agregar)"
        ESCRIBIR "3. Iniciar Dispositivo"
        ESCRIBIR "===== "
        opcion = LEER_ENTRADA_USUARIO()
        SI opcion == "1" ENTONCES
            LLAMAR menu_setup()
        SINO SI opcion == "2" ENTONCES
            LLAMAR menu_manejo_plantas(config_data)
        SINO SI opcion == "3" ENTONCES
            DECLARAR ajustes = seleccionar_planta_para_iniciar(config_data)
            SI ajustes NO ES NULO ENTONCES
                LLAMAR bucle_principal(ajustes)
            ESCRIBIR "Regresando al menú principal..."
        FIN_SI
    SINO
        ESCRIBIR "Opción inválida."
    FIN_SI
    FIN_MIENTRAS
    FIN_PROCEDIMIENTO
// --- Ejecución ---
INICIO-INTENTAR
    LLAMAR menu_principal()
FINALMENTE
    // Asegurarse de que todo esté apagado si el programa termina
    ESCRIBIR "Apagando todos los actuadores."
    driver.stop_all()
    control_extractor(FALSO)
    control_servo(0)
    ESCRIBIR "Sistema detenido."
    FIN-INTENTAR
FIN_PROGRAMA

```

9.4.2 Pseudocodigo detallado composta

```
INICIO
    // --- Configuración ---
    INICIALIZAR todos los pines (sensores y actuadores).
    INICIALIZAR Sensor DHT11, Servo, Pantalla OLED.
    // --- Configuración del PID de Temperatura ---
    SETPOINT_TEMP = 60.0
    pid_temp = CREAR_PID(Kp, Ki, Kd, Setpoint=SETPOINT_TEMP, Limites=(0, 100))
    // --- Configuración de Umbrales ---
    UMBRAL_MQ135 = 400
    UMBRAL_HUMEDAD_BAJA = 30
    UMBRAL_LUZ_ALTA = 900
    // --- Bucle Principal ---
    MIENTRAS (Verdadero)
    // --- 1. Leer TODOS los sensores ---
    lecturas = leer_todos_los_sensores()
    // (temp_int, hum_int, temp_ext, hum_ext, h1, h2, h3, mq135, lluvia, ldr)
    // --- 2. Validar lecturas ---
    SI (lecturas_son_invalidas):
        mostrar_error_en_pantalla_oled("Error Sensores")
        CONTINUAR_BUCLE // Saltar este ciclo de control
    // --- 3. Ejecutar Lógicas de Control ---
    // Lógica 3: Humedad (Bomba)
    humedad_promedio = (lecturas.h1 + lecturas.h2 + lecturas.h3) / 3
    SI (humedad_promedio < UMBRAL_HUMEDAD_BAJA):
        activar_bomba(True, duracion=10) // Función que la enciende 10 seg
    // Lógica 4: Luz (Servo)
    SI (lecturas.ldr > UMBRAL_LUZ_ALTA):
        servo.mover_angulo(90)
    SINO:
        servo.mover_angulo(0)
        // --- Lógicas Combinadas para el Ventilador y Rodillos ---
        // Lógica 1: PID de Temperatura
        salida_pid = pid_temp.update(lecturas.temp_interna)
        // Lógica 2: Calidad de Aire
        ventilador_por_aire = False
        SI (lecturas.mq135 > UMBRAL_MQ135):
            activar_motor_rodillos(True)
            ventilador_por_aire = True
        SINO:
            activar_motor_rodillos(False)
    // --- Decisión Final del Ventilador ---
    // El ventilador se enciende SI el PID lo pide O SI el aire es malo
    SI (salida_pid > 10) O (ventilador_por_aire == True):
        activar_ventilador_extractor(True)
    // Nota: La salida_pid (0-100) podría usarse para controlar la *velocidad*
```

```

// si tuvieras un control PWM, pero para un relé, basta con encenderlo.
SINO:
  activar_ventilador_extractor(False)
// --- 4. Actualizar Pantalla ---
  oled.limpiar()
  oled.texto(f"T: {lecturas.temp_interna} C (Obj: {SETPOINT_TEMP})")
  oled.texto(f"H: {humedad_promedio} %", y=10)
  oled.texto(f"GAS: {lecturas.mq135} ppm", y=20)
  oled.mostrar()
// --- 5. Espera ---
ESPERAR 5 segundos // El control no necesita ser instantáneo
FIN BUCLE
Donde su diagrama de flujo es el siguiente

```

9.4.3 Pseudocodigo composta + plantas

```

INICIO
// --- 1. CONFIGURACIÓN ---
  // Constantes Globales
  DEFINIR Tiempo_Espera_Ciclo = 1 segundo
  DEFINIR Minutos_Simulados_Por_Ciclo = 10
  DEFINIR Umbral_Luz_Dia = 300 // Nivel LDR para que sea "de día"
  // Constantes de la Composta
  DEFINIR Composta_Umbral_Humedad = 50%
  DEFINIR Composta_Umbral_Luz = 700
  DEFINIR Composta_Umbral_Gas = 400 ppm
  DEFINIR Composta_Umbral_Temp = 50°C
  // Perfiles de las Plantas
  DEFINIR Perfil_Tomate = {hum_suelo_min: 45, hum_aire_max: 75, temp_max: 32, max_horas_luz: 10}
  DEFINIR Perfil_Helecho = {hum_suelo_min: 65, hum_aire_max: 85, temp_max: 25, max_horas_luz: 4}
  DEFINIR Perfil_Cactus = {hum_suelo_min: 15, hum_aire_max: 40, temp_max: 38, max_horas_luz: 14}
  // Selección de prueba
  DEFINIR Planta_Seleccionada = "Helecho"
  ASIGNAR Perfil_Activo = ObtenerPerfil(Planta_Seleccionada)
// --- 2. INICIALIZACIÓN DE VARIABLES DE SIMULACIÓN (ESTADO) ---
  // Estado Global (Mundo)
  ASIGNAR Hora_Actual = 6
  ASIGNAR Minuto_Actual = 0
  ASIGNAR Luminosidad_Global (LDR) = 0
  ASIGNAR Horas_Luz_Acumuladas_Planta = 0.0
  // Estado de la Composta
  ASIGNAR Compost_Temperatura = 25°C
  ASIGNAR Compost_Humedad_Suelo = 60%
  ASIGNAR Compost_Gas_Nivel = 150 ppm
  // Estado del Vivero
  ASIGNAR Planta_Temperatura = 22°C

```

```

ASIGNAR Planta_Humedad_Aire = 60%
ASIGNAR Planta_Humedad_Suelo = 50%
// --- 3. INICIALIZACIÓN DE ACTUADORES ---
ASIGNAR Estado_Bomba_Compost = APAGADO
ASIGNAR Estado_Agitador_Compost = APAGADO
ASIGNAR Estado_Ventilador_Compost = APAGADO
ASIGNAR Estado_Bomba_Planta = APAGADO
ASIGNAR Estado_Ventilador_Planta = APAGADO
ASIGNAR Estado_Cubierta_Cerrada = FALSO // (Está abierta)
// --- 4. BUCLE PRINCIPAL DE CONTROL ---
MIENTRAS (Verdadero) HACER
// --- 4A. ACTUALIZAR FÍSICA (SIMULACIÓN) ---
// Avanzar el reloj y el sol
Llamar a Función Actualizar_Simulacion_Global(Minutos_Simulados_Por_Ciclo)
// Avanzar Hora_Actual y Minuto_Actual
// Calcular Luminosidad_Global basado en la hora
// SI es de día, Incrementar Horas_Luz_Acumuladas_Planta
// SI es medianoche, Horas_Luz_Acumuladas_Planta = 0
FIN Función
// Calcular física de la composta
Llamar a Función Actualizar_Estado_Compost()
// Aumentar Compost_Temperatura (calor natural)
// SI Ventilador_Compost está ENCENDIDO, bajar Compost_Temperatura
// Reducir Compost_Humedad_Suelo (evaporación)
// SI Bomba_Compost está ENCENDIDA, subir Compost_Humedad_Suelo
// Aumentar Compost_Gas_Nivel
// SI Agitador_Compost o Ventilador_Compost están ENCENDIDOS, bajar Compost_Gas_Nivel
FIN Función
// Calcular física de las plantas
Llamar a Función Actualizar_Estado_Plantas()
// Ajustar Planta_Temperatura hacia la temperatura ambiente (basado en sol)
// SI Ventilador_Planta está ENCENDIDO, bajar Planta_Temperatura
// Reducir Planta_Humedad_Suelo (evaporación)
// SI Bomba_Planta está ENCENDIDA, subir Planta_Humedad_Suelo
// Ajustar Planta_Humedad_Aire (basado en ventilador, bomba, humedad suelo)
FIN Función
// --- 4B. LEER SENSORES (SIMULACIÓN) ---
ASIGNAR Lecturas_Compost = Leer_Sensores_Compost() // (Lee variables simuladas + ruido)
ASIGNAR Lecturas_Plantas = Leer_Sensores_Plantas() // (Lee variables simuladas + ruido)
// --- 4C. EJECUTAR LÓGICA DE CONTROL (COMPOSTA) ---
SI (Lecturas_Compost.hum_suelo < Composta_Umbral_Humedad) ENTONCES
    Estado_Bomba_Compost = ENCENDIDO
SINO
    Estado_Bomba_Compost = APAGADO
FIN SI
SI (Lecturas_Compost.gas > Composta_Umbral_Gas) ENTONCES
    Estado_Agitador_Compost = ENCENDIDO

```

```

SINO
    Estado_Agitador_Compost = APAGADO
FIN SI
SI (Lecturas_Compost.temp > Composta_Umbral_Temp) ENTONCES
    Estado_Ventilador_Compost = ENCENDIDO
SINO
    Estado_Ventilador_Compost = APAGADO
FIN SI
// Decisión de cubierta para la composta
ASIGNAR Cubierta_Requerida_Compost = (Lecturas_Compost.ldr > Composta_Umbral_Luz)
// --- 4D. EJECUTAR LÓGICA DE CONTROL (PLANTAS) ---
SI (Lecturas_Plantas.hum_suelo < Perfil_Activo.hum_suelo_min) ENTONCES
    Estado_Bomba_Planta = ENCENDIDO
SINO
    Estado_Bomba_Planta = APAGADO
FIN SI
ASIGNAR Planta_Temp_Alta = (Lecturas_Plantas.temp > Perfil_Activo.temp_max)
ASIGNAR Planta_Hum_Aire_Alta = (Lecturas_Plantas.hum_aire > Perfil_Activo.hum_aire_max)
SI (Planta_Temp_Alta O Planta_Hum_Aire_Alta) ENTONCES
    Estado_Ventilador_Planta = ENCENDIDO
SINO
    Estado_Ventilador_Planta = APAGADO
FIN SI
// Decisión de cubierta para las plantas
ASIGNAR Cubierta_Requerida_Plantas = (Horas_Luz_Acumuladas_Planta > Perfil_Activo.max_horas_luz)
// --- 4E. LÓGICA DE ACTUADOR COMPARTIDO (CUBIERTA) ---
SI (Cubierta_Requerida_Compost O Cubierta_Requerida_Plantas) ENTONCES
    Estado_Cubierta_Cerrada = VERDADERO
SINO
    Estado_Cubierta_Cerrada = FALSO
FIN SI
// --- 4F. SALIDA DE DATOS ---
// Imprimir solo la línea de números para el plotter de Thonny
IMPRIMIR (Compost_Temperatura, Compost_Humedad_Suelo, Planta_Temperatura, Planta_Humedad_Suelo, Planta_Humedad_Aire)
// --- 4G. ESPERA ---
ESPERAR (Tiempo_Espera_Ciclo)
FIN MIENTRAS
FIN

```

9.4.4 Código del dashboard web de EcoSyncra.

```

const { SerialPort, ReadlineParser } = require('serialport');
const express = require('express');
const http = require('http');
const { Server } = require("socket.io");
const app = express();
const server = http.createServer(app);
const io = new Server(server);

```

```

app.get('/', (req, res) => { res.sendFile(__dirname + '/index.html'); });
// --- CONFIGURACIÓN ---
const PUERTO_SERIE = 'COM3'; //<----- CAMBIAR AL COM QUE RECONOZCA EN TU ESP32
const BAUD_RATE = 115200;
try {
  const port = new SerialPort({ path: PUERTO_SERIE, baudRate: BAUD_RATE });
  const parser = port.pipe(new ReadlineParser({ delimiter: '\r\n' }));

  // --- MENSAJE 1: Conexión Física ---
  console.log('-----');
  console.log('🔌 PUERTO ${PUERTO_SERIE} ABIERTO CORRECTAMENTE');
  console.log('-----');

  parser.on('data', (linea) => {
    // Ver datos crudos para depurar
    // console.log(`ESP32 Raw: ${linea}`);
    if (linea.trim().startsWith('{')) {
      io.emit('data', linea);
    }
  });
  io.on('connection', (socket) => {
    // --- MENSAJE 2: Conexión Web ---
    console.log('✅ USUARIO CONECTADO AL DASHBOARD (Navegador abierto)');
    socket.on('update-settings', (data) => {
      console.log('📡 Enviando comando: ${data}');
      port.write(data + '\r\n');
    });
    socket.on('disconnect', () => {
      console.log('❌ Usuario desconectado');
    });
  });
  server.listen(3000, () => {
    // --- MENSAJE 3: Servidor Listo ---
    console.log('🚀 SERVIDOR LISTO. Da click aquí: http://localhost:3000');
    console.log('-----');
  });
} catch (err) {
  console.error('Error Fatal:', err.message);
}

```

9.4.4.1. Código main de funcionamiento del sistema

```

import machine

import time

import dht

import json

import sys

import uselect

# =====

# PINES

# =====

SOIL_PIN = 39; DHT_PIN = 21; MQ_PIN = 34; LDR_PIN = 36

SERVO_PIN = 2; EXTRACTOR_PIN = 4

```

```

STBY_PIN = 18; PWMA_PIN = 23; AIN1_PIN = 19; AIN2_PIN = 22

PWMB_PIN = 16; BIN1_PIN = 5; BIN2_PIN = 17

# =====
# CONSTANTES DE CALIBRACIÓN (¡AJUSTA ESTO!)
# =====

# IMPORTANTE: Reemplaza 77 con tu valor calibrado

SERVO_DUTY_STOP = 77

# Valores para mover (ajustar si va muy rápido)

SERVO_DUTY_OPEN = 40

SERVO_DUTY_CLOSE = 115

ROOF_MOVE_TIME = 3000 # 3 segundos

LOOP_DELAY_MS = 2000

MIX_INTERVAL = 3600 * 1000

WATER_PUMP_TIME = 3000

MIXER_TIME = 5000

# =====
# HARDWARE
# =====

class MotorDriver:

    def __init__(self, pwma, ain1, ain2, pwmb, bin1, bin2, stby):

        self.stby = machine.Pin(stby, machine.Pin.OUT); self.stby.value(1)

        self.pwma = machine.PWM(machine.Pin(pwma), freq=1000, duty=0)

        self.ain1 = machine.Pin(ain1, machine.Pin.OUT); self.ain2 = machine.Pin(ain2, machine.Pin.OUT)

        self.pwmb = machine.PWM(machine.Pin(pwmb), freq=1000, duty=0)

        self.bin1 = machine.Pin(bin1, machine.Pin.OUT); self.bin2 = machine.Pin(bin2, machine.Pin.OUT)

    def set_motor_a(self, speed):

        self.pwma.duty(abs(speed))

```

```

        if speed == 0: self.ain1.value(1); self.ain2.value(1)

        else: self.ain1.value(1); self.ain2.value(0)

    def set_motor_b(self, speed):

        self.pwmb.duty(abs(speed))

        if speed == 0: self.bin1.value(1); self.bin2.value(1)

        else: self.bin1.value(1); self.bin2.value(0)

# INICIALIZACIÓN

try:

    dht_sensor = dht.DHT11(machine.Pin(DHT_PIN))

    soil_sensor = machine.ADC(machine.Pin(SOIL_PIN)); soil_sensor atten(machine.ADC.ATTN_11DB)

    mq_sensor = machine.ADC(machine.Pin(MQ_PIN)); mq_sensor.atten(machine.ADC.ATTN_11DB)

    ldr_sensor = machine.ADC(machine.Pin(LDR_PIN)); ldr_sensor.atten(machine.ADC.ATTN_11DB)

    # Servo inicializado YA frenado

    servo = machine.PWM(machine.Pin(SERVO_PIN), freq=50, duty=SERVO_DUTY_STOP)

    extractor = machine.Pin(EXTRACTOR_PIN, machine.Pin.OUT); extractor.value(0)

    driver = MotorDriver(PWMA_PIN, AIN1_PIN, AIN2_PIN, PWMB_PIN, BIN1_PIN, BIN2_PIN, STBY_PIN)

except Exception as e:

    print(f"Error HW: {e}")

def read_sensors():

    try:  dht_sensor.measure();  return  (dht_sensor.temperature(),  dht_sensor.humidity(),  soil_sensor.read(),  mq_sensor.read(),
ldr_sensor.read())

    except: return None

def move_servo_duty(duty_val): servo.duty(duty_val)

# =====

```

```

# LÓGICA PRINCIPAL

# =====

poller = select.poll(); poller.register(sys.stdin, select.POLLIN)

# Perfiles y estados iniciales

current_settings = { "nombre": "Esperando...", "humedad_suelo_min": 4096, "luz_min": 0, "temp_max": 100, "co2_max": 4096 }

pump_active = False; pump_start_time = 0

roof_is_open = False; roof_moving = False; roof_start_time = 0

mixer_active_timer = False; mixer_start_time = 0; mixer_active_co2 = False

def main_loop():

    global current_settings, pump_active, pump_start_time, roof_is_open, roof_moving, roof_start_time

    global mixer_active_timer, mixer_start_time, mixer_active_co2

    last_sensor_time = 0

    last_mix_check = time.time()

    # Asegurar freno al inicio del bucle

    move_servo_duty(SERVO_DUTY_STOP)

    while True:

        current_ms = time.time()

        # 1. COMANDOS (Entrada Serial)

        if poller.poll(0):

            try:

                line = sys.stdin.readline().strip()

                if line:

                    cmd = json.loads(line)

```

```
if "settings" in cmd:

    current_settings = cmd["settings"]

    print(json.dumps({"status": "OK", "msg": f"Perfil: {current_settings['nombre']}"}))

    if current_settings['nombre'] == "Riego OFF":

        pump_active = False; driver.set_motor_a(0)

except: pass
```

2. SENSORES Y CONTROL DE ACTUADORES

```
if time.time_diff(current_ms, last_sensor_time) > LOOP_DELAY_MS:
```

```
    data = read_sensors()
```

```
    if data:
```

```
        t, h, s, c, l = data
```

```
    # --- ANALISIS DE CAUSAS (DEBUG) ---
```

```
    motivo_ext = "OFF"
```

```
    motivo_mix = "OFF"
```

```
    co2_alto = c > current_settings['co2_max']
```

```
    temp_alta = t > current_settings['temp_max']
```

```
    # --- LÓGICA EXTRACTOR (CORREGIDA) ---
```

```
    if co2_alto:
```

```
        extractor.value(1)
```

```
        motivo_ext = "CO2 ALTO"
```

```
    elif temp_alta:
```

```
        extractor.value(1)
```

```
        motivo_ext = "TEMP ALTA"
```

```
    else:
```

```
        extractor.value(0) # Solo se apaga si NI CO2 NI TEMP son altos
```

```

# --- LÓGICA MEZCLADOR ---

if co2_alto:

    if not mixer_active_co2:

        driver.set_motor_b(600); mixer_active_co2 = True

        motivo_mix = "CO2 ALTO"

    else:

        if mixer_active_co2:

            driver.set_motor_b(0); mixer_active_co2 = False


# --- LÓGICA RIEGO ---

if s > current_settings['humedad_suelo_min'] and not pump_active:

    pump_active = True; pump_start_time = current_ms

    driver.set_motor_a(800)


# --- LÓGICA TECHO ---

if not roof_moving:

    if l > current_settings['luz_min'] and not roof_is_open:

        roof_moving = True; roof_start_time = current_ms

        move_servo_duty(SERVO_DUTY_OPEN); roof_is_open = True

    elif l < current_settings['luz_min'] and roof_is_open:

        roof_moving = True; roof_start_time = current_ms

        move_servo_duty(SERVO_DUTY_CLOSE); roof_is_open = False


# --- REPORTE JSON COMPLETO (FINAL) ---

estado_mezclador = 0

if mixer_active_timer or mixer_active_co2:

    estado_mezclador = 1

msg = {

```

```

"tipo": "sensores",

"temp": t, "humidity": h, "soil": s, "co2": c, "light": l,

"perfil_actual": current_settings['nombre'],

# ¡ESTADOS DE ACTUADORES!

"estados": {

    "bomba": 1 if pump_active else 0,

    "techo": "ABIERTO" if roof_is_open else "CERRADO",

    "extractor": extractor.value(),

    "mezclador": estado_mezclador

},

"debug": {

    "causa_ext": motivo_ext,

    "causa_mix": motivo_mix

}

}

print(json.dumps(msg))

last_sensor_time = current_ms

# 3. TIMERS

if pump_active and time.time_diff(current_ms, pump_start_time) > WATER_PUMP_TIME:

    pump_active = False; driver.set_motor_a(0)

if roof_moving and time.time_diff(current_ms, roof_start_time) > ROOF_MOVE_TIME:

    roof_moving = False; move_servo_duty(SERVO_DUTY_STOP) # Freno Calibrado

# Temporizador para Mezcla Periódica (si no está activa por CO2)

if not mixer_active_co2:

    if time.time_diff(current_ms, last_mix_check) > MIX_INTERVAL:

```

```

        mixer_active_timer = True; mixer_start_time = current_ms

        driver.set_motor_b(600); last_mix_check = current_ms

    if mixer_active_timer and time.ticks_diff(current_ms, mixer_start_time) > MIXER_TIME:

        mixer_active_timer = False; driver.set_motor_b(0)

    time.sleep_ms(10)

if __name__ == "__main__":

    time.sleep(1)

    main_loop()

```

9.4.5. Código de la interfaz web

```

!DOCTYPE html>

<html lang="es">

<head>

    <meta charset="UTF-8">

    <title>Ecosyncra Dashboard</title>

    <script src="https://cdn.rawgit.com/Mikhus/canvas-gauges/gh-pages/download/2.1.7/radial/gauge.min.js"></script>

    <style>

        body { font-family: 'Segoe UI', sans-serif; background: #f4f7f6; color: #333; padding: 20px; text-align: center; }

        h1 { color: #2c5d43; }

        .container { background: #fff; border-radius: 15px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); margin: 20px auto; padding: 20px; max-width: 1000px; }

        /* Panel de Control */

        #control-panel { border-left: 6px solid #2c5d43; text-align: left; max-width: 800px; }

        select, input { font-size: 1rem; padding: 10px; width: 100%; margin-bottom: 10px; border: 1px solid #ccc; border-radius: 5px; box-sizing: border-box; }

        button { width: 100%; padding: 12px; background-color: #2c5d43; color: white; font-weight: bold; cursor: pointer; border: none; border-radius: 8px; }

        button:hover { background-color: #3e805d; }

        #custom-inputs { display: none; background: #f8f9fa; padding: 15px; border-radius: 8px; border: 1px dashed #ccc; margin-bottom: 10px; }

        /* ESTILOS DE ESTADO (BADGES) - ¡RECUPERADOS! */

        .status-row { display: flex; justify-content: space-around; margin-top: 20px; padding-top: 20px; border-top: 1px solid #eee; }

```

```

.status-item { font-size: 1.1em; font-weight: bold; }

.badge {

    display: inline-block; padding: 5px 12px; border-radius: 20px;

    color: white; font-size: 0.8em; text-transform: uppercase; min-width: 40px; text-align: center;

}

.on { background-color: #28a745; box-shadow: 0 0 10px #28a74580; } /* Verde brillante */

.off { background-color: #6c757d; } /* Gris */

.open { background-color: #17a2b8; } /* Azul para techo */

.closed { background-color: #6c757d; }

pre { background: #222; color: #0f0; padding: 15px; border-radius: 5px; text-align: left; overflow-x: auto; font-family: monospace; max-width: 800px; margin: 0 auto;}

#gauges-container { display: flex; flex-wrap: wrap; justify-content: center; gap: 20px; }

</style>

</head>

<body>

<h1>🌱 Hydrosys Control Center</h1>

<div id="control-panel" class="container">

    <h2>⚙️ Panel de Control</h2>

    <label>Perfil de Cultivo:</label>

    <select id="profile-select" onchange="toggleCustom()">

        <option value="tomate">Tomate (Estándar)</option>

        <option value="menta">Menta (Húmedo)</option>

        <option value="cactus">Cactus (Seco)</option>

        <option value="custom">🎨 Personalizar...</option>

        <option value="manual_off">🛑 APAGAR TODO</option>

    </select>

    <div id="custom-inputs">

        <input type="text" id="cust-name" placeholder="Nombre (Ej. Lechuga)">

        <input type="number" id="cust-soil" placeholder="Suelo Max (3000)">

        <input type="number" id="cust-light" placeholder="Luz Min (2000)">

    </div>

    </div>

</body>

```

```

<input type="number" id="cust-co2" placeholder="CO2 Max (300)">

<input type="number" id="cust-temp" placeholder="Temp Max (28)">

</div>

<button id="save-button"> 📤 ENVIAR COMANDO</button>

<div style="margin-top:15px;">

  Perfil Activo: <strong id="current-profile-display" style="color:#2c5d43">---</strong>

</div>

<div class="status-row">

  <div class="status-item"> 💧 Bomba <span id="badge-pump" class="badge off">OFF</span></div>

  <div class="status-item"> 🏠 Techo <span id="badge-roof" class="badge closed">CERRADO</span></div>

  <div class="status-item"> 🌀 Extractor <span id="badge-ext" class="badge off">OFF</span></div>

  <div class="status-item"> 🍷 Mezclador <span id="badge-mix" class="badge off">OFF</span></div>

</div>

<div style="font-size: 0.8em; color: #666; margin-top: 5px;">

  Razón Extractor: <span id="debug-ext">---</span>

</div>

</div>

<div class="container">

  <div id="gauges-container">

    <canvas id="gauge-temp"></canvas>

    <canvas id="gauge-hum"></canvas>

    <canvas id="gauge-soil"></canvas>

    <canvas id="gauge-light"></canvas>

    <canvas id="gauge-co2"></canvas>

  </div>

</div>

<div class="container">

  <h3>Terminal de Datos</h3>

  <pre id="data-display">Esperando conexión...</pre>

</div>

```

```
<script src="/socket.io/socket.io.js"></script>
```

```
<script>
```

```
// Configuración
```

```
function mapRange(x, in_min, in_max, out_min, out_max) {
```

```
    let result = (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
```

```
    return Math.min(Math.max(result, out_min), out_max);
```

```
}
```

```
const SOIL_MIN=4095, SOIL_MAX=1900, LIGHT_MIN=4095, LIGHT_MAX=130;
```

```
const perfiles = {
```

```
    "tomate": { "nombre": "Tomate", "humedad_suelo_min": 3000, "luz_min": 2000, "temp_max": 28, "co2_max": 200 },
```

```
    "menta": { "nombre": "Menta", "humedad_suelo_min": 2200, "luz_min": 1000, "temp_max": 25, "co2_max": 250 },
```

```
    "cactus": { "nombre": "Cactus", "humedad_suelo_min": 4000, "luz_min": 3000, "temp_max": 35, "co2_max": 400 },
```

```
    "manual_off": { "nombre": "Riego OFF", "humedad_suelo_min": 4096, "luz_min": 0, "temp_max": 100, "co2_max": 4096 }
```

```
};
```

```
function toggleCustom() {
```

```
    document.getElementById('custom-inputs').style.display =
```

```
        (document.getElementById('profile-select').value === 'custom') ? 'block' : 'none';
```

```
}
```

```
// Gauges
```

```
var gTemp = new RadialGauge({ renderTo: 'gauge-temp', title: "Temp", units: "°C", width: 160, height: 160, minValue: 0, maxValue: 50 }).draw();
```

```
var gHum = new RadialGauge({ renderTo: 'gauge-hum', title: "H. Amb", units: "%", width: 160, height: 160, minValue: 0, maxValue: 100 }).draw();
```

```
var gSoil = new RadialGauge({ renderTo: 'gauge-soil', title: "H. Suelo", units: "%", width: 160, height: 160, minValue: 0, maxValue: 100, highlights: [{"from": 0,"to": 30,"color":"red"}, {"from": 60,"to": 100,"color":"green"}] }).draw();
```

```
var gLight = new RadialGauge({ renderTo: 'gauge-light', title: "Luz", units: "%", width: 160, height: 160, minValue: 0, maxValue: 100, highlights: [{"from": 80,"to": 100,"color":"yellow"}] }).draw();
```

```
var gCO2 = new RadialGauge({ renderTo: 'gauge-co2', title: "CO2", units: "PPM", width: 160, height: 160, minValue: 0, maxValue: 500, highlights: [{"from": 0,"to": 200,"color":"green"}, {"from": 300,"to": 500,"color":"red"}] }).draw();
```

```
const socket = io();
```

```
const el = {
```

```
    json: document.getElementById('data-display'),
```

```
    profile: document.getElementById('current-profile-display'),
```

```
    pump: document.getElementById('badge-pump'),
```

```

    roof: document.getElementById('badge-roof'),

    ext: document.getElementById('badge-ext'),

    mix: document.getElementById('badge-mix'),

    debugExt: document.getElementById('debug-ext')

};

function updateBadge(element, isActive, textOn="ON", textOff="OFF") {

    if (isActive) {

        element.className = "badge on";

        element.innerText = textOn;

    } else {

        element.className = "badge off";

        element.innerText = textOff;

    }

}

function updateRoofBadge(element, stateText) {

    if (stateText === "ABIERTO") {

        element.className = "badge open";

        element.innerText = "ABIERTO";

    } else {

        element.className = "badge closed";

        element.innerText = "CERRADO";

    }

}

socket.on('data', (jsonString) => {

    try {

        const data = JSON.parse(jsonString);

        el.json.textContent = JSON.stringify(data, null, 2);

        if (data.tipo === "sensores") {

            // Actualizar Gauges

            if(data.temp) gTemp.value = data.temp;

```

```

if(data.humidity) gHum.value = data.humidity;

if(data.soil) gSoil.value = mapRange(data.soil, SOIL_MIN, SOIL_MAX, 0, 100);

if(data.light) gLight.value = mapRange(data.light, LIGHT_MIN, LIGHT_MAX, 0, 100);

if(data.co2) gCO2.value = data.co2;

if(data.perfil_actual) el.profile.textContent = data.perfil_actual;

// --- ACTUALIZAR BADGES DE ESTADO ---

if (data.estados) {

    updateBadge(el.pump, data.estados.bomba === 1);

    updateBadge(el.ext, data.estados.extractor === 1);

    updateBadge(el.mix, data.estados.mezclador === 1);

    updateRoofBadge(el.roof, data.estados.techo);

}

if (data.debug) {

    el.debugExt.innerText = data.debug.causa_ext || "---";

}

}

if (data.status === "OK") console.log("ESP32:", data.msg);

} catch (e) { console.error(e); }

});

document.getElementById('save-button').addEventListener('click', () => {

    const sel = document.getElementById('profile-select').value;

    let settings = (sel === 'custom') ? {

        "nombre": document.getElementById('cust-name').value || "Custom",

        "humedad_suelo_min": parseInt(document.getElementById('cust-soil').value),

        "luz_min": parseInt(document.getElementById('cust-light').value),

        "temp_max": parseInt(document.getElementById('cust-temp').value),

        "co2_max": parseInt(document.getElementById('cust-co2').value)

    } : perfiles[sel];

    socket.emit('update-settings', JSON.stringify({ "settings": settings }));

    alert("Enviando...");

```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```